Google certifications - Associate Cloud Engineer

Program details:

https://services.google.com/fh/files/misc/associate_cloud_engineer_exam_guide_english.pdf

Courses:

Cloud infrastructure: Foundation

- Introduction
- Virtual Network
- Virtual Machines

Cloud Infrastructure: Core Services

- IAM
- DATA STORAGE
- Resource manager
- Resource Monitoring

Cloud Infrastructure: Scaling and Automation

- Interconnecting Networks
- Load Balancing and autoscaling
- Infrastructure automation
- Managed Services

Metadata

Cohort ID: ACE02-GETCERT/2024.11.26

From C:\google_certification\EXT_ACE02-GETCERT-2024.11.26\RESOURCES\1-ACE02-GETCERT_2024.11.26_KICKOFF Deck



Earn 6 badgsans prepare for the exam:

Program elements Exam voucher information Get your exam voucher by filling this form. Please allow 24-72 hours to receive the voucher (from the request date)

. Please attempt the exam by 24th Feb, 2025. Technical mentorship provided by a dedicated trainer in a format of: ●

Need to complete 6 out of 6 LABS – those with stars * before Feb/6/2025

Test should be 24th Feb, 2025

Support

Qwiklabs support can be reached via: a. Email: support@qwiklab.com b. Qwiklabs Support Portal c. Chat support 2. While contacting Qwiklabs support please provide the following: a. Your login email address b. Description of an issue to be solved c. Screenshots of the issue (if needed)

For peer to peer support and a direct communication channel with your trainer. ace02-getcert_q4_2024@ googlegroups.com

Other mail

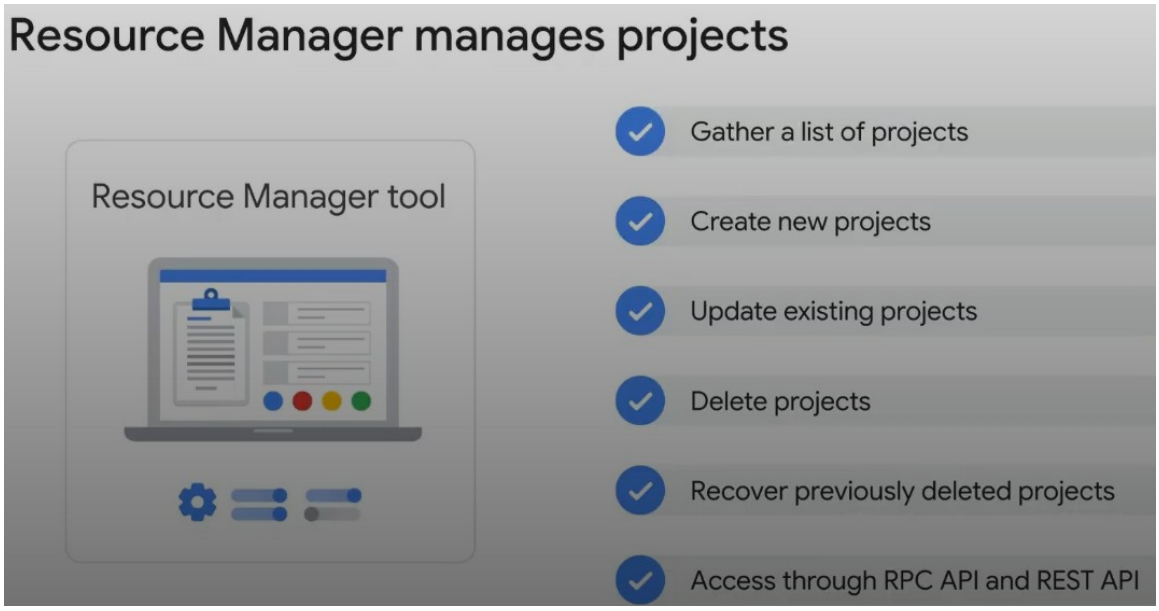
Also bhalamish20@gmail.com

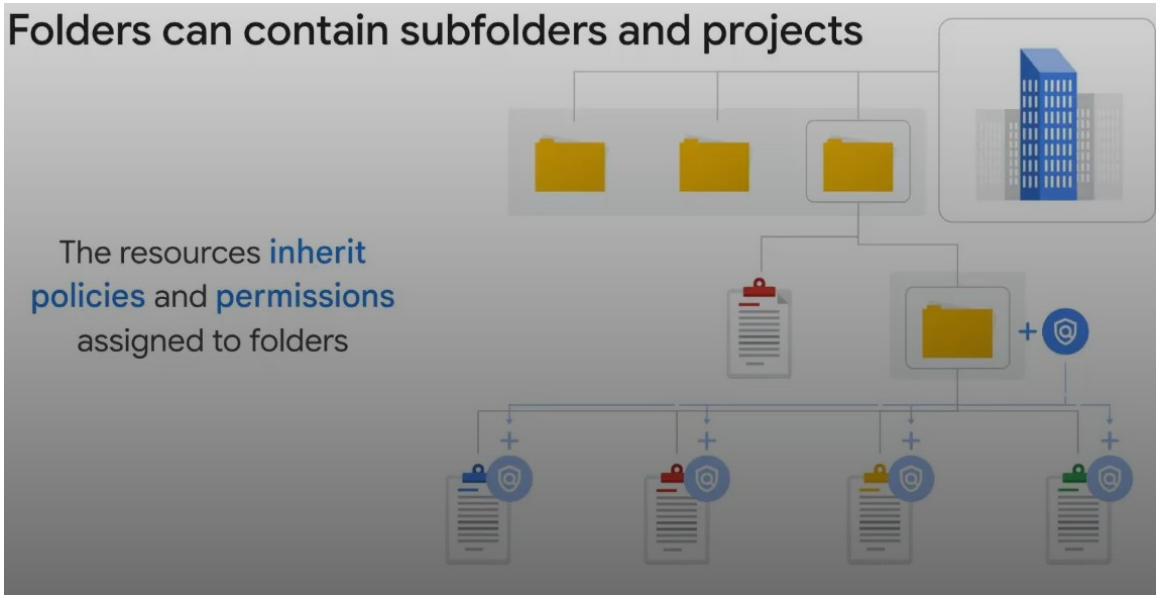
A Google Cloud project is an organizing entity for your Google Cloud resources. (like resource group?)

A project associates objects and services with billing.

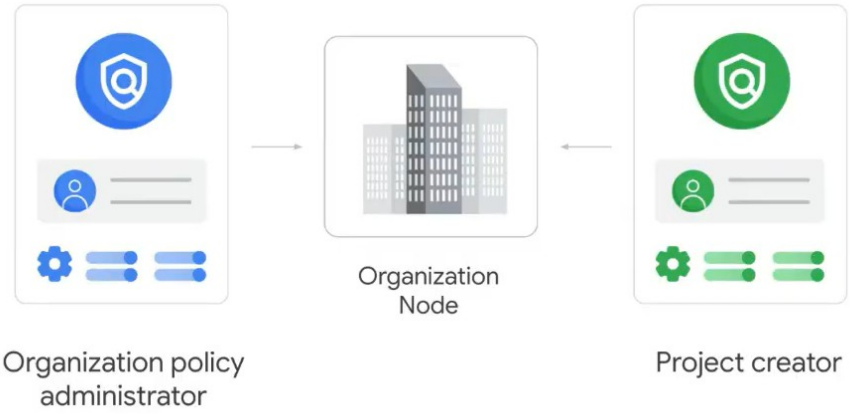
like azure creating credits, + starting module

Cymbal superstore =~ eshopOnWe (Azure)





Special roles are associated with the Organization Node



Good advice to create project create role

Apendix A

Firestore, or **Cloud Firestore**, is a scalable and fully managed NoSQL document

Google Cloud DNS is a scalable, reliable, and managed Domain Name System (DNS) service that runs on the same infrastructure as Google.

Google Cloud Bigtable is a fully managed, scalable NoSQL database service designed for large analytical and operational workloads. It offers low-latency data access and high throughput, making it ideal for applications such as real-time analytics, machine learning, and time-series data processing.

egress refers to **outbound traffic** leaving a network, system, or application and heading toward an external network or destination

RPC (Remote Procedure Call) API is a type of API that allows one system to execute a function or procedure in another system over a network as if it were a local function.

Google Cloud Run is a fully managed platform that allows you to deploy and run containerized applications. It abstracts away infrastructure management, enabling developers to focus solely on code. Cloud Run is designed to handle stateless applications and automatically scales up or down based on traffic, ensuring efficiency and cost-effectiveness.

Cloud Run, a managed compute platform that lets you run stateless containers via web requests or Pub/Sub events.

The **Cloud Foundation Toolkit (CFT)** is a collection of open-source **Infrastructure as Code (IaC)** templates and examples provided by Google Cloud. These templates enable developers and IT administrators to deploy Google Cloud resources consistently and efficiently using industry best practices. The toolkit supports popular IaC frameworks like **Terraform** and **Deployment Manager**.

Google Cloud Spanner is a fully managed, scalable, and globally distributed relational database service designed for mission-critical applications. It combines the benefits of traditional relational databases with the horizontal scalability of NoSQL databases, offering strong transactional consistency and high availability.

TensorFlow is an open-source machine learning framework developed by Google. It is designed to build, train, and deploy machine learning and deep learning models. TensorFlow supports a wide range of tasks, from numerical computation to large-scale machine learning workflows, and is widely used for research and production environments.

Apache Spark is an open-source distributed computing system designed for processing large-scale data.

Google Cloud Dataproc is a fully managed cloud service for running **Apache Spark**, **Apache Hadoop**, and other open-source big data frameworks on Google Cloud infrastructure. It simplifies the creation, management, and scaling of clusters for big data processing, enabling users to process and analyze large datasets efficiently. (The equivalent of the big data system Hadoop, but operating as a service),

Appendix-B Good to know

Security over Hardware

server boards and the networking equipment in Google data centers are custom-designed by Google

Google also designs custom chips, including a hardware security chip that's currently being deployed on both servers and peripherals.

Google server machines use a variety of technologies to ensure that they are booting the correct

software stack, such as cryptographic signatures over the BIOS, bootloader, kernel, and base operating system image.

Service deployment layer, where the key feature is encryption of inter-service communication

Google services that are being made available on the internet, register themselves with an infrastructure service called the Google Front End, which ensures that all TLS connections

Are ended using a public-private key pair and an X.509 certificate from a Certified Authority (CA), as well as following best practices such as supporting perfect forward secrecy.

The GFE additionally applies protections against Denial of Service attacks.

The final layer is Google's Operational security layer which provides four key features.

Intrusion detection (זיהוח חדירה) – alerts

Reducing insider risk

Google aggressively limits and actively monitors the activities of employees who have been granted administrative access to the infrastructure.

Employee U2F use.

To guard against phishing attacks against Google employees, employee accounts require use of U2F-compatible Security Key

Stringent software development practices

requires two-party review of new code.

Libraries that prevent them from introducing certain classes of security bugs.

Vulnerability Rewards Program where we pay anyone who is able to discover and inform us of bugs in our infrastructure or applications

TensorFlow, an open source software library for machine learning developed inside Google, is at the heart of a strong open source ecosystem.

IaaS Billing: when you run an instance for more than 25% of a month, Compute Engine automatically gives you a discount for every incremental minute you use for that instance

Billing budget: budget can be a fixed limit, or it can be tied to another metric; for example, a percentage of the previous month's spend. (Alerts are generally set at 50%, 90% and 100%, but can also be customized)

Billing quotas: Google Cloud also implements quotas, which are designed to prevent the over-consumption of resources because of error or a malicious attack, protecting both account owners and the Google Cloud community as a whole.

project level quotas:

rate quotas: example: by default, the GKE service implements a quota of 3,000 calls to its API from each Google Cloud project every 100 seconds, After that 100 seconds, the limit is reset.

allocation quotas: govern the number of resources you can have in your projects. For example, by default, each Google Cloud project has a quota allowing it no more than 15 Virtual Private Cloud networks.

Firewall policies

Link: https://cloud.google.com/firewall/docs/firewall-policies-overview

Hierarchical firewall policies

Global network firewall policies

Regional network firewall policies

Policy and rule evaluation order

Security - Hierarchical firewall policies

Policies based on hierarchy – from organization to folders to projects

You need to attach it to resource

Hierarchical firewall policy rules can be used to configure Layer 7 inspection of the matched traffic, such as intrusion prevention service.

Configure intrusion prevention service

intrusion prevention service without TLS – Layer 7 inspection - 5 steps

1. Create a security profile of type Threat prevention here
2. Create a security profile group with the security profile here
3. Create a firewall endpoint in the same zone as your workloads where you want to enable threat prevention. See Create a firewall endpoint.
4. Associate the firewall endpoint with one or more VPC networks. Make sure that you're running your workloads in the same zone as the firewall endpoint. see Create firewall endpoint associations.
5. add a firewall policy rule with a security profile group configured. here

intrusion prevention service with TLS – Layer 7 inspection - 6 steps

1. Create a security profile of type Threat prevention here
2. Create a security profile group with the security profile here
3. Create a CA pool and a trust config, and add them to your TLS inspection policy. To learn how to enable TLS inspection in Cloud NGFW, see Set up TLS inspection.
4. Create a firewall endpoint in the same zone as your workloads where you want to enable threat prevention. See Create a firewall endpoint.
5. Associate the firewall endpoint with one or more VPC networks. Make sure that you're running your workloads in the same zone as the firewall endpoint. see Create firewall endpoint associations.
6. add a firewall policy rule with a security profile group configured. here

Appendix C – Quiz – good questions

Cloud Computing

What type of cloud computing service lets you bind your application code to libraries that Cloud Computinggive access to the infrastructure your application needs?

- checkPlatform as a service (correct)
- Hybrid cloud
- Infrastructure as a service
- Software as a service
- Virtualized data centers

Connecting network to google VPC



https://www.cloudskillsboost.google/course_templates/60/video/513912

core storage services options:

cloud storage - like Azure

url: https://www.cloudskillsboost.google/course_templates/60/video/513916

PROJECT < BUCKET < OBJECT)BLOB + METADAT(

Example:
gs://cloud-training/gcpfci/my-excellent-blog.png

BUCKET: cloud-training

FOLDER: gcpfci

OBJECT: my-excellent-blog.png

Create storage using gc

export LOCATION=US

student_02_6f9042fbd6bc@cloudshell:~ (qwiklabs-gcp-02-813805e53a70)$ gcloud storage buckets create -l $LOCATION gs://$DEVSHELL_PROJECT_ID

Set role read to all:

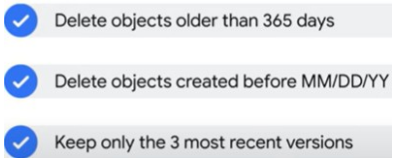
gsutil acl ch -u allUsers:R gs://$DEVSHELL_PROJECT_ID/my-excellent-blog.png

Authorization: IAM role + ACLs (Access Control List)

ACL: Scope - who can access and perform an action

Action: READ / WRITE

lifecycle policy – similar to retention policy



Object storage is a computer data storage architecture that manages data as "objects" and not as a file and folder hierarchy (file storage), or as chunks of a disk (block storage).

stored in a packaged format which contains

the binary form of the actual data itself,

relevant associated meta-data (such as date created, author, resource type, and permissions), and a globally unique identifier.

Examples include

serving website content

storing data for archival and disaster recovery

distributing large data objects to end users via Direct Download.

They all packed in buckets

Immutable = do not edit them, a new version is created with every change made.

Administrators have the option to

1. allow each new version to completely overwrite the older one
2. keep track of each change made to a particular object by enabling “versioning” within a bucket.

Permissions:

Using IAM roles (Roles are inherited from project to bucket to object.)

access control lists (ACLs)

1. a scope, which defines who can access and perform an action. This can be a specific user or group of users.
2. permission, which defines what actions can be performed, like read or write.

Maintenance Cost: storing and retrieving large amounts of object data can quickly become expensive, Cloud Storage also offers lifecycle management policies.

Types

Standard Storage: considered best for

frequently accessed,

“hot,” data

data that’s stored for only brief periods of time.

Nearline Storage: considered best for

storing infrequently accessed data,

like reading or modifying data on average once a month or less.

Examples data backups, long-tail multimedia content, or data archiving.

Coldline Storage:

low-cost option for storing infrequently accessed data

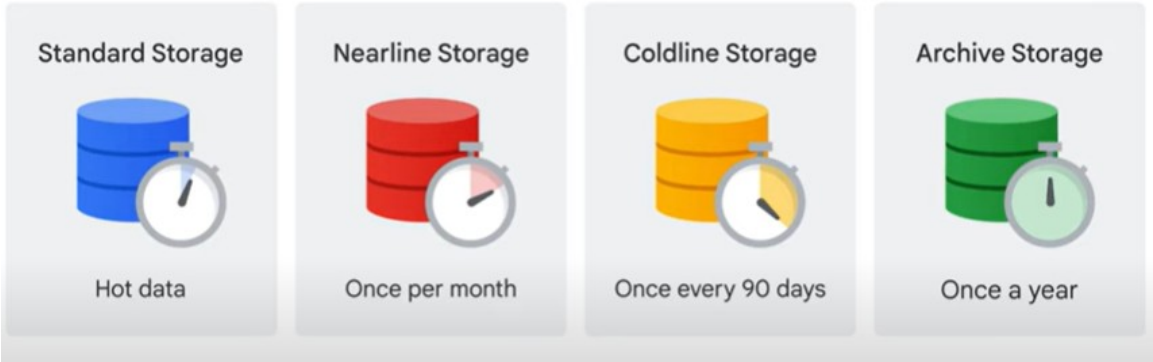
meant for reading or modifying data, at most, once every 90 days

Archive Storage:

lowest-cost option,

used ideally for data archiving, online backup, and disaster recovery.

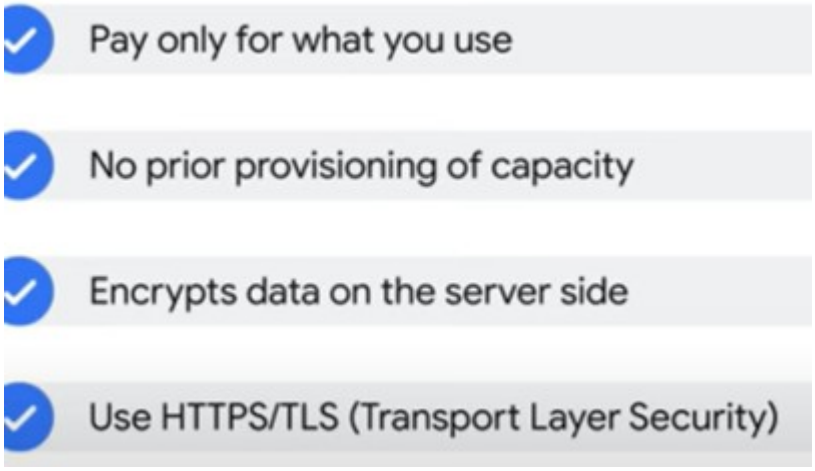
data that you plan to access less than once a year,



	Standard	Neraline	Coldline	Archive
Data	Hot frequently accessed	Once a month	Every 90 days	Once a year
Stored data	brief periods of time			
Examples		1. data backups 2. long-tail multimedia content 3. data archiving		for data archiving, online backup, and disaster recovery
Cost			low-cost	lowest-cost







Storage Transfer:

- using gcloud command (gcp cli)
- Chrome - > drag and drop option in the Cloud Console
- Storage Transfer Service:

  schedule and manage batch transfers to Cloud Storage

  from another cloud provider, a different Cloud Storage region, or from an HTTP(S) endpoint.

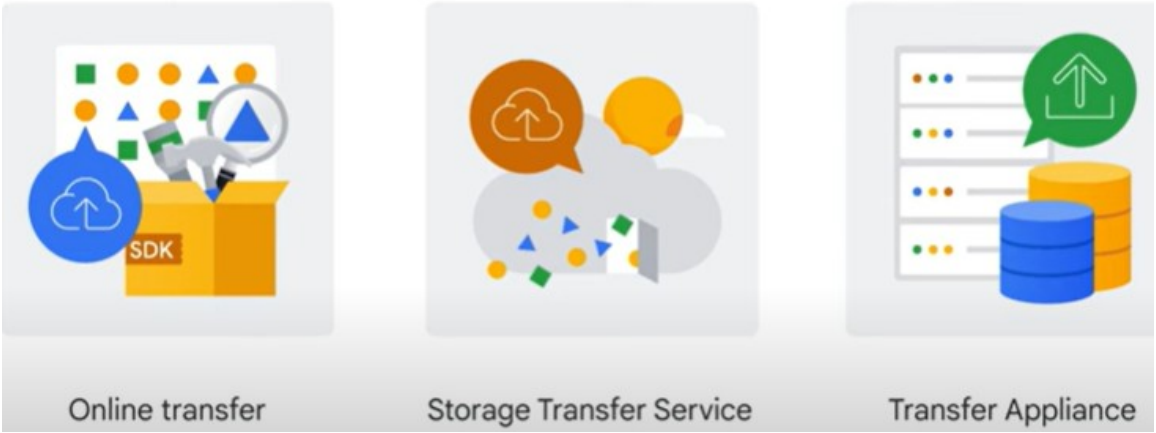
- Transfer Appliance:

  rackable, high-capacity storage server that you lease from Google Cloud

  connect it to your network,

  load it with data,

  ship it to an upload facility where the data is uploaded to Cloud Storage.

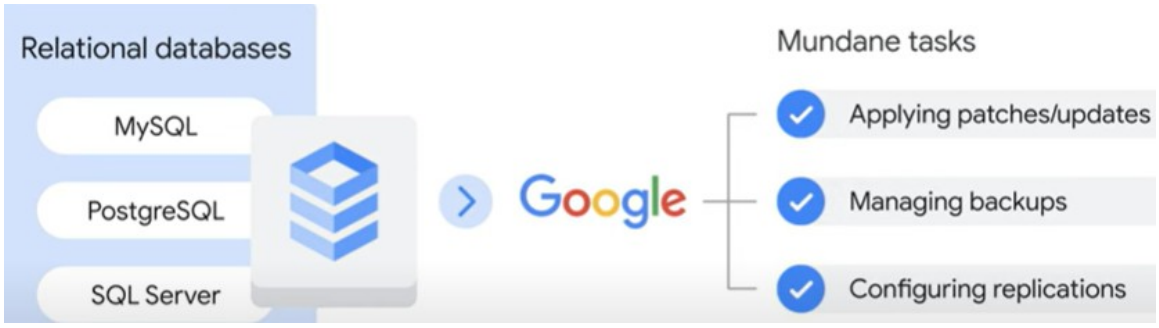


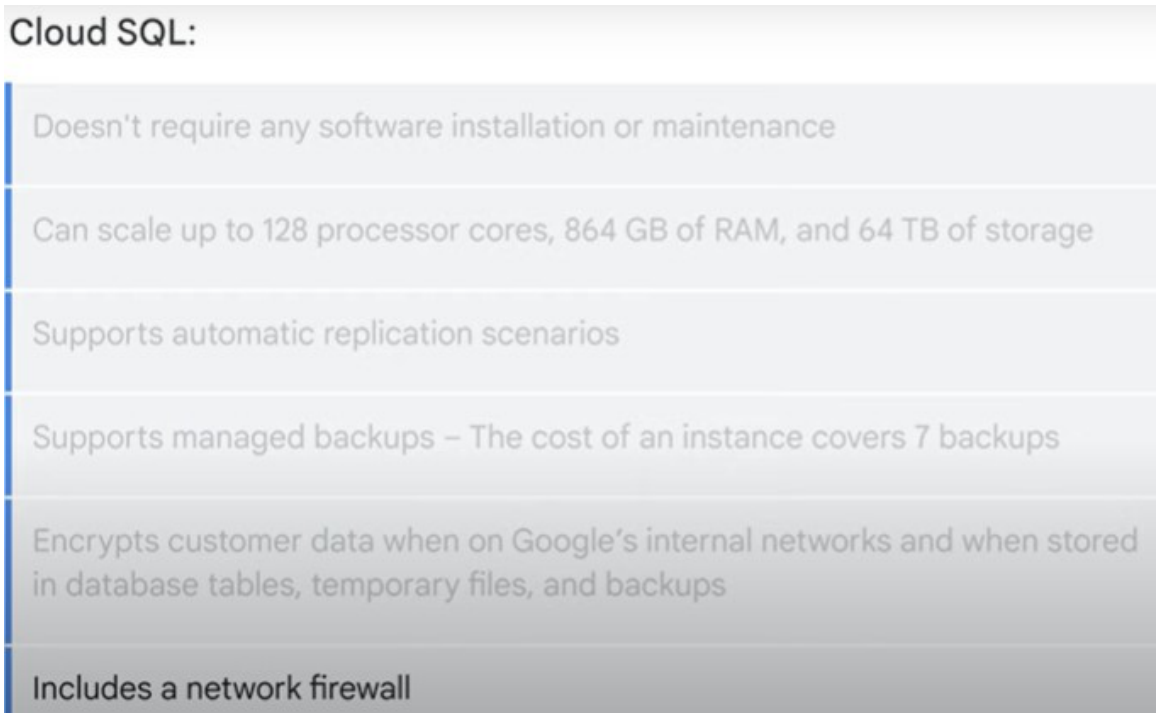
Additional storage transfer loading:

import and export tables to and from both BigQuery and Cloud SQL.

Cloud SQL - fully managed relational databases

, including MySQL, PostgreSQL, and SQL Server as a service





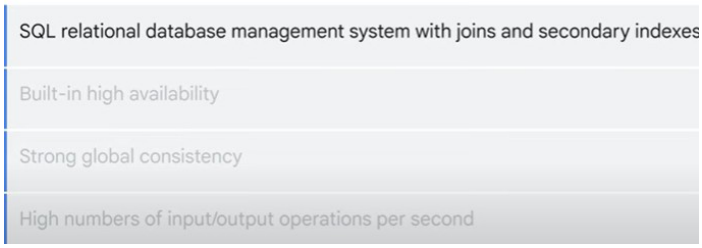
From LAB:

Spanner – fully managed relational DB



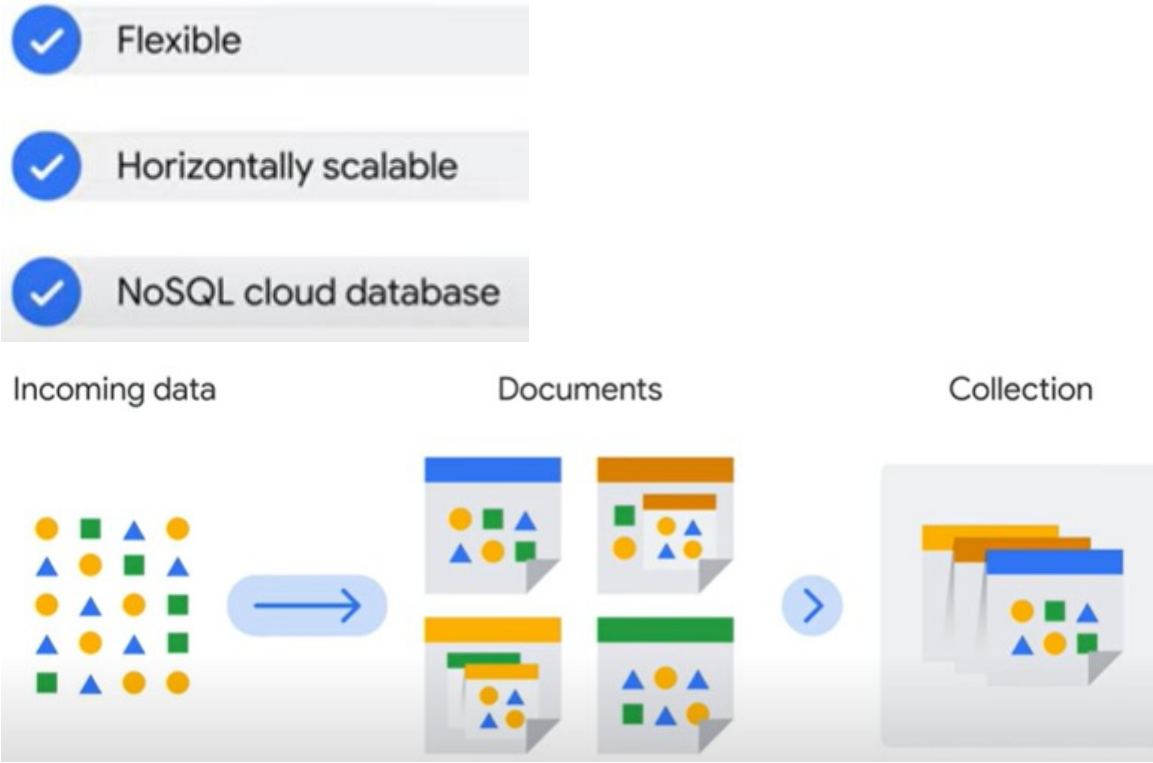
Fully managed relational database

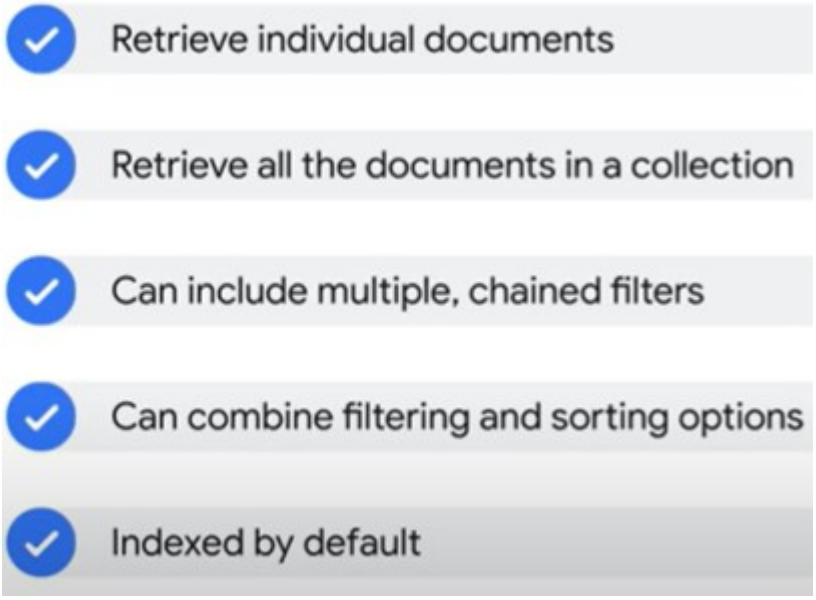
Spanner is especially suited for applications that require:



Firestore – NoSQL database - like mongodb / elastic: documents -> collection

Option	Best for	Capacity
Cloud Storage	Storing immutable blobs larger than 10 MB	Petabytes Max unit size: 5 TB per object
Cloud SQL	• Full SQL support for an online transaction processing system • Web frameworks and existing applications	Up to 64 TB
Spanner	• Full SQL support for an online transaction processing system • Horizontal scalability	Petabytes
Firestore	Massive scaling and predictability together with real time query results and offline query support	Terabytes Max unit size: 1 MB per entity
Bigtable	• Storing large amount of structured objects • Does not support SQL queries and multi-row transactions • Analytical data with heavy read and write events	Petabytes Max unit size: 10 MB p/cell, 100 MB p/row



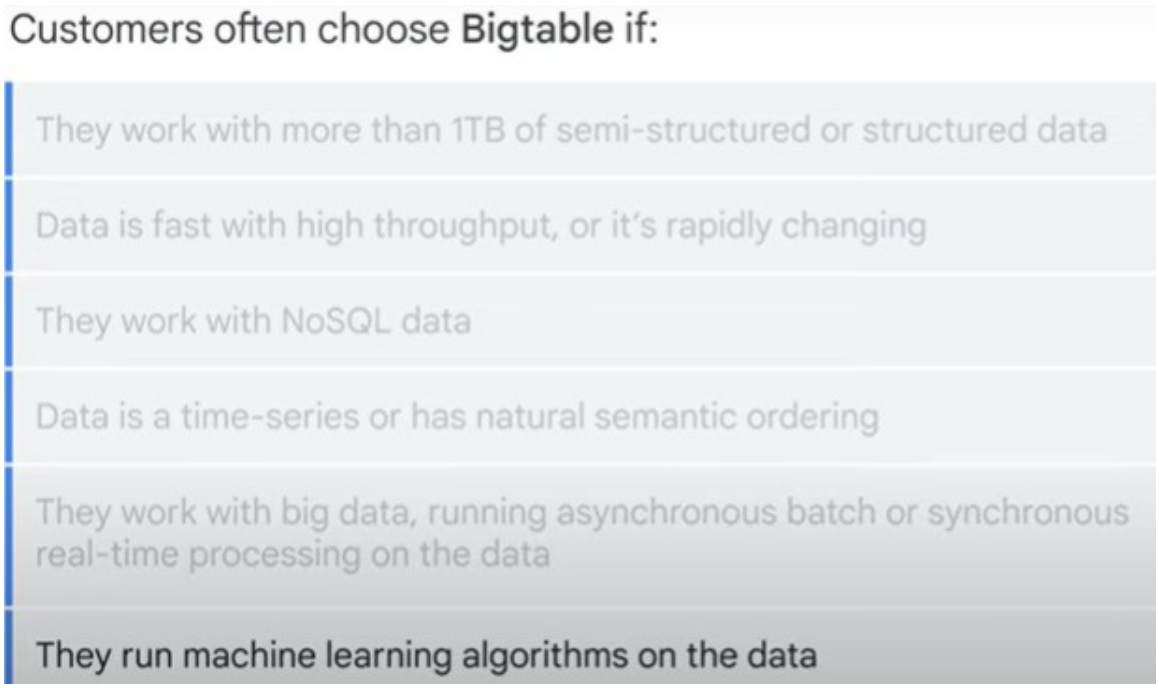


Query performance is based on results set and not data sets

It caches data – so can work in offline



BigTable – NoSQL big data database service.



Can be

Using APIs, call REST APIs

stream data

Bacth process (like Hadoop, data flow, spark etc)

K8s – GKE

Create k1 cluster:

gcloud container clusters create k1

Developing applications in the Cloud

Cloud Run – serverless, stateless containers via web requests or Pub/Sub events

managed compute platform that runs stateless containers via web requests or Pub/Sub events.

built on Knative, an open API and runtime environment built on Kubernetes.

Take source -> build image -> deploy as pod + service

Buildpacks – like CI to build an image

Pricing module: while a container is handling web requests, with a granularity of 100ms, and when it's starting or shutting down

use Cloud Run to run any binary, as long as it's compiled for Linux sixty-four bit.

As long asthey server http request (include COBOL)

Development - Cloud Run functions, event-driven app

Deploy image to CloudRun

https://www.cloudskillsboost.google/course_templates/60/labs/513931

Prompt Engineering - Generative AI and LLM (large language models)

Generative AI –

- encompasses a broader range of models capable of generating various types of content beyond just text
- capable of creating text, images, or other data using generative models, often in response to prompts (specific instruction, question, or cue given to a computer).

LLM - huge object model containing a massive dataset of text.

LLM - specifically refers to a subset of generative AI models focusing on language tasks.

- Large = The size of the training dataset, up to petabyte scale.
- Parameters = the memories and knowledge that the machine has learned during model training.
- General-purpose means that the models can sufficiently solve common problems.

LLMs are pre-trained and fine-tuned, means…

- …that they have been pre-trained for a general purpose with a large dataset…
- ...and then fine-tuned for specific goals with a much smaller dataset.

How are LLMs trained?

- When you submit a prompt to an LLM, it calculates the probability of the correct answer from its pre-trained model. – LLM often assume that the prompt is true.

- The probability is determined through a task called pre-training.
- Pre-training an LLM involves feeding a massive dataset of text, images, and code to the model so that it can learn the underlying structure and patterns of the language.
- sometimes the LLM gives a completely wrong answer.- is called a hallucination

Hallucinations הלוסינייישנס

- are words or phrases that are generated by the model that are often nonsensical or grammatically incorrect.
- they might not be aware of your business's proprietary or domain-specific data.
- OR they do not have access to real-time information.
- LLMs only understand the information that is explicitly given to them in the prompt.
- do not have the ability to ask for more context information
- LLM does not know anything outside of what it was trained on, and it cannot truly know if that information is accurate.

Reasons for hallucinations

- The model is not trained on enough data.
- The model is trained on noisy or dirty data.
- The model is not given enough context.
- The model is not given enough constraints.

Output of hallucinations

- make the output text difficult to understand.
- the model more likely to generate incorrect or misleading information.

Gemini will solve the hallucinations – Halleluiah

- access to a massive range of data,
- including Google Cloud documentation, tutorials, and samples.

prompt - is the text that you feed to the model

prompt engineering

- – a way of articulating your prompts to get the best response from the model.
- The better structured a prompt is, the better the output from the model will be.

Prompt – types

- zero-shot
  - do not contain any context or examples to assist the model.
  Example: What is the capital of Israel?
  LLM Problem: what is capital

- one-shot
  – provide one example to the model for context
  Example: What is the capital of Israel?
  Like Italy is the capital of Rome

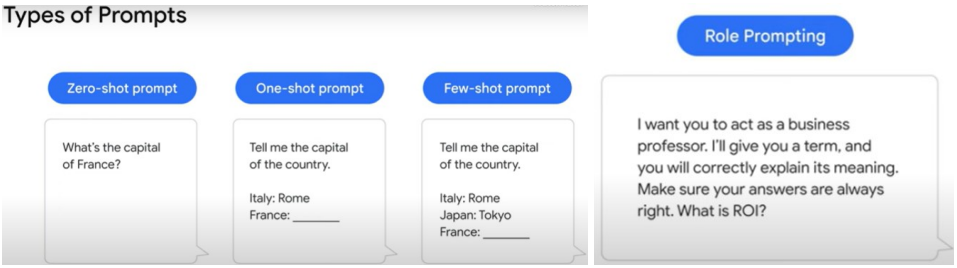
- few-shot
  - provide at list 2 examples to the model for context
  Example: What is the capital of Israel?
  Like Italy is the capital of Rome
  And Tokyo is the capital of Japan

- role prompts
  - a frame of reference –

  what is required and in what context.

  " I am professor in the university, seeking for ….

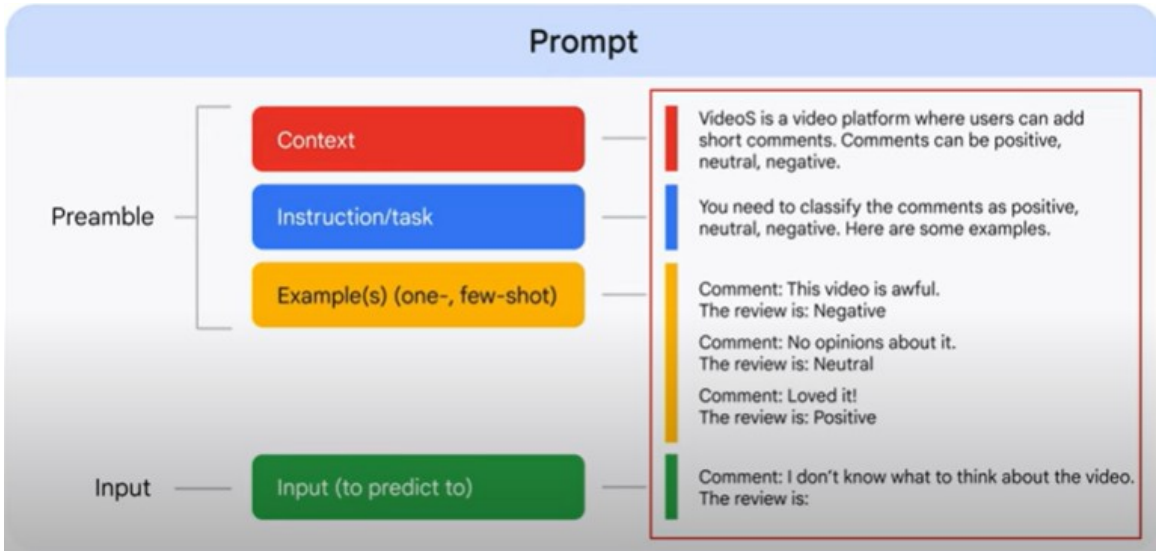
  How can I use gcloud to create VPC (context: gcloud, require: VPC)



LLM parts:

Preamble: introductory text

Input: central request – (what do I want from it)





Best practices:

- write detailed and explicit instructions
- define boundaries – what to do rather than what not to do

  If the model gets stuck,

  give it a few 'fallback' outputs that work in various situations

  something like "I'm still learning about that" to use when unsure

- adopt a persona for your input. – You're a cloud architect
- keep each sentence concise

  break long sentences into a series of shorter sentences and simpler tasks.

Example:

You're a cloud architect.

You want to build a Google Cloud VPC network that can be centrally managed.

You also connect to other VPC networks in your company's other regions.

You don't want to have many different sets of firewall policies to maintain.

What sort of network architecture would you recommend?”

Gemini proposes a hub-and-spoke network architecture – BOOM 🔥

Summarize:

https://www.cloudskillsboost.google/course_templates/60/video/513935

In module 1

you were introduced to Google Cloud and cloud computing.

Specifically, you explored:

The concept of managed infrastructure and managed services,

through IaaS and PaaS.

The Google Cloud network.

Google Cloud’s focus on security throughout our infrastructure.

How Google publishes key elements of technology using open source licenses.

Google Cloud’s pricing structure and billing tools.

In module 2

you learned about the Google Cloud Resource Hierarchy,

four levels: resources, projects, folders, and an organization node.

Defining policies and their downward inheritance.

When to use Identity and Access Management, or IAM,

four ways to access and interact with Google Cloud:

Google Cloud console

Cloud SDK and Cloud Shell,

APIs,

Google Cloud App.

In module 3

you explored how Compute Engine works,

with a focus on virtual machines and virtual networking.

The VPC, or virtual private cloud.

Compute Engine’s Autoscaling feature.

important Google Virtual Private Cloud compatibility features, like routing tables, firewalls, VPC peering and shared VPC,

all of which result in the need for less network management.

Load Balancing, a fully distributed, software-defined, managed service for all your traffic.

compared how on-premises or other-cloud networks

can be interconnected with a Google VPC.

In module 4

Google Cloud's five core storage options:

Cloud Storage, Bigtable, Cloud SQL, Spanner, and Firestore.

4 storage classes that make up Cloud Storage:

1. Standard Storage, which is used for frequently accessed hot data,
2. Nearline Storage
3. Coldline Storage, which are used for less-frequently accessed cool data,
4. Archive Storage.

In module 5

containers, which are invisible boxes around your code and its dependencies.

introduced to containers, along with: Kubernetes

Google Kubernetes Engine (GKE):

a Google-hosted managed Kubernetes service in the cloud.

In module 6

Developing applications in the cloud.

Cloud Run:

a managed compute platform

lets you run stateless containers via web requests or Pub/Sub events.

Cloud Functions:

a lightweight, event-based, asynchronous compute solution

to create single-purpose functions.

in module 7

combine Google Cloud knowledge with prompt engineering

to improve Gemini responses.

What is generative AI?

What is a large language model?

What is prompt engineering?

Identifying prompt engineering best practices.

PDF:

Module 1: Introduction to Google Cloud

Module 2: Resources and Access in the Cloud

Module 3: Virtual Machines and Networks in the Cloud

Module 4: Storage in the Cloud

Module 5: Containers in the Cloud

Module 6: Applications in the Cloud

Module 7: Prompt Engineering

Course Summary

gcloud

gcloud – get external ip:

gcloud compute instances list --filter=name:gcelab2 --format='value(EXTERNAL_IP)'

gcloud logs

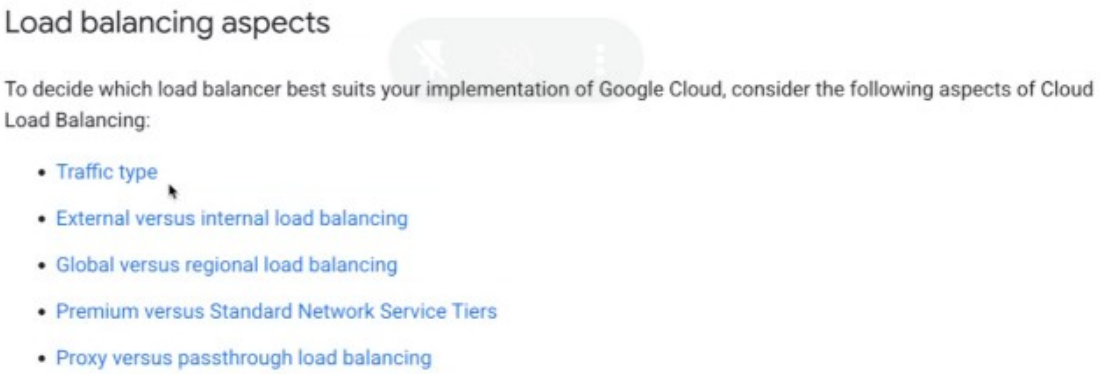
https://www.cloudskillsboost.google/course_templates/648/labs/484534

LB, VM with web server:

https://www.cloudskillsboost.google/course_templates/648/labs/484535

LB and create instances based on template:

https://cloud.google.com/load-balancing/docs/network/networklb-backend-service



Load balancing aspects

To decide which load balancer best suits your implementation of Google Cloud, consider the following aspects of Cloud Load Balancing:

- Traffic type
- External versus internal load balancing
- Global versus regional load balancing
- Premium versus Standard Network Service Tiers
- Proxy versus passthrough load balancing

LB create

1. Create a static external IP address for your load balancer:

2. ```
   gcloud compute addresses create network-lb-ip-1 \
     --region us-east4
   ```

   Copied!

   content_copy

   Output:

   ```
   Created
   [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-03-xxxxxxxxxxx/regions/us-east4/addresses/network-lb-ip-1].
   ```

3. Add a legacy HTTP health check resource:

   ```
   gcloud compute http-health-checks create basic-check
   ```

   Copied!

   content_copy

4. Add a target pool in the same region as your instances. Run the following to create the target pool and use the health check, which is required for the service to function:

5. ```
   gcloud compute target-pools create www-pool \
     --region us-east4 --http-health-check basic-check
   ```

Copied!

content_copy

6. Add the instances to the pool:

```
gcloud compute target-pools add-instances www-pool \
    --instances www1,www2,www3
```

Copied!

content_copy

8. Add a forwarding rule:

```
gcloud compute forwarding-rules create www-rule \
     --region  us-east4 \
     --ports 80 \
     --address network-lb-ip-1 \
    --target-pool www-pool
```

Send traffic to your instances

1. Enter the following command to view the external IP address of the www-rule forwarding rule used by the load balancer:

```
gcloud compute forwarding-rules describe www-rule --region us-east4
```

Copied!

content_copy

2. Access the external IP address:

```
IPADDRESS=$(gcloud compute forwarding-rules describe www-rule --region us-east4 --format="json" | jq -r .IPAddress)
```

Copied!

content_copy

3. Show the external IP address:

```
echo $IPADDRESS
```

Copied!

content_copy

4. Use the `curl` command to access the external IP address, replacing `IP_ADDRESS` with an external IP address from the previous command:

```
while true; do curl -m1 $IPADDRESS; done
```

Google Cloud health checking systems (`130.211.0.0/22` and `35.191.0.0/16`).

Create Load Balancer and managed groups :

HL steps:

PREREQUISITS:

1. **instance template** : configuration for the VM instances (machine type, startup script, tags, etc.).
2. **managed instance group (MIG) :** uses the template to manage a group of instances, providing scaling, replication, and lifecycle management
1. **Set the Named Port -** associate the name http with port 80
3. **Create a Firewall Rule**

HTTP(S) Load Balancer:

1. Create a Health Check
2. Create a Backend Service
3. Add the MIG to the backend service:
4. Create a URL Map : The URL map routes requests to the backend service
5. Create a Target HTTP Proxy: handles HTTP requests and forwards them to the backend
6. Reserve a Global IP Address
7. Create a Forwarding Rule: directs traffic to the load balancer - **Connecting External IP to Load Balancer**:

Detailed steps:

PREREQUISITS

2. **instance template:**

```
gcloud compute instance-templates create lb-backend-template \
  --region=us-central1\
  --network=default \
  --subnet=default \
  --tags=allow-health-check \
  --machine-type=e2-medium \
  --image-family=debian-11 \
  --image-project=debian-cloud \
  --metadata=startup-script='#! /bin/bash
apt-get update
apt-get install -y nginx
service nginx start
sed -i "s/nginx/Google Cloud Platform - $HOSTNAME/" /var/www/html/index.nginx-debian.html'
```

3. **create managed instance group**

```
gcloud compute instance-groups managed create lb-backend-group \
   --template=lb-backend-template \
   --size=2 \
   --zone=us-central1-a
```

4. **Set the Named Port -** associate the name `http` with port `80`

```
gcloud compute instance-groups set-named-ports lb-backend-group \
   --named-ports=http:80 \
   --zone=us-central1-a
```

5. **Create a Firewall Rule**

```
gcloud compute firewall-rules create allow-tcp-rule-556 \
```

```
  --direction=INGRESS \
  --priority=1000 \
  --network=default \
  --action=ALLOW \
  --rules=tcp:80 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=allow-health-check
```

HTTP(S) Load Balancer:

1. Create Health Check:

```
gcloud compute health-checks create http http-health-check \
  --port=80 \
  --request-path=/ \
  --check-interval=10s \
  --timeout=5s \
  --unhealthy-threshold=2 \
  --healthy-threshold=2
```

2. Create a Backend Service

```
gcloud compute backend-services create web-backend-service \
  --protocol=HTTP \
  --health-checks=http-health-check \
  --global
```

3. Add the Managed Instance Group to the backend service:

```
gcloud compute backend-services add-backend web-backend-service \
  --instance-group=lb-backend-group \
  --instance-group-zone=us-central1-a \
  --global
```

4. Create a URL Map

```
gcloud compute url-maps create web-map \
  --default-service=web-backend-service
```

5. Create a Target HTTP Proxy

```
gcloud compute target-http-proxies create http-lb-proxy \
  --url-map=web-map
```

6. Create a Forwarding Rule – connect external ip address to LB

```
gcloud compute addresses create web-lb-ip
gcloud compute forwarding-rules create http-content-rule \
  --address=web-lb-ip \
  --global \
  --target-http-proxy=http-lb-proxy \
  --ports=80
```

7. Verify the Setup

```
gcloud compute addresses describe web-lb-ip --global --format="get(address)"
```

Brows or using curl to confirm the load balancer serves traffic.

DNS Load Balancing

DNS load balancing is a technique where Domain Name System (DNS) is used to distribute traffic across multiple IP addresses.

Instead of routing requests to a single IP, DNS servers return different IP addresses in response to queries, enabling traffic distribution among multiple backend services.

2. How DNS Load Balancing Works in GCP

GCP provides **Cloud DNS**, a **managed DNS service**, which supports load balancing at the **DNS level**. This can be done in different ways:

1. **Round-robin DNS**:
    - Multiple **A or AAAA records** are configured for a domain (e.g., www.example.com → multiple IPs).
    - Clients resolve the domain and get one of the IPs in a round-robin manner.
    - This approach is simple but lacks **failover** (clients might cache failed IPs).
2. **GeoDNS / Latency-Based Routing**:

- Uses **Google Cloud DNS policies** to return different IPs based on the client's geographic location.
- This directs users to the nearest or lowest-latency server.

3. **Weighted DNS Routing**:
   - Distributes traffic proportionally based on preconfigured weightings.
   - Useful for **gradual rollout of changes** (e.g., 80% traffic to one backend, 20% to another).
4. **Failover with DNS TTL**:
   - Short **Time-to-Live (TTL)** values allow quick updates in case of failure.
   - Requires health checks and external automation to remove failed instances.

3. When to Use DNS Load Balancing

✅ **Multi-Region GCP Deployments**: Direct users to the closest or least-latency region.
✅ **Distributing Traffic Across Multiple Services**: Use DNS to balance across multiple backend servers (e.g., multi-cloud setup).
✅ **High Availability**: DNS can be configured with multiple IPs to prevent single points of failure.
✅ **Minimal Infrastructure Overhead**: No need for GCP's **HTTP(S) Load Balancer** when DNS-level balancing suffices.

4. Limitations of DNS Load Balancing

⚠ **DNS Caching**: Clients and ISPs cache DNS responses, causing delays in failover.
⚠ **No Intelligent Health Checking**: Unlike GCP's Load Balancer, DNS doesn't automatically remove unhealthy instances.
⚠ **Latency in Failover**: If an instance fails, clients may continue to use its IP due to DNS caching.

Alternatives

- **GCP HTTP(S) Load Balancer** (Recommended for most GCP applications).
- **Cloud Load Balancing** (for TCP/UDP-based services).
- **Traffic Director** (for service mesh-based traffic control).

Global External Application Load Balancer (Global External ALB)

The **Global External ALB** is a **fully managed Layer 7 (HTTP/S) load balancer** that distributes traffic across multiple **regions** worldwide. It provides **low-latency, high-availability, and intelligent routing** for internet-facing applications.

Key Features:

✅ **Global Load Balancing**: Routes traffic to the nearest healthy backend across multiple regions.
✅ **Layer 7 (HTTP/S) Routing**: Supports advanced routing based on URL paths, headers, cookies, etc.
✅ **Autoscaling & Health Checking**: Automatically scales backend instances and ensures only healthy endpoints receive traffic.
✅ **Built-in DDoS Protection**: Integrated with **Cloud Armor** for security against attacks.
✅ **TLS/SSL Termination**: Offloads SSL encryption to the load balancer, reducing backend workload.
✅ **CDN Integration**: Works with **Cloud CDN** for caching and faster content delivery.

---

2. How It Works

1. **Users Request Your Application**:
   - A user accesses your website (`https://example.com`).
   - The request goes to **Google's global Anycast network**.

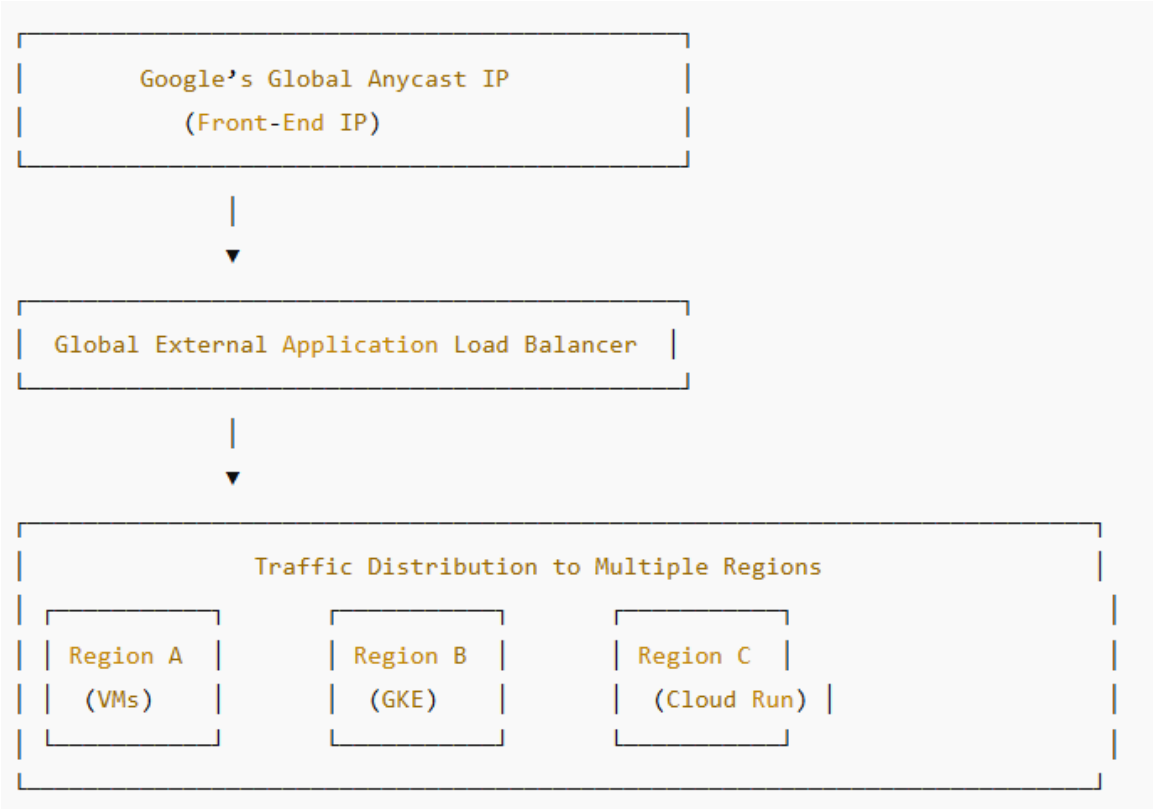
2. **Traffic Is Routed to the Closest Healthy Region**:
   - Google's **Edge Network** determines the nearest backend based on **latency and availability**.
   - Traffic is routed to backend instances in one of the **multiple GCP regions**.

3. **Intelligent Load Balancing Across Backends**:
   - The **backend services** can be **VM instances, GKE clusters, or serverless applications (Cloud Run, App Engine, Cloud Functions)**.
   - Load balancing uses **round-robin, least connections, or weighted routing** strategies.

4. **Response Sent Back via Google's Network**:
   - The response is returned to the client via **Google's private backbone**, ensuring **low latency**.

---

3. Architecture Diagram



4. Use Cases

✅ Multi-Regional Web Applications

- Deploy a website globally, automatically routing users to the **nearest available region**.

✅ Disaster Recovery & Failover

- If one region **fails**, the load balancer **automatically** redirects traffic to another healthy region.

✅ Microservices Routing

- Direct requests to different **backend services** (e.g., API Gateway, Cloud Run, or GKE) based on **URL paths**.

✅ Secure & Scalable API Endpoints

- Expose APIs to the public while using **Cloud Armor** for security against DDoS attacks.

✅ CDN-Accelerated Websites

- Enable **Cloud CDN** to cache content globally for **faster load times**.

---

5. Key Benefits Over Other Load Balancers

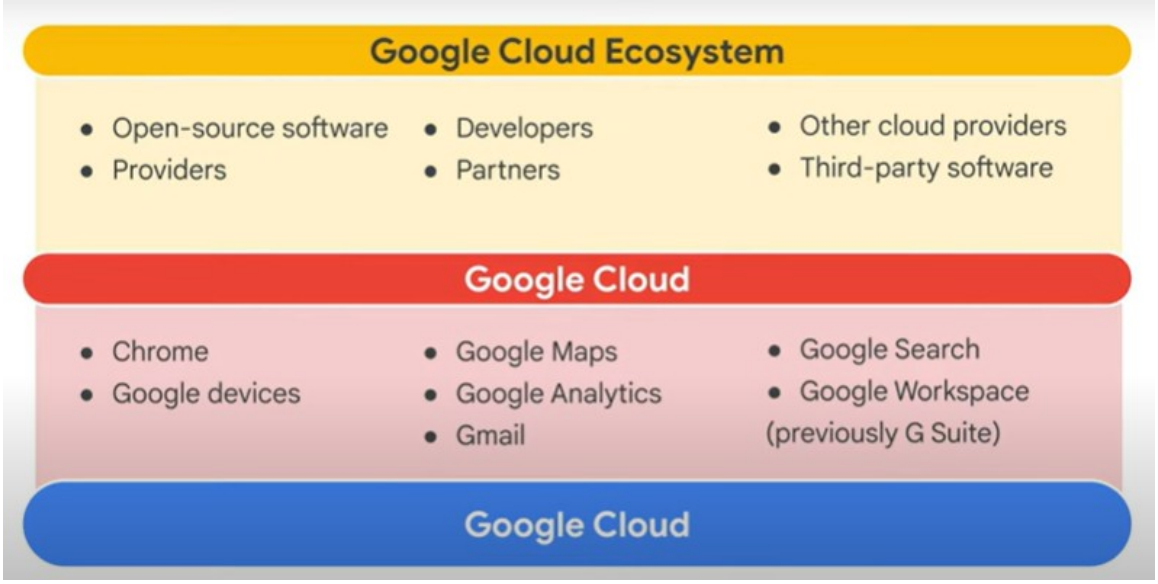
Feature	Global External ALB	Regional External ALB	Classic HTTP LB
Global Routing	✅ Yes	❌ No (Only in One Region)	✅ Yes
Multi-Region Backend Support	✅ Yes	❌ No	✅ Yes
Autoscaling	✅ Yes	✅ Yes	✅ Yes
Advanced Traffic Management	✅ Yes (Traffic Steering, Failover)	❌ Limited	❌ Limited
Integrated Cloud Armor (DDoS Protection)	✅ Yes	✅ Yes	✅ Yes
Cloud CDN Support	✅ Yes	✅ Yes	✅ Yes
Backend Options	VMs, GKE, Cloud Run, App Engine	VMs, GKE, Cloud Run	VMs, GKE, App Engine

Comparison: DNS Load Balancing vs. Global External Application Load Balancer (ALB)

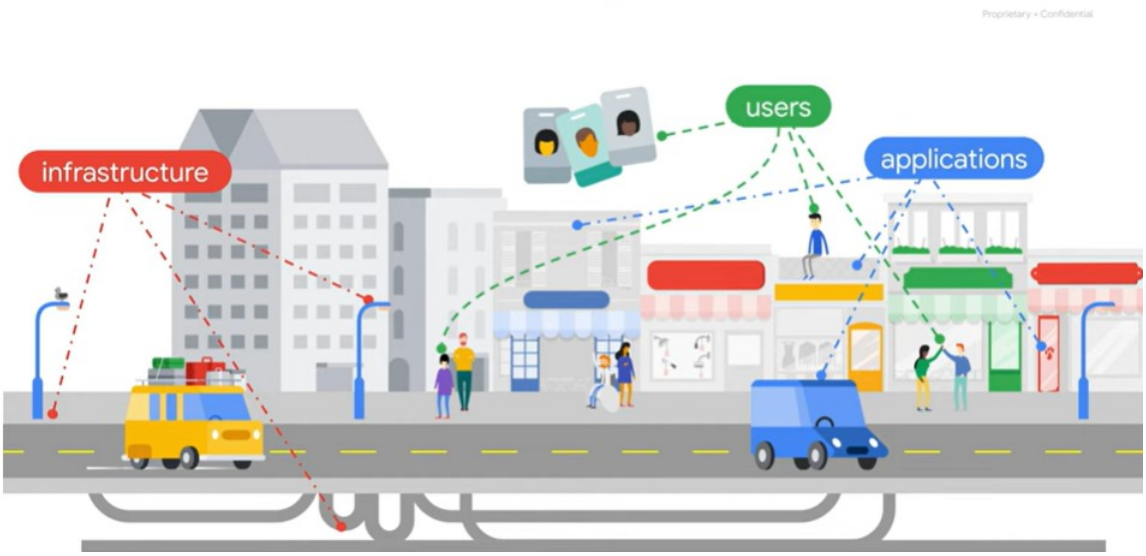
Feature	DNS Load Balancing	Global External ALB
Layer	Layer 3 (DNS)	Layer 7 (HTTP/S)
Traffic Routing	Returns different IPs per DNS request	Routes traffic dynamically based on policies
Health Checks	No built-in health checks (requires external automation)	Built-in health checks to ensure only healthy backends receive traffic
Failover Speed	Slower (depends on DNS TTL and client caching)	Instant failover (real-time health checks)
Routing Flexibility	Limited (GeoDNS, weighted routing)	Advanced (URL path-based, session affinity, load balancing algorithms)
Global Scaling	Global via multiple DNS records	Google's Anycast network routes traffic to nearest healthy region
DDoS Protection	None (must use Cloud Armor separately)	Built-in DDoS protection with Cloud Armor
SSL Termination	Not possible	Offloads SSL termination to GCP (reduces backend load)
Caching & CDN	External (Cloud CDN, third-party)	Integrated with Cloud CDN for performance optimization
Cost	Lower (just Cloud DNS cost)	Higher (traffic-based pricing, but more features)

Cloud infrastructure: Foundation

Google Cloud Echo System



GCP itself is a computing solution platform that really encompasses three core features: infrastructure, platform, and software.



Remark:

App Engine & Cloud Functions

App Engine is GCP's fully managed platform as a service framework.

You just focus on your code and let Google deal with all the provisioning and resource management.

can be learn in “Developing Applications with Google Cloud Platform" course series.

Cloud Run

managed compute platform, serverless, that lets you run stateless containers via web requests or Pub/Sub events.

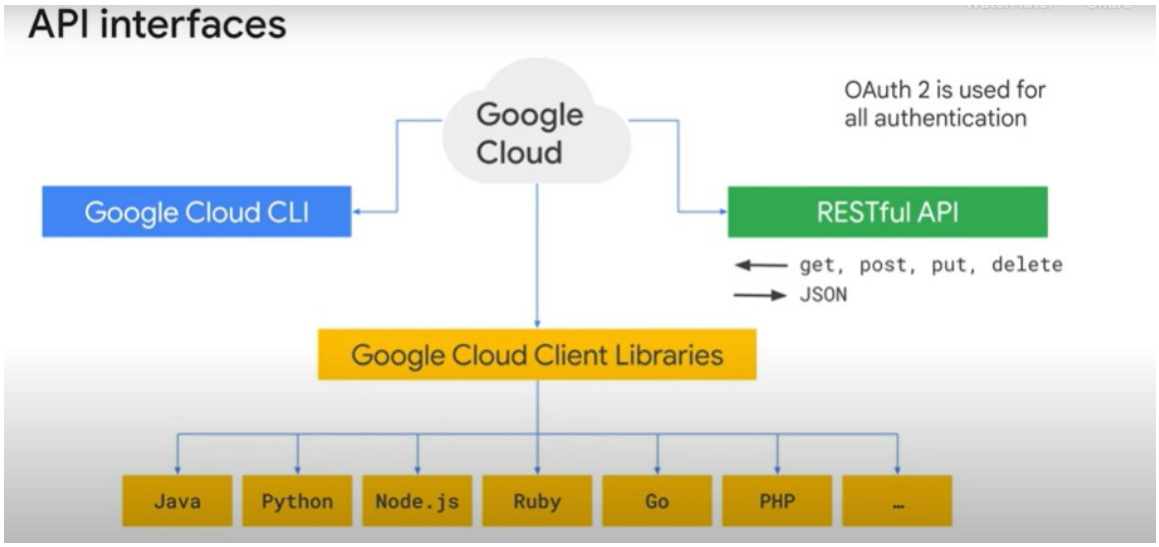
It is built on Knative, an open API and runtime environment built on Kubernetes that gives you freedom to move your workloads across different environments and platforms.

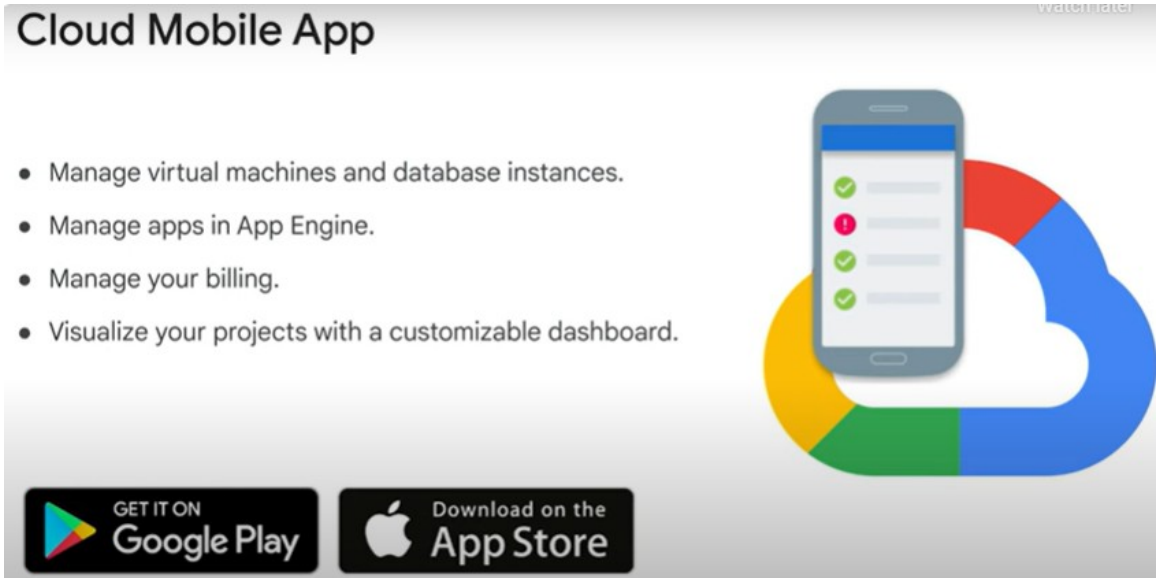
It can be fully managed on Google Cloud, on Google Kubernetes Engine, or anywhere Knative runs.

.

4 ways to interact:

- REST API
- Google CLI / cloudshell (tmp vm + 5 GB of persistent disk + Google Cloud CLI pre-installed.)
- Mobile app
- Console





Setting cloud shell

gcloud compute regions list

INFRACLASS_REGION=[YOUR_REGION]

mkdir infraclass

echo INFRACLASS_REGION=$INFRACLASS_REGION > ~/infraclass/config

INFRACLASS_PROJECT_ID=[YOUR_PROJECT_ID]

```
echo INFRACLASS_PROJECT_ID=$INFRACLASS_PROJECT_ID >>
~/infraclass/config
echo 'source infraclass/config' >> .profile
```

Compare between App Engine, GKE, Cloud Run, Compute Engine

Feature	**App Engine** 🟢	**GKE (Kubernetes)** ⚫	**Cloud Run** 🟣	**Compute Engine** 🟠
Management	Fully managed	Self-managed	Fully managed	Manual setup
Scaling	Auto-scaling	Auto & manual	Auto-scaling	Manual scaling
Customization	Limited	High	Limited	Full control
Stateful Apps	Not ideal	Yes	No	Yes
Traffic Splitting	✅ Yes	❌ No (manual)	✅ Yes	❌ No (manual)
Best For	Web apps, APIs	Microservices	Containers	Custom VMs, legacy apps

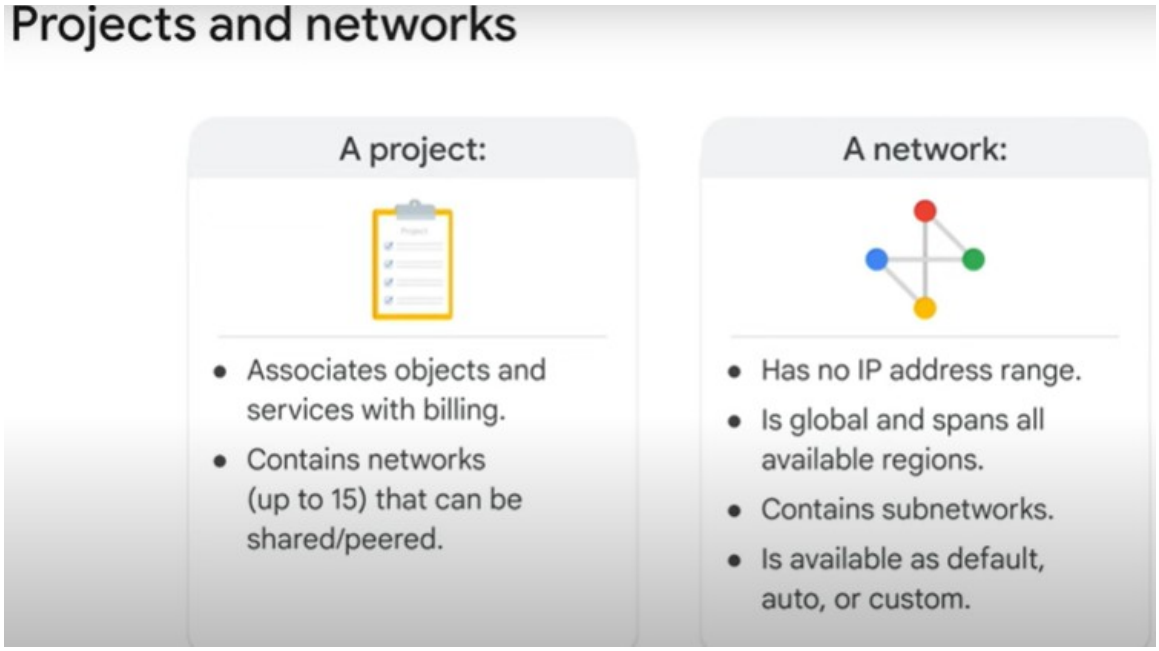
Google Cloud Marketplace deploy packages by providing pre-defined templates with **Deployment Manager Cloud service**

Marketplace – install Bitnami Jenkins

https://www.cloudskillsboost.google/course_templates/50/labs/485503

setting new project

gcloud config set project=<project id>



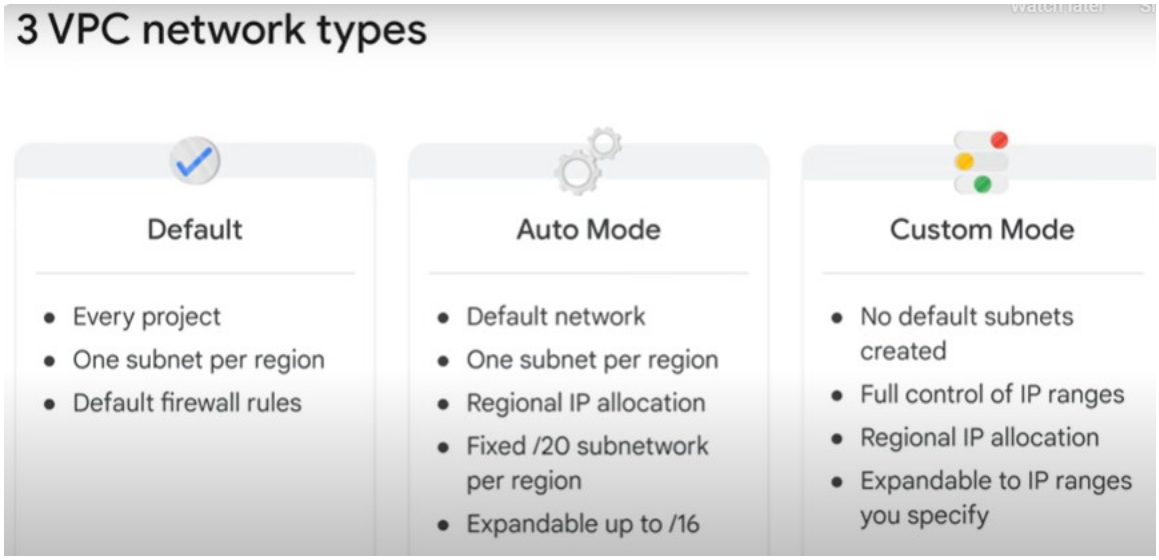
Every project is provided with a default VPC network with preset subnets and firewall rules.

Project -> Default VPC network -> firewall + subnets

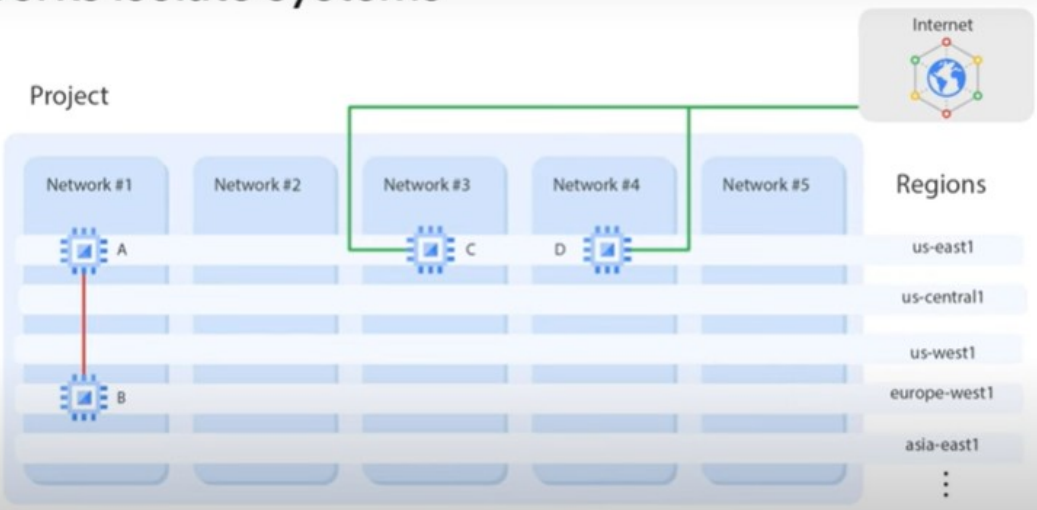
Subnet -> for every region with non-overlapping CIDR blocks and firewall rules

allow ingress traffic for ICMP, RDP, and SSH traffic from anywhere

allow ingress traffic from within the default network for all protocols and ports

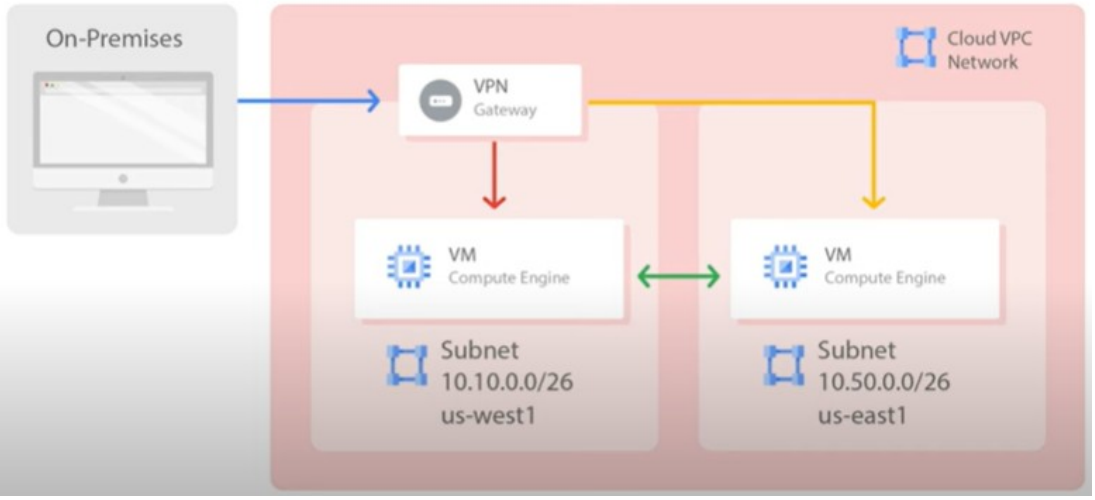


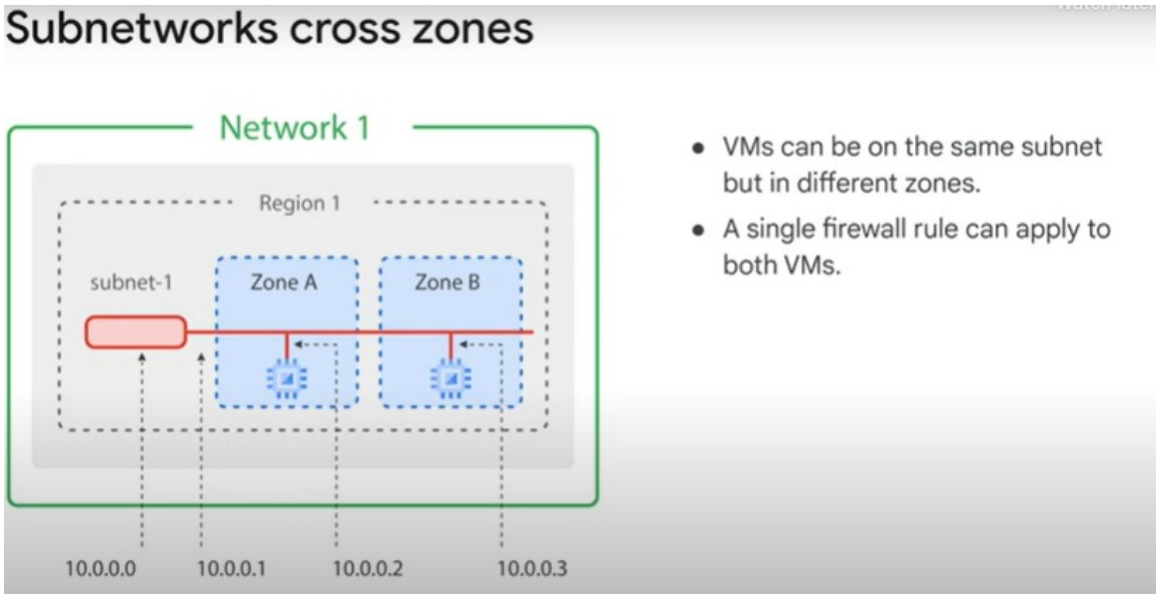
Networks isolate systems



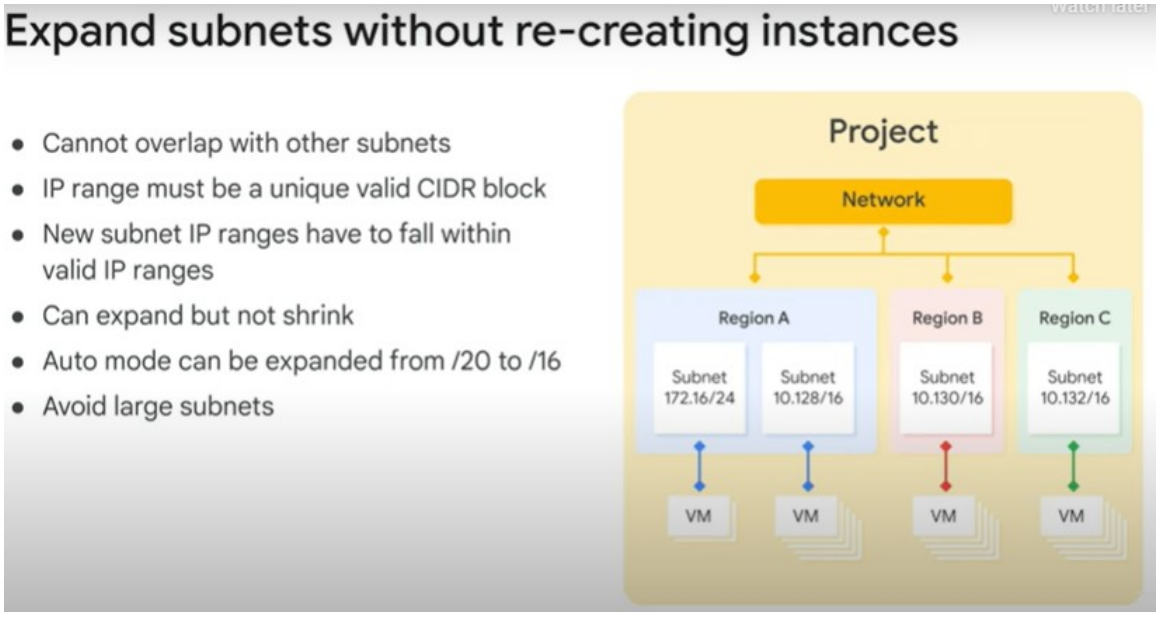
- **A** and **B** can communicate over internal IPs *even though they are in different regions.*
- **C** and **D** must communicate over external IPs *even though they are in the same region.*

Google's VPC is global

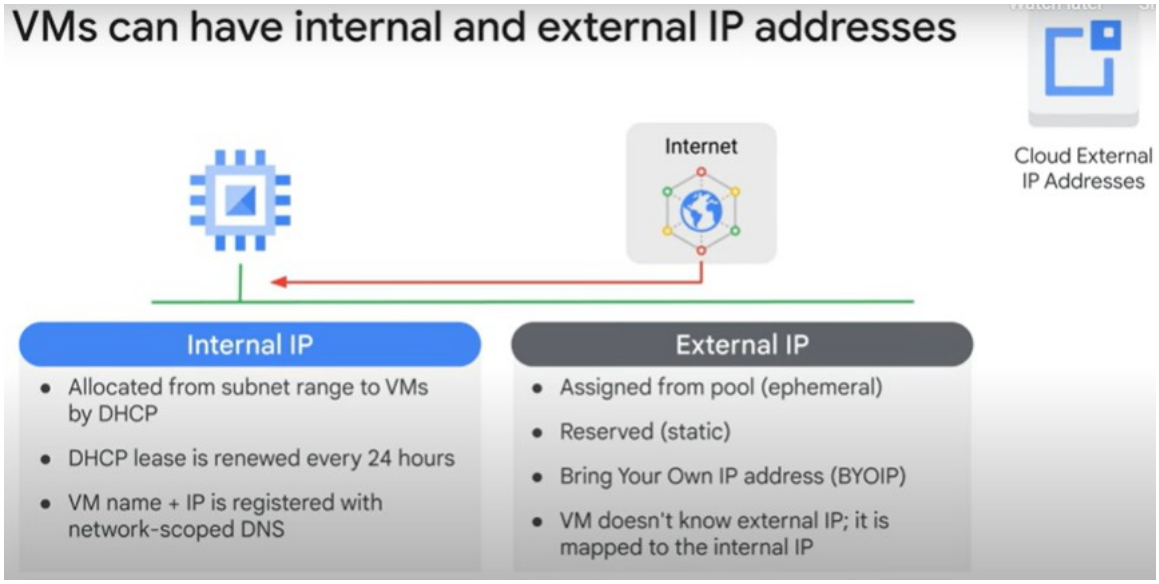


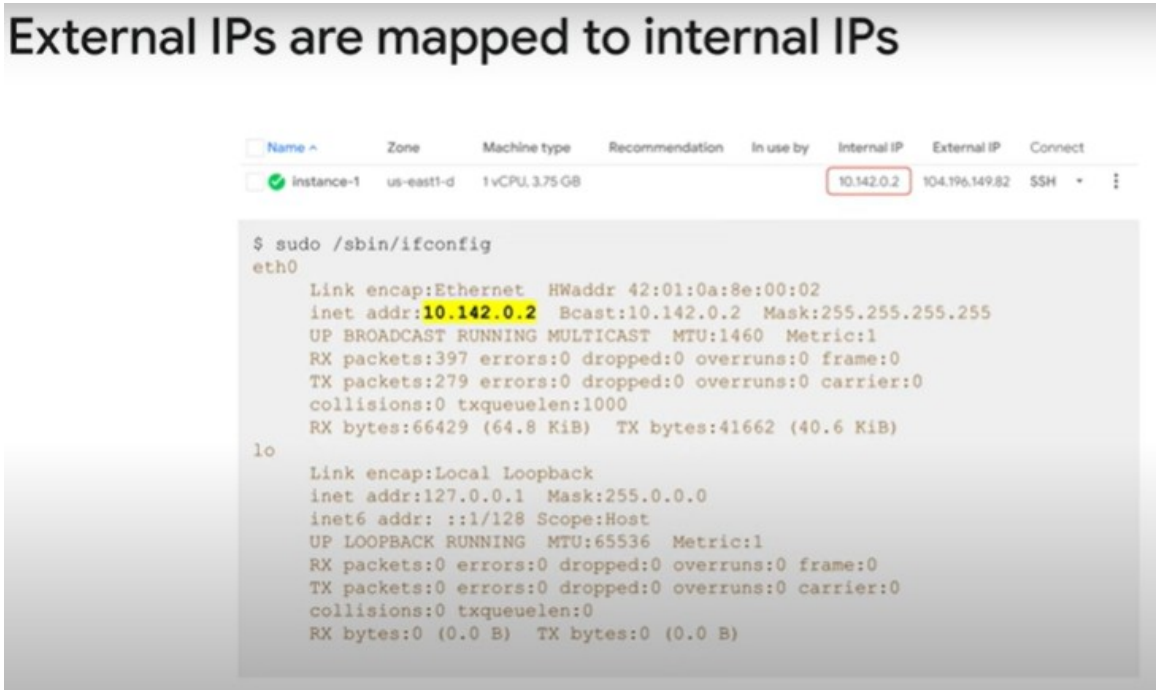


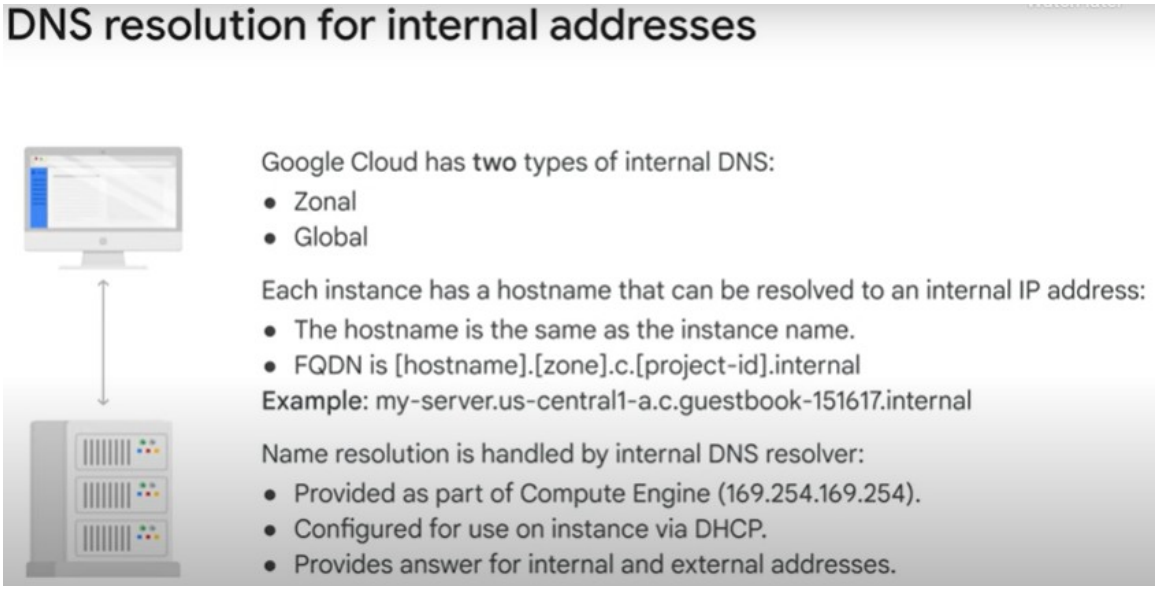
Google Cloud VPCs let you increase the IP address space of any subnets without any workload shutdown or downtime.



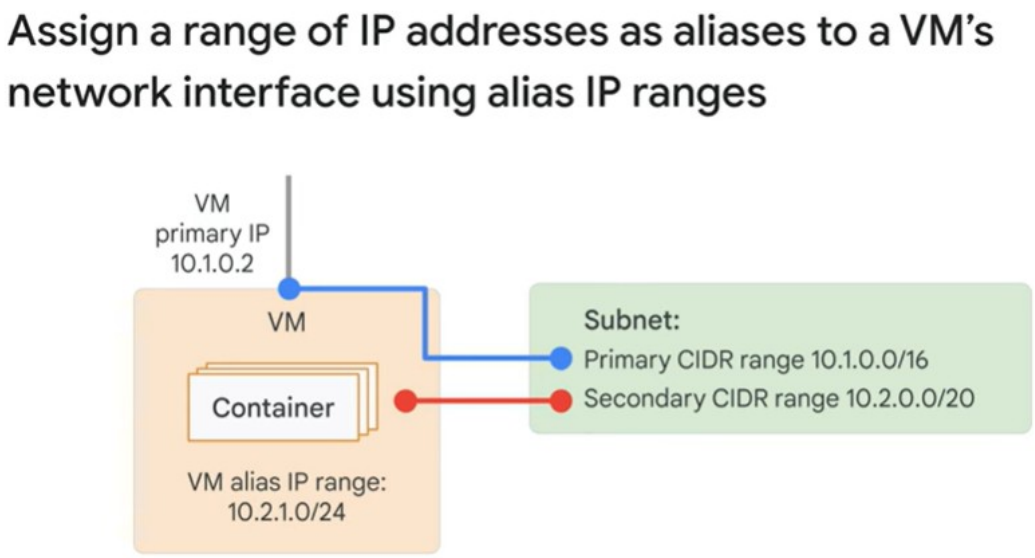
Expand subnet







FQDN = fully qualified domain name,



Look on IPAddressesForDefaultDomain.pdf file – for specific domains

Configure Private Google Access:
https://cloud.google.com/vpc/docs/configure-private-google-access#ip-addr-defaults

Routes and firewall Rules

A route is a mapping of an IP range to a destination

Every network has:

- Routes that let instances in a network send traffic directly to each other.
- A default route that directs packets to destinations that are outside the network.

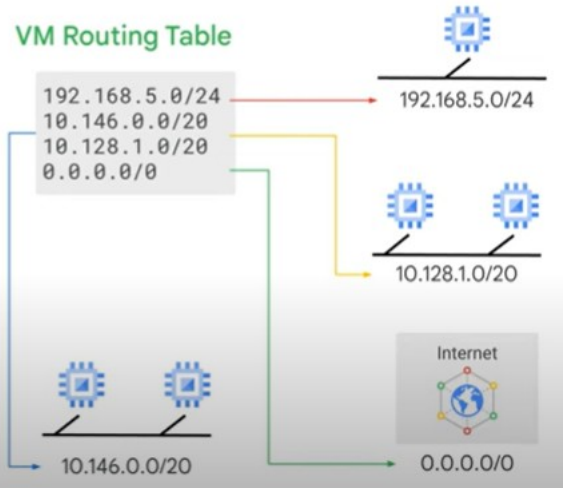
Firewall rules must also allow the packet.



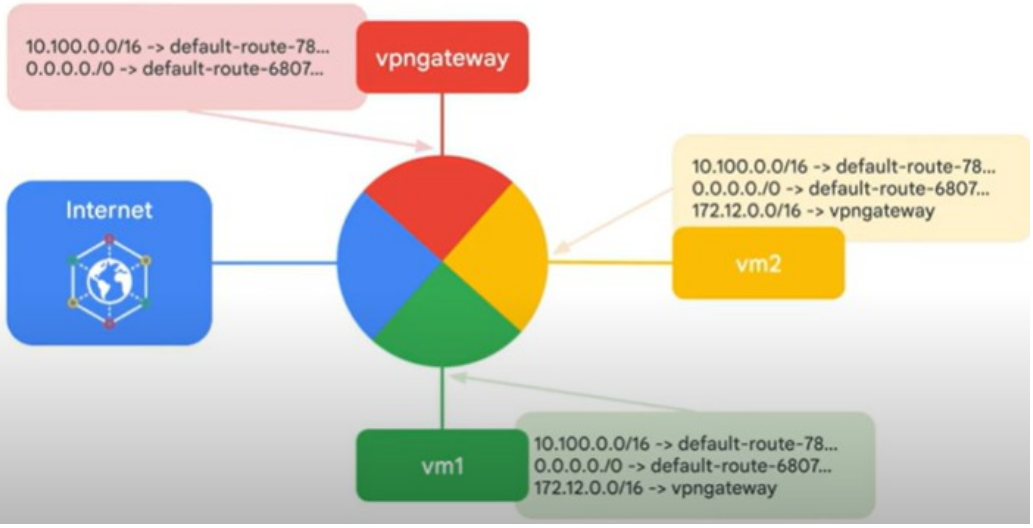
Cloud Routes

Routes map traffic to destination networks

- Apply to traffic egressing a VM.
- Forward traffic to most specific route.
- Are created when a subnet is created.
- Enable VMs on same network to communicate.
- Destination is in CIDR notation.
- Traffic is delivered only if it also matches a firewall rule.



Instance routing tables



Firewall rules protect your VM instances from unapproved connections

- VPC network functions as a distributed firewall.
- Firewall rules are applied to the network as a whole.
- Connections are allowed or denied at the instance level.
- Firewall rules are stateful.
- Implied deny all ingress and allow all egress.

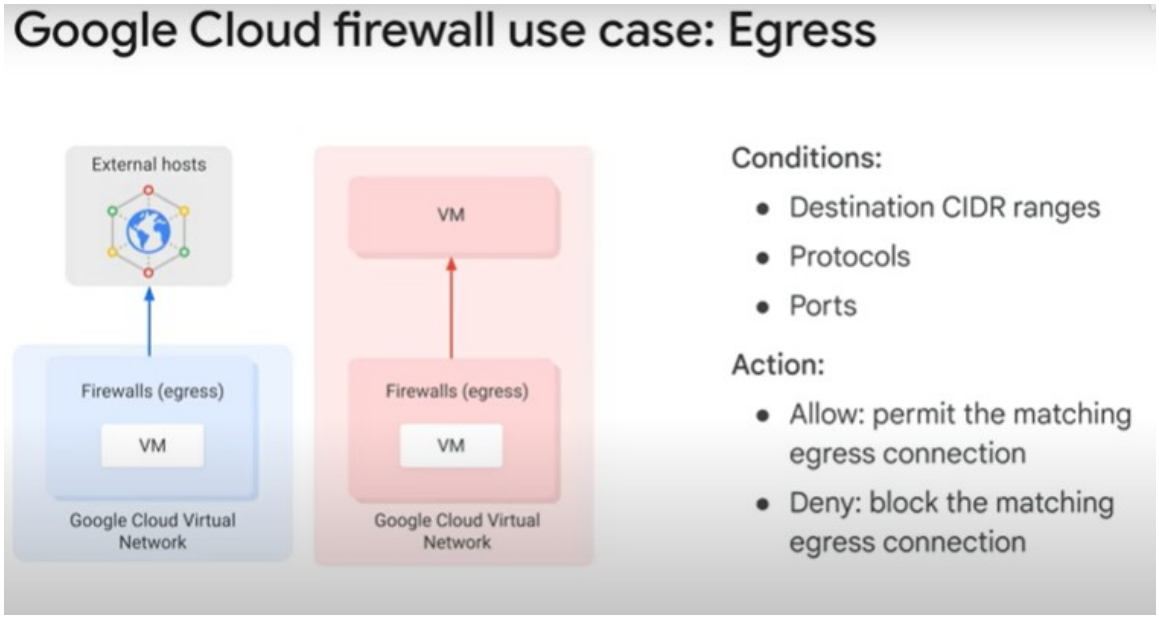


Cloud Firewall Rules

A firewall rule is composed of...

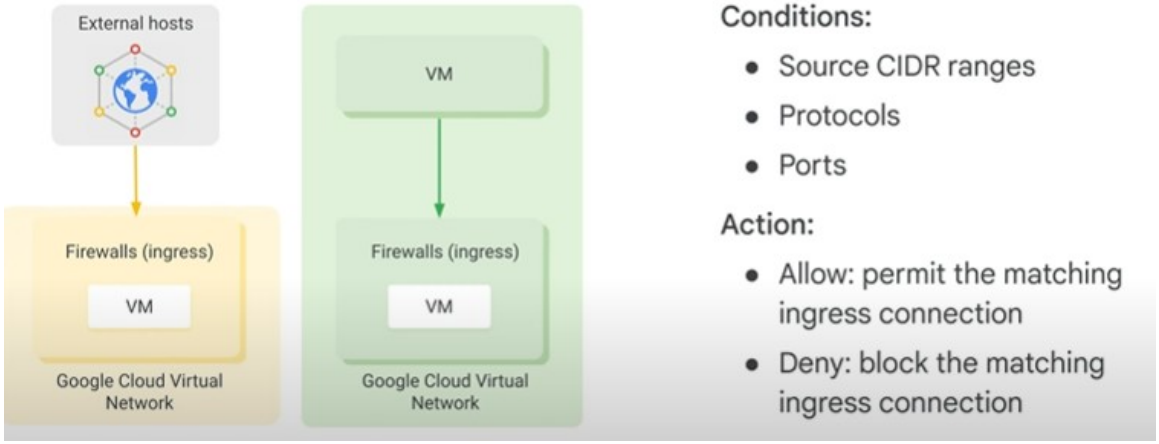
Parameter	Details
direction	Inbound connections are matched against ingress rules only.
	Outbound connections are matched against egress rules only.
source or destination	For the ingress direction, sources can be specified as part of the rule with IP addresses, source tags or a source service account.
	For the egress direction, destinations can be specified as part of the rule with one or more ranges of IP addresses.
protocol and port	Any rule can be restricted to apply to specific protocols only or specific combinations of protocols and ports only.
action	To allow or deny packets that match the direction, protocol, port, and source or destination of the rule.
priority	Governs the order in which rules are evaluated; the first matching rule is applied.
Rule assignment	All rules are assigned to all instances, but you can assign certain rules to certain instances only.

Egress firewall rules control outgoing connections



Ingress firewall rules protect against incoming connections

Google Cloud firewall use case: Ingress



Network pricing (subject to change)

Traffic type	Price
Ingress	No charge
Egress to the same zone (internal IP address)	No charge
Egress to Google products (YouTube, Maps, Drive)	No charge
Egress to a different Google Cloud service (within same region; exceptions)	No charge
Egress between zones in the same region (per GB)	$0.01
Egress to the same zone (external IP address, per GB)	$0.01
Egress between regions within the US and Canada (per GB)	$0.01
Egress between regions, not including traffic between US regions	Varies by region

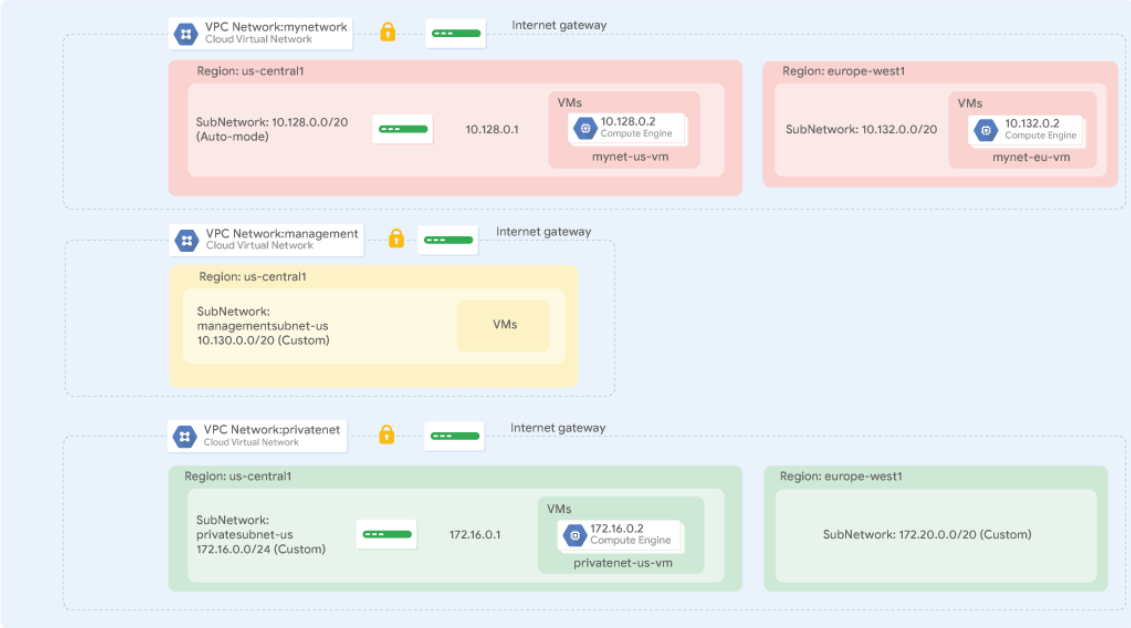
VPC networks are by default isolated private networking domains. Therefore, no internal IP address communication is allowed between networks, unless you set up mechanisms such as VPC peering or VPN.

VPC A → VPC peering or VPN ← VPC B

LAB - Networking

https://www.cloudskillsboost.google/course_templates/50/labs/485519

https://www.cloudskillsboost.google/course_sessions/20806141/video/485520



managementnet

gcloud compute networks create managementnet --project=qwiklabs-gcp-02-8f0eeb4e0cb6 --subnet-mode=custom --mtu=1460 --bgp-routing-mode=regional --bgp-best-path-selection-mode=legacy

gcloud compute networks subnets create managementsubnet-us --project=qwiklabs-gcp-02-8f0eeb4e0cb6 --range=10.240.0.0/20 --stack-type=IPV4_ONLY --network=managementnet --region=us-central1

privatenet:

student_01_58d22866e455@cloudshell:~ (qwiklabs-gcp-02-8f0eeb4e0cb6)$ gcloud compute networks create privatenet --subnet-mode=custom

Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-02-8f0eeb4e0cb6/global/networks/privatenet].

NAME: privatenet

SUBNET_MODE: CUSTOM

BGP_ROUTING_MODE: REGIONAL

IPV4_RANGE:

GATEWAY_IPV4:

Instances on this network will not be reachable until firewall rules

are created. As an example, you can allow all internal traffic between

instances as well as SSH, RDP, and ICMP by running:

$ gcloud compute firewall-rules create <FIREWALL_NAME> --network privatenet --allow tcp,udp,icmp --source-ranges <IP_RANGE>

$ gcloud compute firewall-rules create <FIREWALL_NAME> --network privatenet --allow tcp:22,tcp:3389,icmp

student_01_58d22866e455@cloudshell:~ (qwiklabs-gcp-02-8f0eeb4e0cb6)$ gcloud compute networks subnets create privatesubnet-us --network=privatenet --region=us-central1 --range=172.16.0.0/24

Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-02-8f0eeb4e0cb6/regions/us-central1/subnetworks/privatesubnet-us].

NAME: privatesubnet-us

REGION: us-central1

NETWORK: privatenet

RANGE: 172.16.0.0/24

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

student_01_58d22866e455@cloudshell:~ (qwiklabs-gcp-02-8f0eeb4e0cb6)$ gcloud compute networks subnets create privatesubnet-notus --network=privatenet --region=europe-west4 --range=172.20.0.0/20

Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-02-8f0eeb4e0cb6/regions/europe-west4/subnetworks/privatesubnet-notus].

NAME: privatesubnet-notus

REGION: europe-west4

NETWORK: privatenet

RANGE: 172.20.0.0/20

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

student_01_58d22866e455@cloudshell:~ (qwiklabs-gcp-02-8f0eeb4e0cb6)$ gcloud compute networks list

NAME: managementnet

SUBNET_MODE: CUSTOM

BGP_ROUTING_MODE: REGIONAL

IPV4_RANGE:

GATEWAY_IPV4:

NAME: mynetwork

SUBNET_MODE: CUSTOM

BGP_ROUTING_MODE: REGIONAL

IPV4_RANGE:

GATEWAY_IPV4:

NAME: privatenet

SUBNET_MODE: CUSTOM

BGP_ROUTING_MODE: REGIONAL

IPV4_RANGE:

GATEWAY_IPV4:

student_01_58d22866e455@cloudshell:~ (qwiklabs-gcp-02-8f0eeb4e0cb6)$

gcloud compute networks subnets list --sort-by=NETWORK

NAME: managementsubnet-us

REGION: us-central1

NETWORK: managementnet

RANGE: 10.240.0.0/20

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

NAME: mynetwork

REGION: us-central1

NETWORK: mynetwork

RANGE: 10.128.0.0/20

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

NAME: mynetwork

REGION: europe-west4

NETWORK: mynetwork

RANGE: 10.164.0.0/20

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

NAME: privatesubnet-us

REGION: us-central1

NETWORK: privatenet

RANGE: 172.16.0.0/24

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

NAME: privatesubnet-notus

REGION: europe-west4

NETWORK: privatenet

RANGE: 172.20.0.0/20

STACK_TYPE: IPV4_ONLY

IPV6_ACCESS_TYPE:

INTERNAL_IPV6_PREFIX:

EXTERNAL_IPV6_PREFIX:

Firewalls rules:

Management:

gcloud compute --project=qwiklabs-gcp-02-8f0eeb4e0cb6 firewall-rules create managementnet-allow-icmp-ssh-rdp --direction=INGRESS --priority=1000 --network=managementnet --action=ALLOW --rules=tcp:22,tcp:3389 --source-ranges=0.0.0.0/0

privatenet:

gcloud compute firewall-rules create privatenet-allow-icmp-ssh-rdp --direction=INGRESS --priority=1000 --network=privatenet --action=ALLOW --rules=icmp,tcp:22,tcp:3389 --source-ranges=0.0.0.0/0

list all firewalls rules:

gcloud compute firewall-rules list --sort-by=NETWORK

NAME: managementnet-allow-icmp-ssh-rdp

NETWORK: managementnet

DIRECTION: INGRESS

PRIORITY: 1000

ALLOW: tcp:22,tcp:3389,icmp

DENY:

DISABLED: False

NAME: mynetwork-allow-custom

NETWORK: mynetwork

DIRECTION: INGRESS

PRIORITY: 65534

ALLOW: all

DENY:

DISABLED: False

NAME: mynetwork-allow-icmp

NETWORK: mynetwork

DIRECTION: INGRESS

PRIORITY: 65534

ALLOW: icmp

DENY:

DISABLED: False

NAME: mynetwork-allow-rdp

NETWORK: mynetwork

DIRECTION: INGRESS

PRIORITY: 65534

ALLOW: tcp:3389

DENY:

DISABLED: False

NAME: mynetwork-allow-ssh

NETWORK: mynetwork

DIRECTION: INGRESS

PRIORITY: 65534

ALLOW: tcp:22

DENY:

DISABLED: False

NAME: privatenet-allow-icmp-ssh-rdp

NETWORK: privatenet

DIRECTION: INGRESS

PRIORITY: 1000

ALLOW: icmp,tcp:22,tcp:3389

DENY:

DISABLED: False

To show all fields of the firewall, please show in JSON format: --format=json

To show all fields in table format, please see the examples in --help.

Create VMS

Create VM in managemenet

gcloud compute instances create instance-20250108-152107managementnet-us-vm --project=qwiklabs-gcp-02-8f0eeb4e0cb6 --zone=us-central1-c --machine-type=e2-micro --network-interface=network-tier=PREMIUM,stack-type=IPV4_ONLY,subnet=managementsubnet-us --metadata=enable-oslogin=true --maintenance-policy=MIGRATE --provisioning-model=STANDARD --service-account=65988263648-compute@developer.gserviceaccount.com --scopes=https://www.googleapis.com/auth/devstorage.read_only,https://www.googleapis.com/auth/logging.write,https://www.googleapis.com/auth/monitoring.write,https://www.googleapis.com/auth/service.management.readonly,https://www.googleapis.com/auth/servicecontrol,https://www.googleapis.com/auth/trace.append --create-disk=auto-delete=yes,boot=yes,device-name=instance-20250108-152107managementnet-us-vm,image=projects/debian-cloud/global/images/debian-12-bookworm-v20241210,mode=rw,size=10,type=pd-balanced --no-shielded-secure-boot --shielded-vtpm --shielded-integrity-monitoring --labels=goog-ec-src=vm_add-gcloud --reservation-affinity=any

terraform:

This code is compatible with Terraform 4.25.0 and versions that are backwards compatible to 4.25.0.

For information about validating this Terraform code, see https://developer.hashicorp.com/terraform/tutorials/gcp-get-started/google-cloud-platform-build#format-and-validate-the-configuration

resource "google_compute_instance" "instance-20250108-152107managementnet-us-vm" {

  boot_disk {

    auto_delete = true

    device_name = "instance-20250108-152107managementnet-us-vm"

```
 initialize_params {
   image = "projects/debian-cloud/global/images/debian-12-bookworm-v20241210"
   size  = 10
   type  = "pd-balanced"
 }

 mode = "READ_WRITE"
}

can_ip_forward      = false
deletion_protection = false
enable_display      = false

labels = {
 goog-ec-src = "vm_add-tf"
}

machine_type = "e2-micro"

metadata = {
 enable-oslogin = "true"
}

name = "instance-20250108-152107managementnet-us-vm"

network_interface {
 access_config {
   network_tier = "PREMIUM"
 }

 queue_count = 0
 stack_type  = "IPV4_ONLY"
 subnetwork  = "projects/qwiklabs-gcp-02-8f0eeb4e0cb6/regions/us-central1/subnetworks/managementsubnet-us"
}
```

```
  scheduling {
    automatic_restart   = true
    on_host_maintenance = "MIGRATE"
    preemptible         = false
    provisioning_model  = "STANDARD"
  }

  service_account {
    email  = "65988263648-compute@developer.gserviceaccount.com"
    scopes = ["https://www.googleapis.com/auth/devstorage.read_only", "https://www.googleapis.com/auth/logging.write",
"https://www.googleapis.com/auth/monitoring.write", "https://www.googleapis.com/auth/service.management.readonly",
"https://www.googleapis.com/auth/servicecontrol", "https://www.googleapis.com/auth/trace.append"]
  }

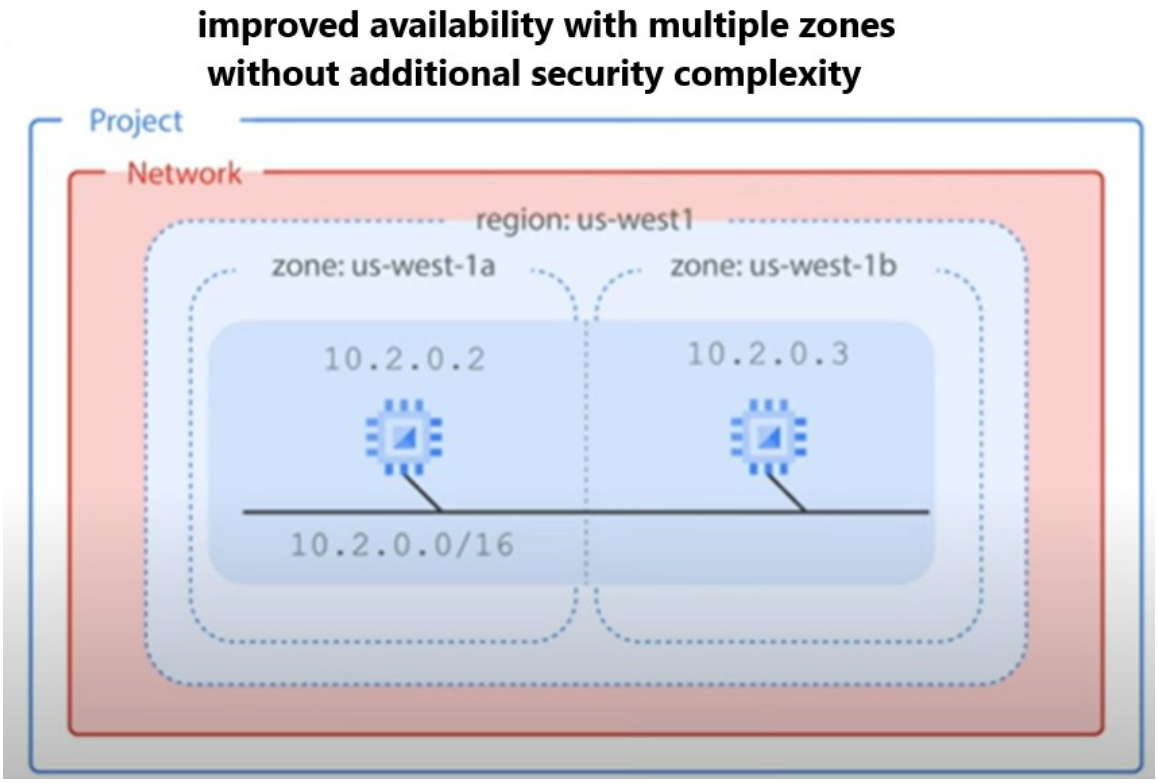
  shielded_instance_config {
    enable_integrity_monitoring = true
    enable_secure_boot          = false
    enable_vtpm                 = true
  }

  zone = "us-central1-c"
}
```

privatenet-us-vm instance

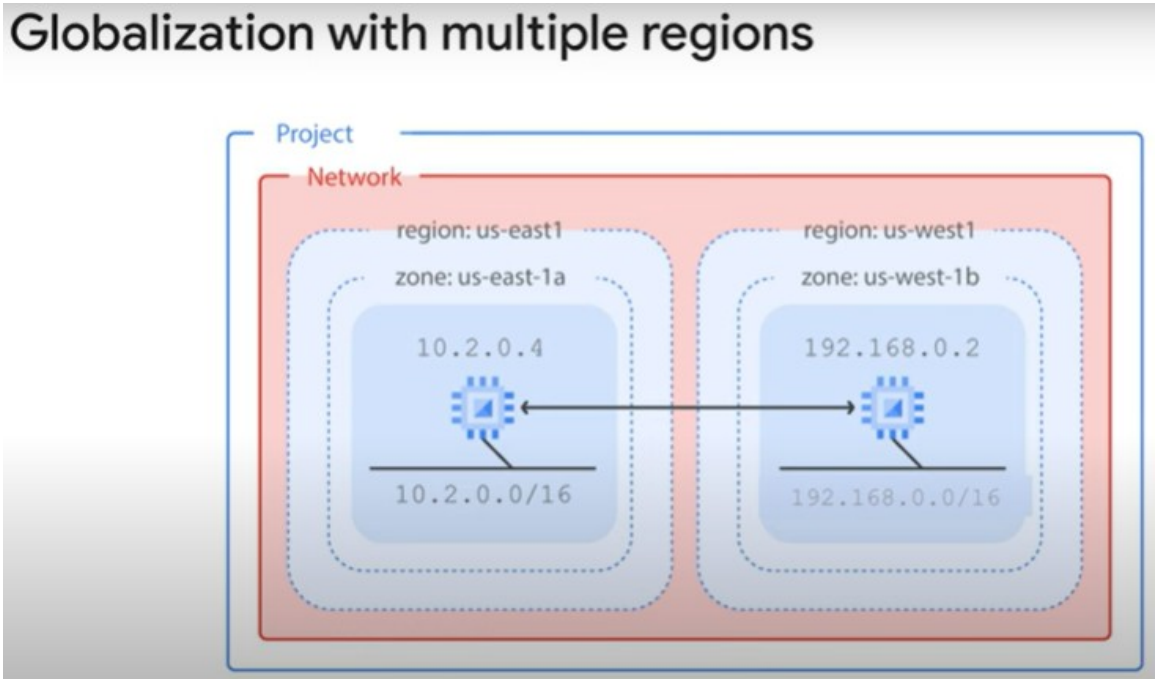
gcloud compute instances create privatenet-us-vm --zone=us-central1-c --machine-type=e2-micro --subnet=privatesubnet-us --image-family=debian-12 --image-project=debian-cloud --boot-disk-size=10GB --boot-disk-type=pd-standard --boot-disk-device-name=privatenet-us-vm

Common network designs



Many zones:

isolation for many types of infrastructure, hardware and software failures.



Multi region:

higher degree of failure independence.

resources spread across different failure domains.

When using a global load balancer like the HTTP load balancer, you can route traffic to the region that is closest to the user.

result in better latency for users and lower network traffic costs for your project.

Security:

security best practice, assign internal IP addresses to your VM instances whenever possible.

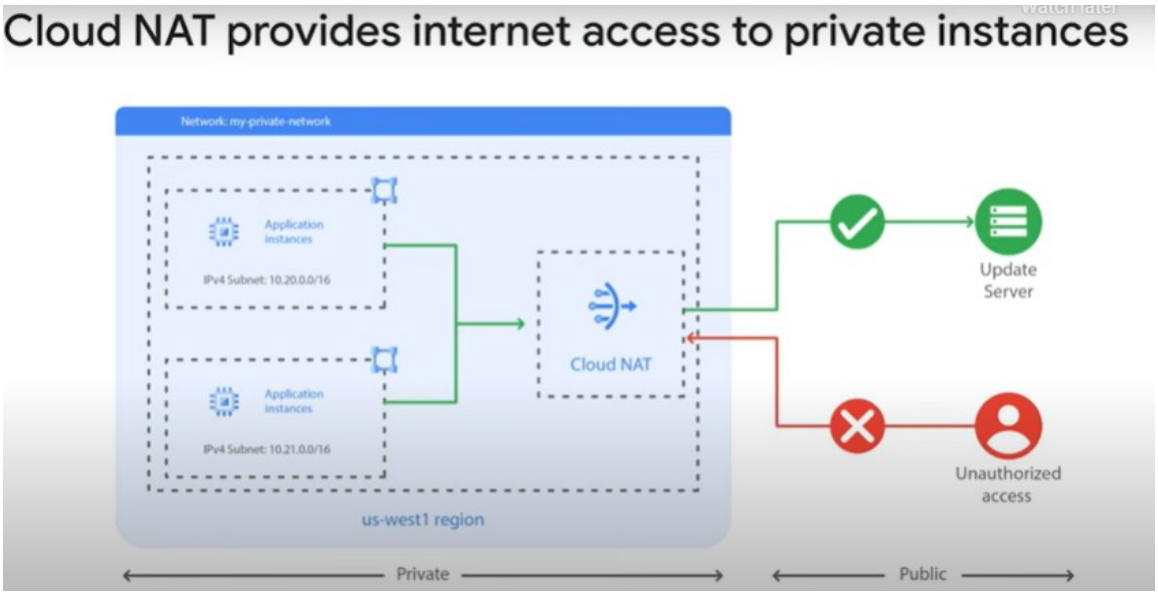
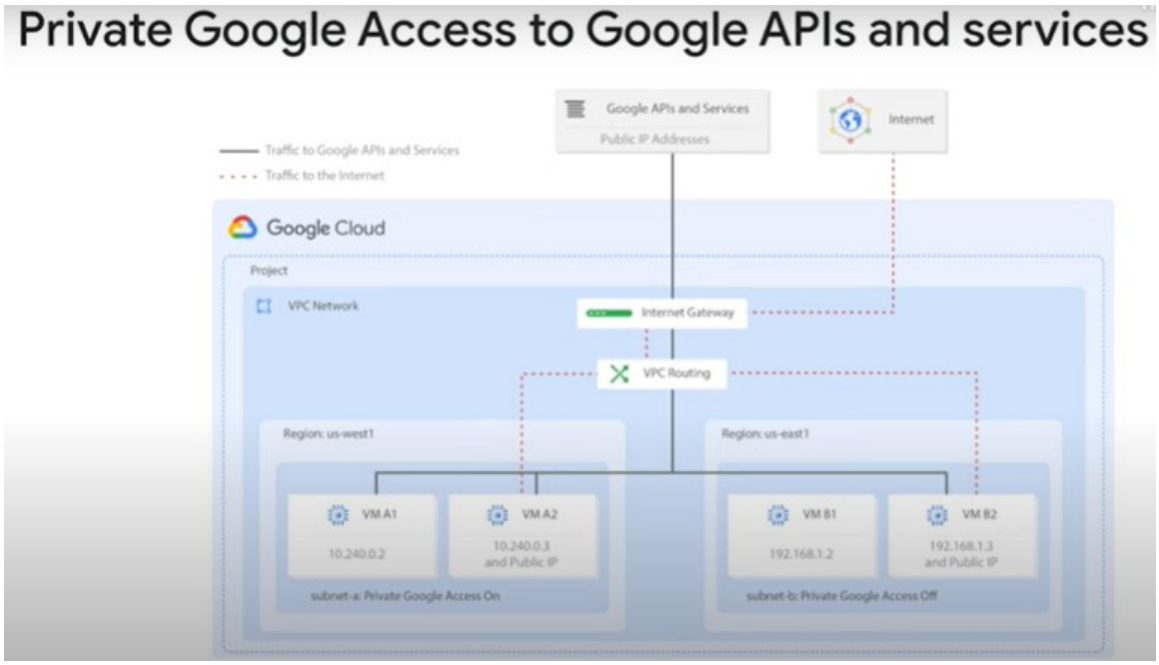
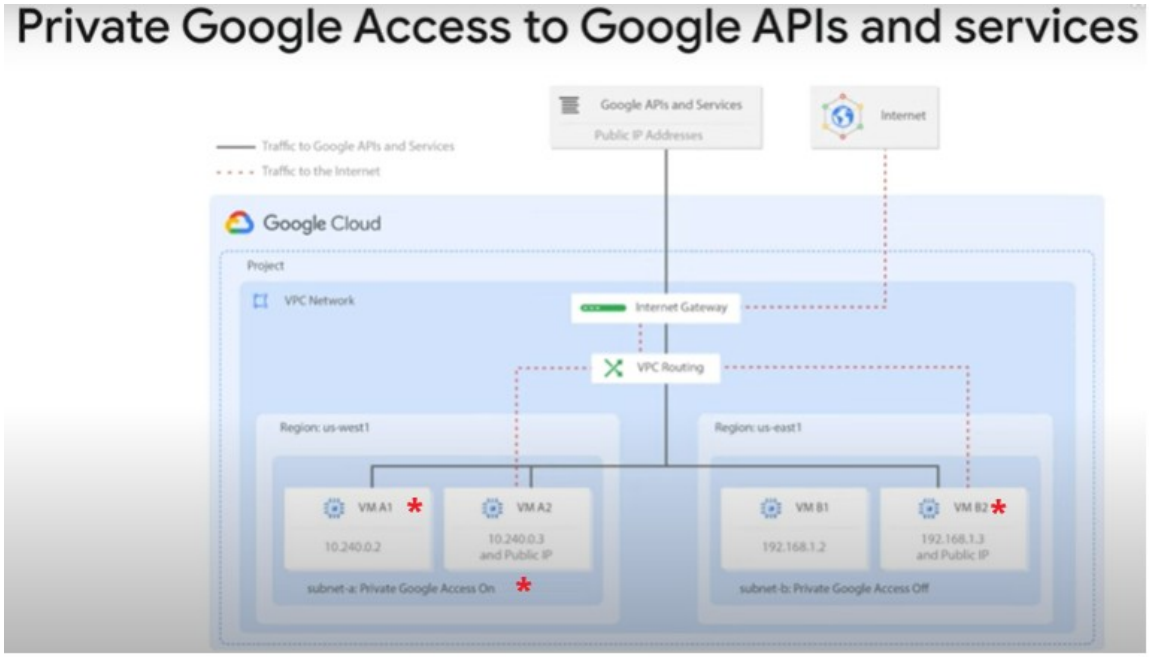


Figure 1 outbound NAT and inbound NAT - Cloud NAT does not Implement inbound NAT !!!!..





*** private Google access enabled -**
this is why VM A1 can access the internet
while VM B2 can not access

On the subnet – for internal ip addresses – where we define NAT,

If we need that the VM will be able to go out, we need to set private google access enabled on the subnet !!

Keep in mind that NAT is just outbound !!

LAB – Internal address + proxy (IAP = Identity-Aware Proxy) tunnel + NAT

https://www.cloudskillsboost.google/course_templates/50/labs/485523

https://www.cloudskillsboost.google/course_templates/50/video/485524

- Configure a VM instance that doesn't have an external IP address
- Connect to a VM instance using an Identity-Aware Proxy (IAP) tunnel
- Enable Private Google Access on a subnet
- Configure a Cloud NAT gateway
- Verify access to public IP addresses of Google APIs and services and other connections to the internet

IAP tunnel (open ssh from 35.235.240.0/20**):**

Note: In order to connect to your private instance using SSH, you need to open an appropriate port on the firewall. IAP connections come from a specific set of IP addresses (35.235.240.0/20). Therefore, you can limit the rule to this CIDR range.

Firewall rule on the network `privatenet`

```
gcloud compute --project=qwiklabs-gcp-01-44e78ef149da firewall-rules create
privatenet-allow-ssh --direction=INGRESS --priority=1000 --network=privatenet
--action=ALLOW --rules=tcp:22 --source-ranges=35.235.240.0/20
```

`create VM with external-ip= None:`

```
gcloud compute instances create vm-internal --project=qwiklabs-gcp-01-
44e78ef149da --zone=us-west1-b --machine-type=e2-medium --network-
interface=stack-type=IPV4_ONLY,subnet=privatenet-us,no-address --
metadata=enable-oslogin=true --maintenance-policy=MIGRATE --provisioning-
model=STANDARD --service-account=1014174709643-
compute@developer.gserviceaccount.com
--scopes=https://www.googleapis.com/auth/devstorage.read_only,https://
www.googleapis.com/auth/logging.write,https://www.googleapis.com/auth/
monitoring.write,https://www.googleapis.com/auth/
service.management.readonly,https://www.googleapis.com/auth/
servicecontrol,https://www.googleapis.com/auth/trace.append --create-disk=auto-
delete=yes,boot=yes,device-name=vm-internal,image=projects/debian-cloud/
global/images/debian-12-bookworm-v20241210,mode=rw,size=10,type=pd-balanced --
no-shielded-secure-boot --shielded-vtpm --shielded-integrity-monitoring --
labels=goog-ec-src=vm_add-gcloud --reservation-affinity=any
```

```
connect to this VM:
gcloud compute ssh vm-internal --zone us-west1-b --tunnel-through-iap
```

Note: When instances do not have external IP addresses,

they can only be reached by other instances on the network

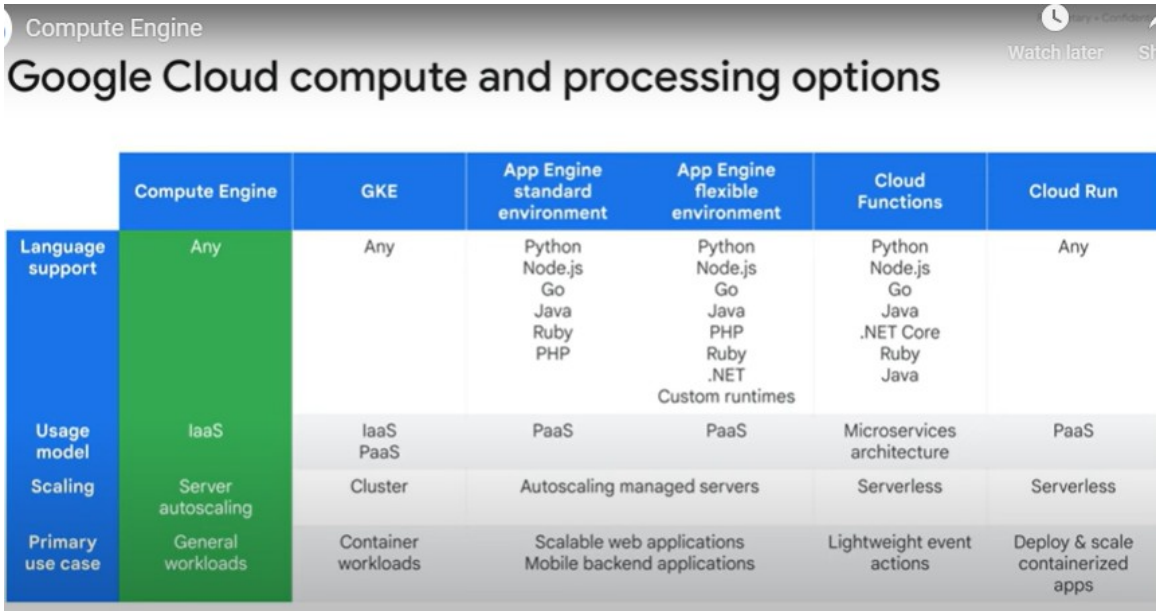
via a managed VPN gateway or via a Cloud IAP tunnel.

Cloud IAP enables context-aware access to VMs via SSH and RDP without bastion hosts.

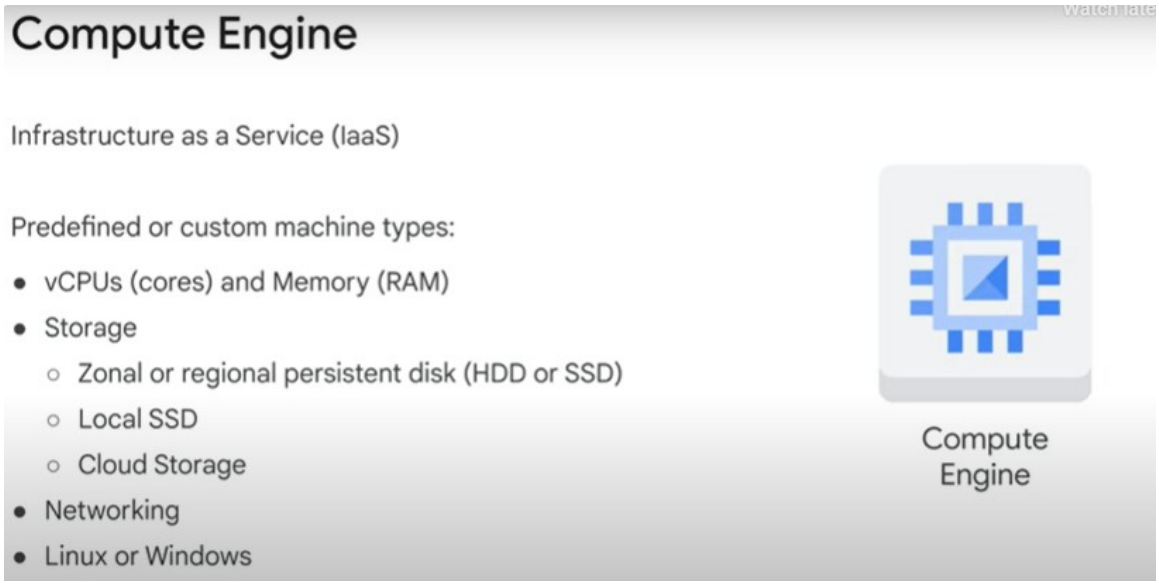
To learn more about this, see the blog post Cloud IAP enables context-aware access to VMs via SSH and RDP without bastion hosts.

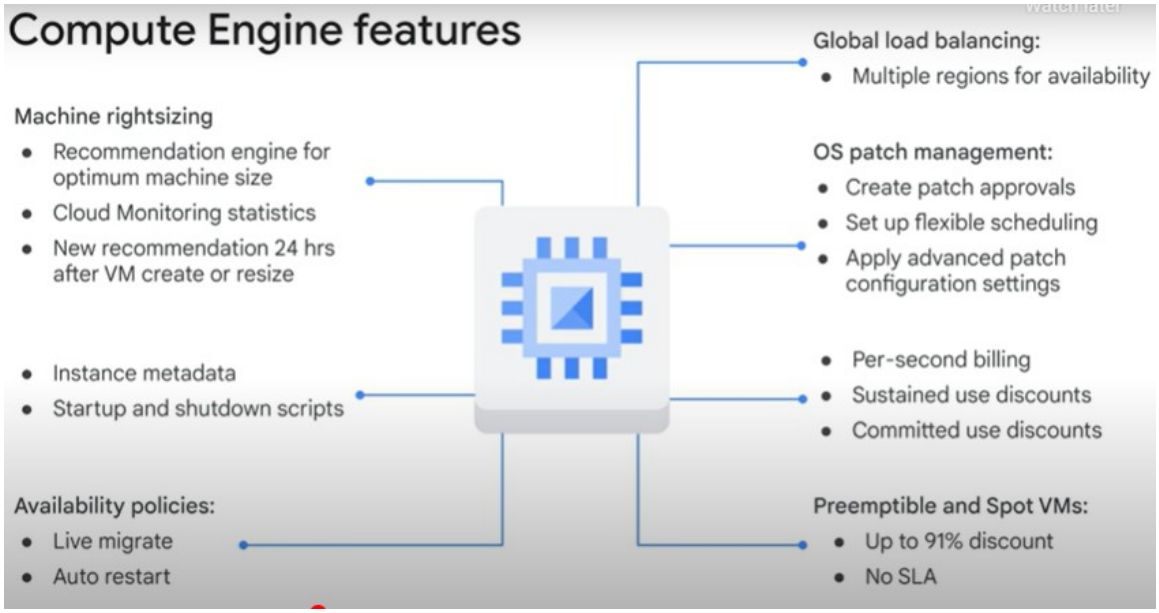
Virtual Machines

Compute Engine service



	Compute Engine	GKE	App Engine standard environment	App Engine flexible environment	Cloud Functions	Cloud Run
Language support	Any	Any	Python Node.js Go Java Ruby PHP	Python Node.js Go Java PHP Ruby .NET Custom runtimes	Python Node.js Go Java .NET Core Ruby Java	Any
Usage model	IaaS	IaaS PaaS	PaaS	PaaS	Microservices architecture	PaaS
Scaling	Server autoscaling	Cluster	Autoscaling managed servers		Serverless	Serverless
Primary use case	General workloads	Container workloads	Scalable web applications Mobile backend applications		Lightweight event actions	Deploy & scale containerized apps





TPU

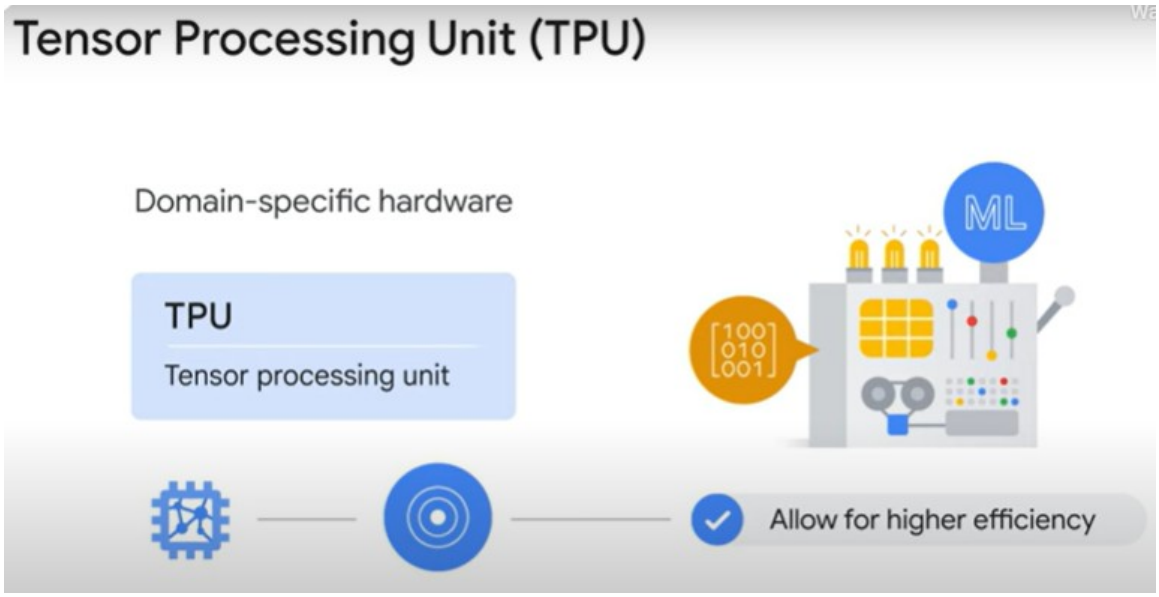
It is important to mention that hardware manufacturers have run up against limitations, and CPUs and GPUs,

can no longer scale to adequately reach the rapid demand for ML.

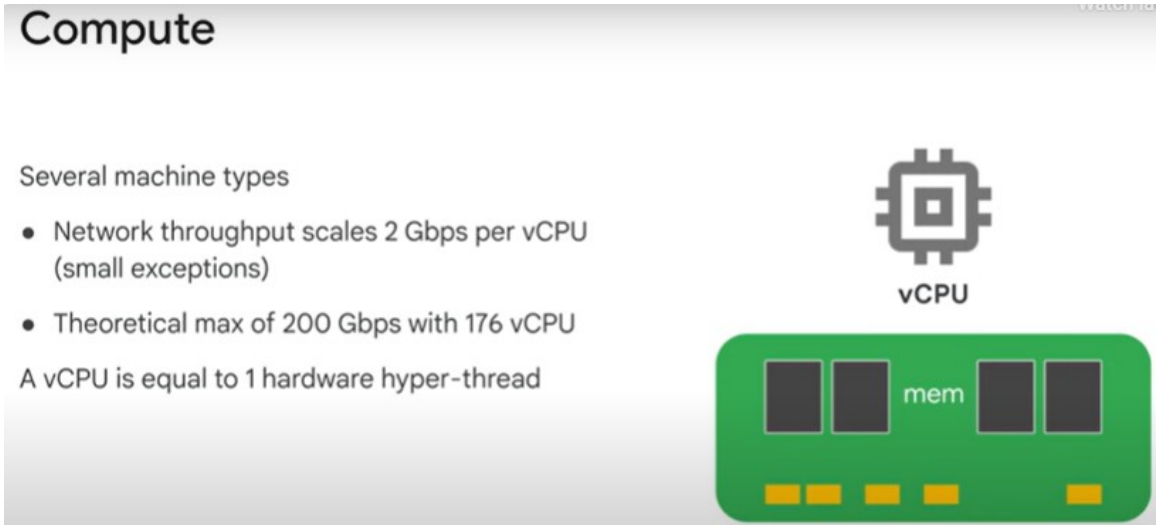
To help overcome this challenge, in 2016 Google introduced the Tensor Processing Unit, or TPU.

TPUs are Google's custom-developed application-specific integrated circuits (ASICs) used to accelerate machine learning workloads.

mostly recommended for models that train for long durations and for large models with large effective batch sizes.



network throughput (bandwidth) is **proportional to the number of virtual CPU (vCPU) cores** in certain cloud environments



Connection between Network and vCPU:

1. **vCPU and Network Throughput**:
    - In many cloud environments, **network bandwidth** is tied to the number of vCPUs in a virtual machine (VM).
    - For example:
        - A VM with 1 vCPU might have a maximum network bandwidth of **2 Gbps**.

        - A VM with 4 vCPUs would scale up to **8 Gbps**.
    - This scaling is typically linear but may vary depending on the VM type and cloud provider.

2. **Why Is Network Bandwidth Linked to vCPUs?**:
    - **Physical Constraints**: Network performance depends on the underlying hardware. Assigning more vCPUs often means the VM is backed by more powerful physical resources, including better network interfaces.
    - **Efficiency**: By tying network bandwidth to vCPUs, cloud providers ensure that VMs with higher processing power also get proportionally higher network throughput, preventing resource bottlenecks.
    - **Resource Balancing**: This approach ensures fair allocation of network bandwidth among VMs.

Scaling vCPU vs. Network Needs:

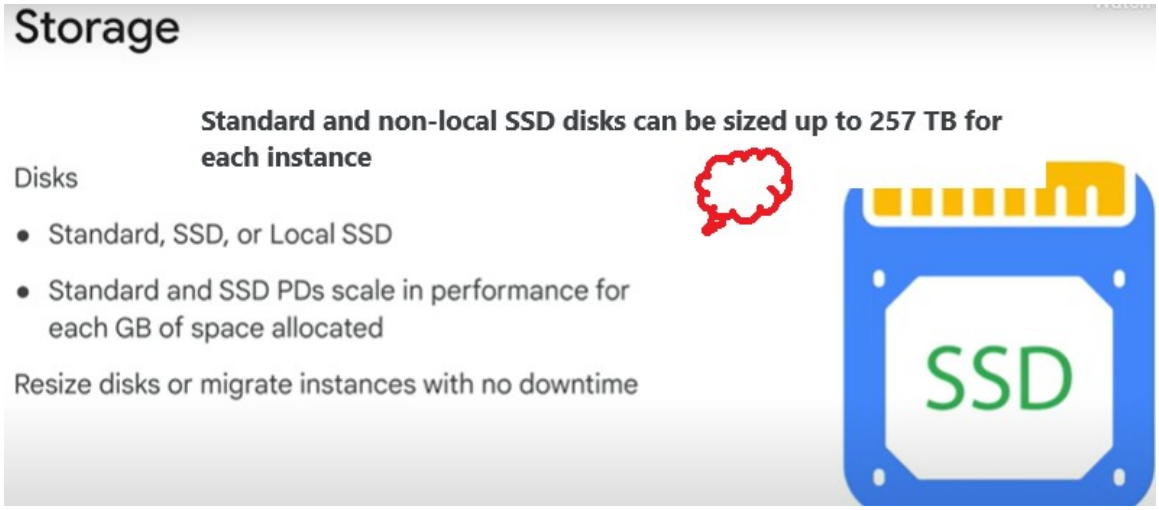
1. **When You Scale vCPUs**:
    - **Primary Driver**: You scale vCPUs based on CPU usage (e.g., when your application needs more processing power).
    - **Side Effect**: As you increase vCPUs, the available network bandwidth often increases proportionally.
    - **Network Is Not the Main Consideration**: In this case, network scaling is a secondary benefit that happens automatically.
2. **When You Scale for Network Throughput**:
    - **Primary Driver**: If your application requires higher network throughput (e.g., high data transfer rates), you may need to scale the VM by increasing vCPUs to achieve the desired bandwidth.
    - **CPU Usage May Not Be High**: Even if your application doesn't need additional compute resources, increasing vCPUs might be necessary to get the bandwidth you need.

Why It Matters:

- **Efficient Scaling**: If you're only concerned about compute power, focus on CPU usage. The network will scale automatically as a side effect.

- **Bandwidth-Limited Applications**: For applications like media streaming, data-intensive workloads, or network-heavy tasks, you might need to scale vCPUs specifically for bandwidth, even if the CPU usage itself is low.
- **Cost Implications**: Tying network performance to vCPUs means that increasing bandwidth might also increase costs because you're scaling other resources (like CPU and memory) that you may not fully utilize.

Storage



Standard and non-local SSD disks can be sized up to **257 TB for** each instance.

Local SSD

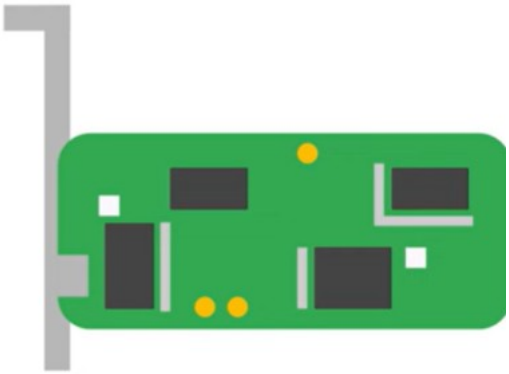
the data that you store on local SSDs persists only until you stop or delete the instance.

local SSD is used as a swap disk,

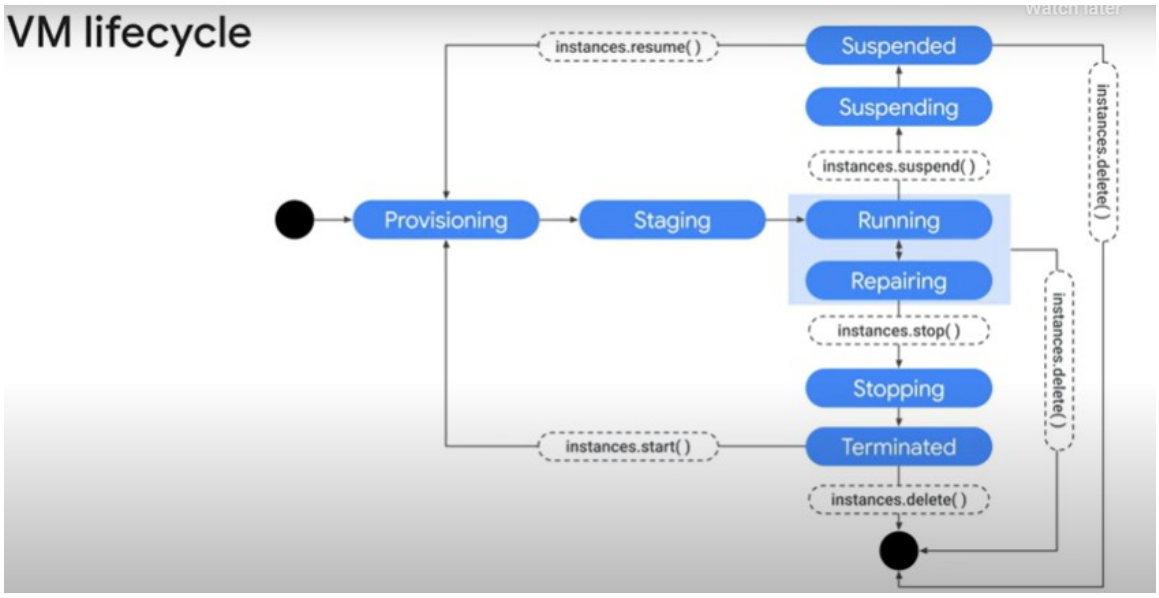
Networking

Robust networking features

- Auto, custom networks
- Inbound/outbound firewall rules
  - IP based
  - Instance/group tags
- Regional HTTPS load balancing
- Network load balancing
  - Does not require pre-warming
- Global and multi-regional subnetworks



VM lifecycle



While your instance is running, you can:

live migrate your virtual machine to another host in the same zone instead of requiring your instance to be rebooted

move your VM to a different zone

take a snapshot of the VM's persistent disk

export the system image

or reconfigure metadata.

stop your virtual machine:

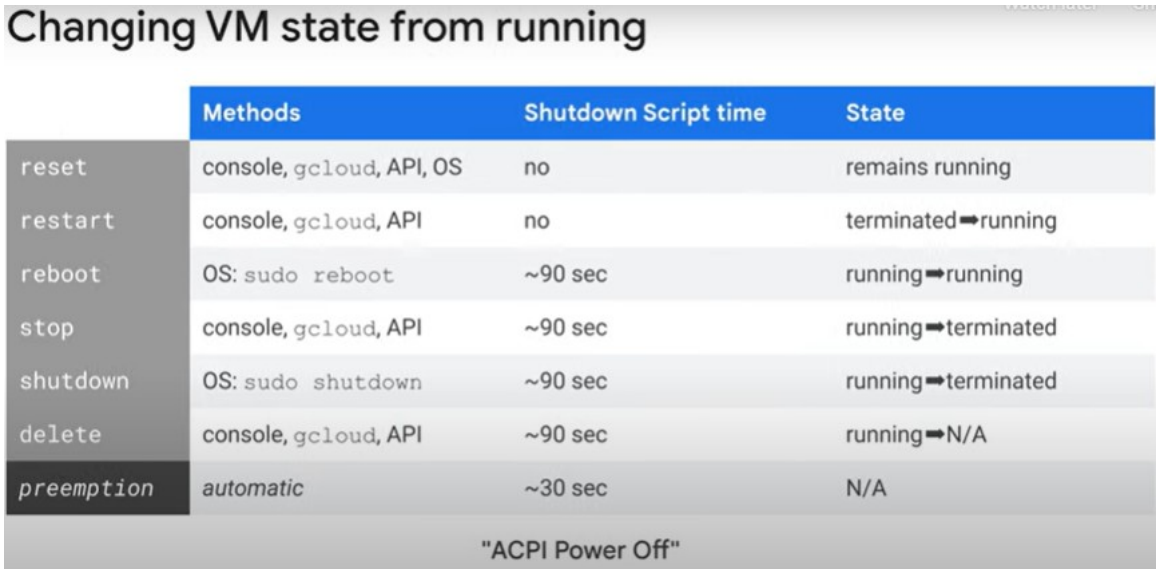
to upgrade your machine by adding more CPU

Repairing occurs

when the VM encounters an internal error

or the underlying machine is unavailable due to maintenance.

Not billed



Patch management service

Patch management is an essential part of managing an infrastructure

Manage OSes easily through Google Cloud.

- Keep infrastructures up-to-date.
- Reduce the risk of security vulnerabilities.

OS patch management:

- Patch compliance reporting
- Patch deployment

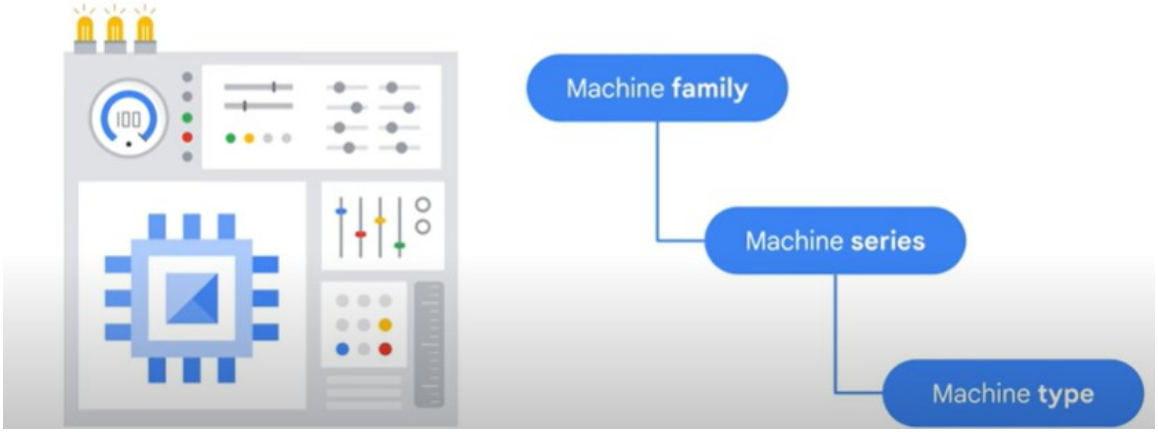


Actions supported on a terminated VM

- Change the machine type
- Migrate the VM instance to another network
- Add or remove attached disks; change auto-delete settings
- Modify instance tags
- Modify custom VM or project-wide metadata
- Remove or set a new static IP
- Modify VM availability policy
- Can't change the image of a stopped VM

Create VM

Machine type structure



Standard E2 VMs have between 2 to 32 vCPUs with a ratio of 0.5 GB to 8 GB of memory per vCPU.

Family: general-purpose -> series: E2 machine series -> type: e2-micro / etc

General-purpose machine family

Series	Workload	Applications	
E2	(Cost-optimized) Day-to-day computing at a lower cost	• Web serving • App serving • Back office apps • Small-medium databases	• Microservices • Virtual desktops • Development environments
N2, N2D, N1	(Balanced) Balanced price/performance across a wide range of VM shapes	• Web serving • App serving • Back office apps	• Medium-large databases • Cache • Media/streaming
Tau T2D, Tau T2A	(Scale-out optimized) Best performance/ cost for scale-out workloads	• Scale-out workloads • Web serving • Containerized microservices	• Media transcoding • Large-scale Java applications

Compute-optimized machine family

Series	Workload	Applications	
C2	Ultra high performance for compute-intensive workloads	• Compute-bound workloads • High-performance web serving • Gaming (AAA game servers) • Ad serving	• High-performance computing (HPC) • Media transcoding • AI/ML
C2D	Ultra high performance for compute-intensive workloads	• Memory-bound workloads • Gaming (AAA game servers) • High-performance computing (HPC)	• High-performance databases • Electronic Design Automation (EDA) • Media transcoding
H3	Ultra high performance for compute-intensive workloads	• High-performance computing (HPC)	• Electronic Design Automation (EDA)

Memory-optimized machine family

Series	Workload	Applications	
M1	Ultra high-memory workloads	• Medium in-memory databases such as SAP HANA • Tasks that require intensive use of memory with higher memory-to-vCPU ratios than the general-purpose high-memory machine types.	• In-memory databases and in-memory analytics, business warehousing (BW) workloads, genomics analysis, SQL analysis services. • Microsoft SQL Server and similar databases.
M2	Ultra high-memory workloads	• Large in-memory databases such as SAP HANA	• In-memory databases and in-memory analytics, business warehousing (BW) workloads, genomics analysis, SQL analysis services, etc.

Accelerator-optimized machine family

Series	Workload	Applications
A2	Optimized for high-performance computing workloads	• CUDA-enabled ML training and inference • HPC • Massive parallelized computation
G2	Optimized for high-performance computing workloads	• Video transcoding • Remote visualization workstation

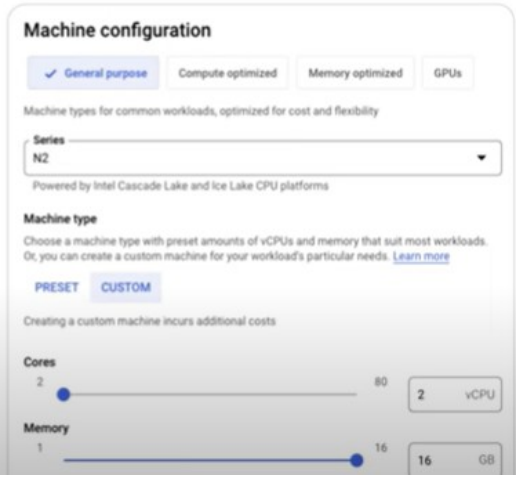
Creating custom machine types

When to select custom:

- Requirements fit between the predefined types
- Need more memory or more CPU

Customize the amount of memory and vCPU for your machine:

- Either 1 vCPU or even number of vCPU
- Up to 8 GB per vCPU
- Total memory must be multiple of 256 MB



Pricing

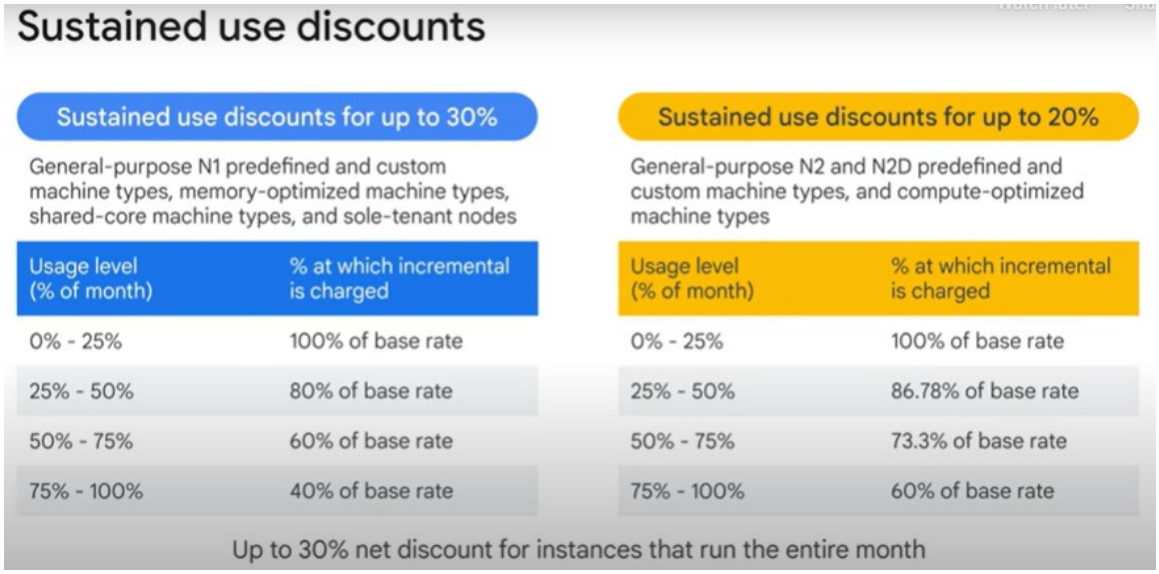
- Per-second billing, with minimum of 1 minute
  - vCPUs, GPUs, and GB of memory
- Resource-based pricing:
  - Each vCPU and each GB of memory is billed separately
- Discounts:
  - Sustained use
  - Committed use
  - Preemptible VM instances
- Recommendation Engine
  - Notifies you of underutilized instances
- Free usage limits

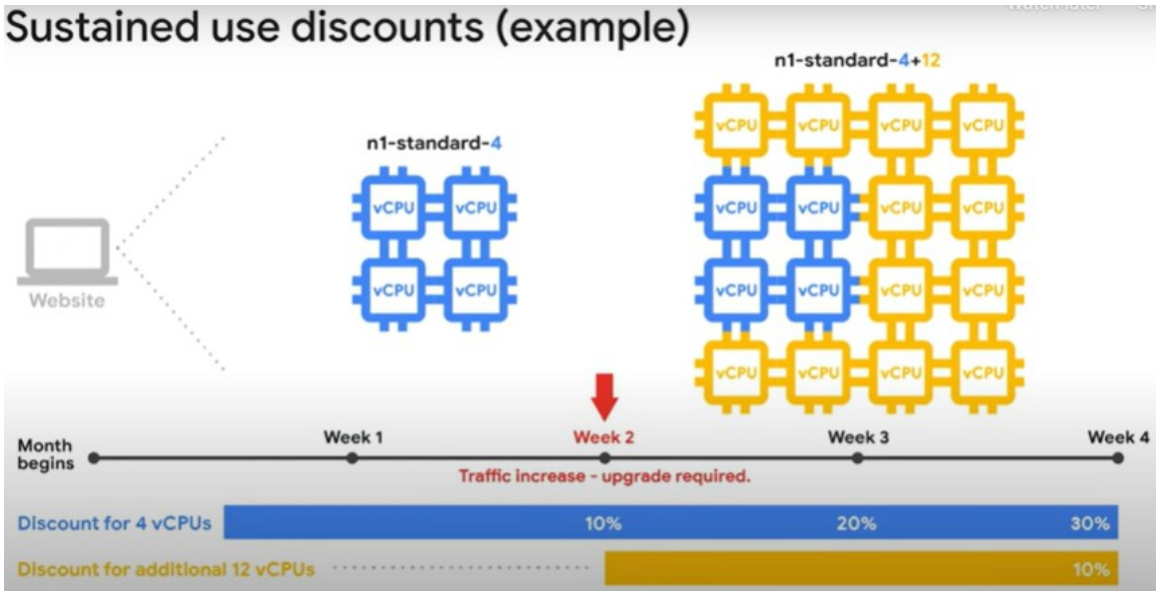
Preemptible and Spot VMs

provide cost-effective compute options

Key Differences between preemptible VMs and Spot VMs

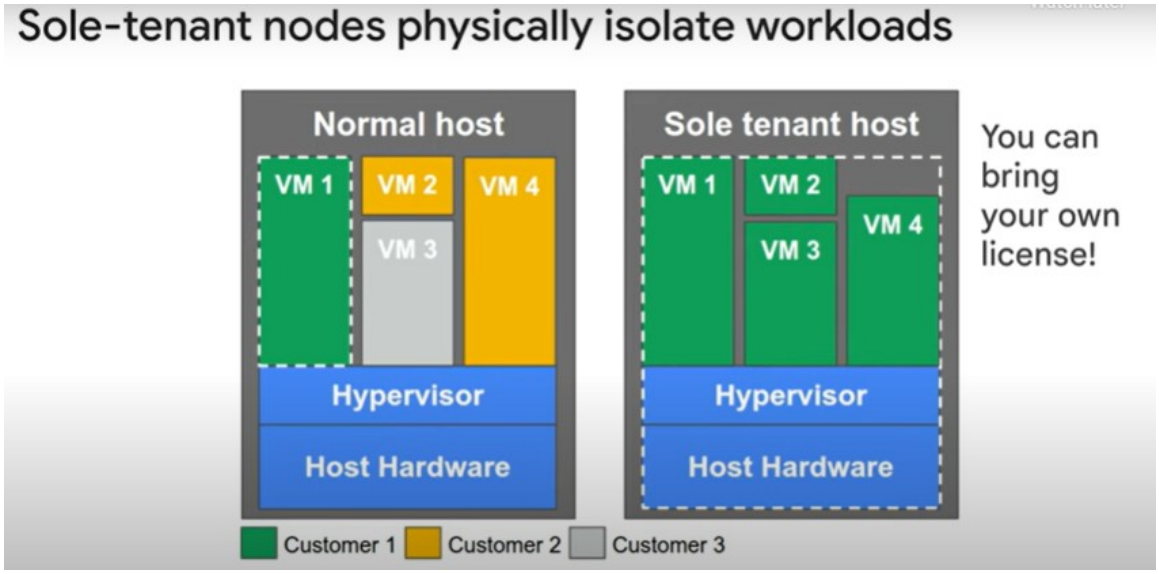
Feature	Preemptible VMs	Spot VMs
Maximum Lifetime	24 hours	No time limit
Pricing	Fixed	Variable (dynamic, based on resource demand)
Replacement	Cannot automatically restart after preemption	Can be automatically restarted if capacity becomes available
Use Cases	Legacy option for cost-saving VMs	Preferred option due to flexibility





Sole-tenant nodes

Use case: payment processing workload that needs to be isolated to meet compliance requirements.



shielded VM - more secure foundation

1. **Verifiable Integrity**:

- Shielded VMs use **integrity monitoring** to detect and prevent tampering with the bootloader, kernel, and critical boot components.
- GCP provides tools like **integrity validation** logs, allowing users to verify the VM's state and confirm that it has booted in a trusted environment.

2. **vTPM Shielding**:
   - Shielded VMs include a **virtual Trusted Platform Module (vTPM)**.
   - vTPM generates and securely stores cryptographic keys, which can only be used if the VM passes its integrity checks.
   - This shielding ensures that data or keys cannot be accessed or misused if the VM is compromised.
3. **Secure Boot**:
   - Secure Boot ensures the VM only runs signed and trusted OS bootloaders, kernels, and drivers. This prevents attackers from injecting malicious code during the boot process.
4. **Integrity Monitoring**:
   - Monitors the VM for integrity violations using measured boot values.
   - Alerts can be configured to notify administrators if any anomalies are detected.
5. **Kernel Integrity Protection**:
   - Shielded VMs protect against unauthorized kernel modifications. Only signed kernels are allowed to load, making it difficult for attackers to inject rootkits or exploit kernel vulnerabilities.

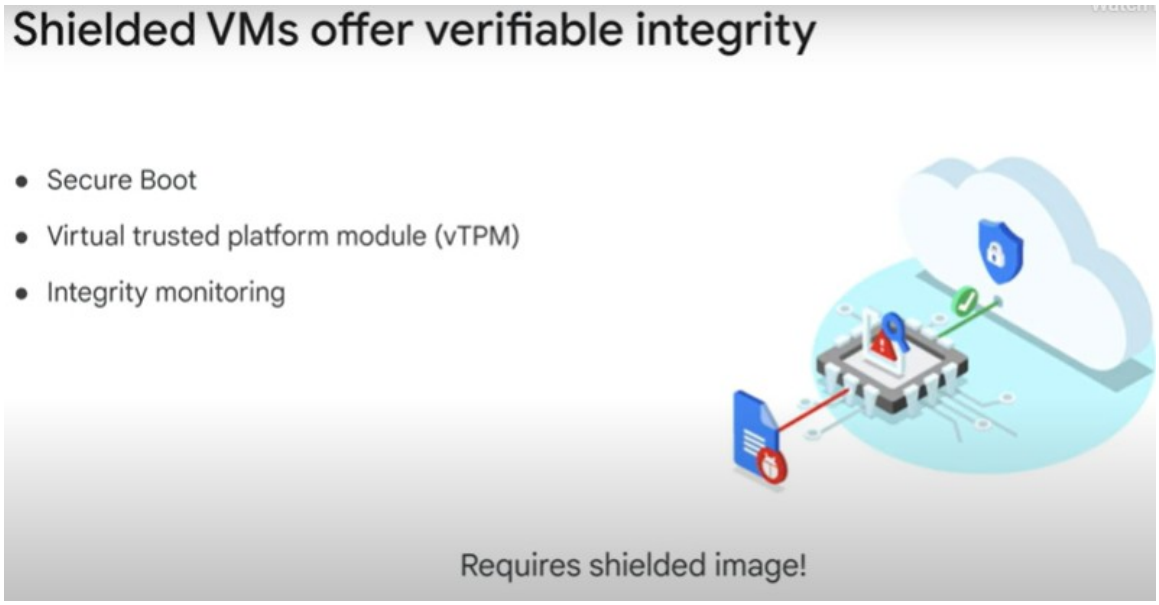
How Shielded VMs Prevent Data Exfiltration

1. **Data Sealing**:
   - Shielded VMs use **vTPM** to seal sensitive data, such as encryption keys or secrets, to the current VM state.
   - If an attacker tries to clone or migrate the VM to another environment, the sealed data becomes unusable.
2. **Integrity Enforcement**:
   - By enforcing boot integrity, Shielded VMs ensure the system is not tampered with at startup, preventing attackers from introducing malicious tools to exfiltrate data.
3. **Isolation**:

- Shielded VMs operate in a secure environment, minimizing exposure to external attack vectors.

Use Cases

- **Regulated Industries**: Financial, healthcare, and government sectors benefit from the enhanced security and compliance offered by Shielded VMs.
- **High-Security Applications**: Workloads requiring cryptographic operations, sensitive data storage, or proprietary intellectual property protection.
- **Compliance Requirements**: For meeting standards like PCI DSS, HIPAA, and FedRAMP.



Confidential VMs

They leverage confidential computing technology to isolate data in use by using secure enclaves, ensuring that the data remains encrypted not just at rest or in transit but also **during processing**.

1. **Memory Encryption**:
    - Confidential VMs use **hardware-based memory encryption** provided by AMD EPYC processors.
    - Data in memory is encrypted using **secure keys** that are generated and managed by the processor, ensuring no external access.
2. **Confidentiality During Processing**:

    - Protects sensitive data while it is being processed in memory, preventing unauthorized access, even from privileged users, GCP administrators, or attackers with root access.

3. **No Code Changes**:
    - Applications do not need to be rewritten or modified to take advantage of the protections offered by Confidential VMs. They work seamlessly with existing workloads.
4. **vTPM Integration**:
    - Confidential VMs include a **virtual Trusted Platform Module (vTPM)** to provide cryptographic assurance of the VM's integrity.
    - vTPM ensures secure boot and protects secrets.
5. **Isolation from GCP and Other Tenants**:
    - Confidential VMs ensure that data is not accessible to Google itself or other tenants in the cloud environment.
6. **Compliance and Security**:
    - Confidential VMs help organizations meet stringent compliance requirements, such as **GDPR**, **HIPAA**, and other data privacy regulations.

How Confidential VMs Work

- **Confidential Computing**: Confidential VMs are built on confidential computing technology that encrypts data in use.
- **Secure Enclaves**: Secure enclaves are isolated regions of memory that are encrypted and inaccessible to the host operating system or hypervisor.
- **AMD SEV (Secure Encrypted Virtualization)**: The technology leverages AMD Secure Encrypted Virtualization (SEV), which provides hardware-based encryption for VM memory.

Benefits of Confidential VMs

1. **Enhanced Data Security**:
    - Protects sensitive data from unauthorized access, even during processing.
2. **Ease of Adoption**:
    - No changes to existing applications or workflows are required.
3. **Zero Trust Model**:

- Helps implement a **zero trust** security model, where no user or process is inherently trusted.

4. **Regulatory Compliance**:
    - Simplifies achieving compliance for workloads handling regulated data.
5. **Integration with Other GCP Services**:
    - Works seamlessly with other GCP products like BigQuery, Google Kubernetes Engine (GKE), and Google Cloud Storage.

Use Cases

1. **Financial Services**:
    - Protecting sensitive customer data while processing financial models or conducting transactions.
2. **Healthcare**:
    - Safeguarding patient information during medical research or diagnostics.
3. **Machine Learning**:
    - Ensuring the confidentiality of training data and machine learning models.
4. **Intellectual Property Protection**:
    - Securing proprietary algorithms and data during computation.

How to Enable Confidential VMs

1. Create a VM instance through the GCP Console, CLI, or API.
2. When creating the VM:
    - Ensure the **Enable Confidential VM Service** option is selected.
    - Use machine types and images compatible with Confidential VMs.
3. Deploy your workloads without the need for code changes.

You can select the Confidential VM service when creating a new VM using the Google Cloud Console, the Compute Engine API, or the gcloud command-line tool.

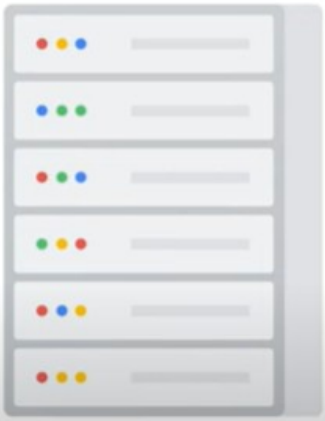
Confidential VMs allow you to encrypt data in use

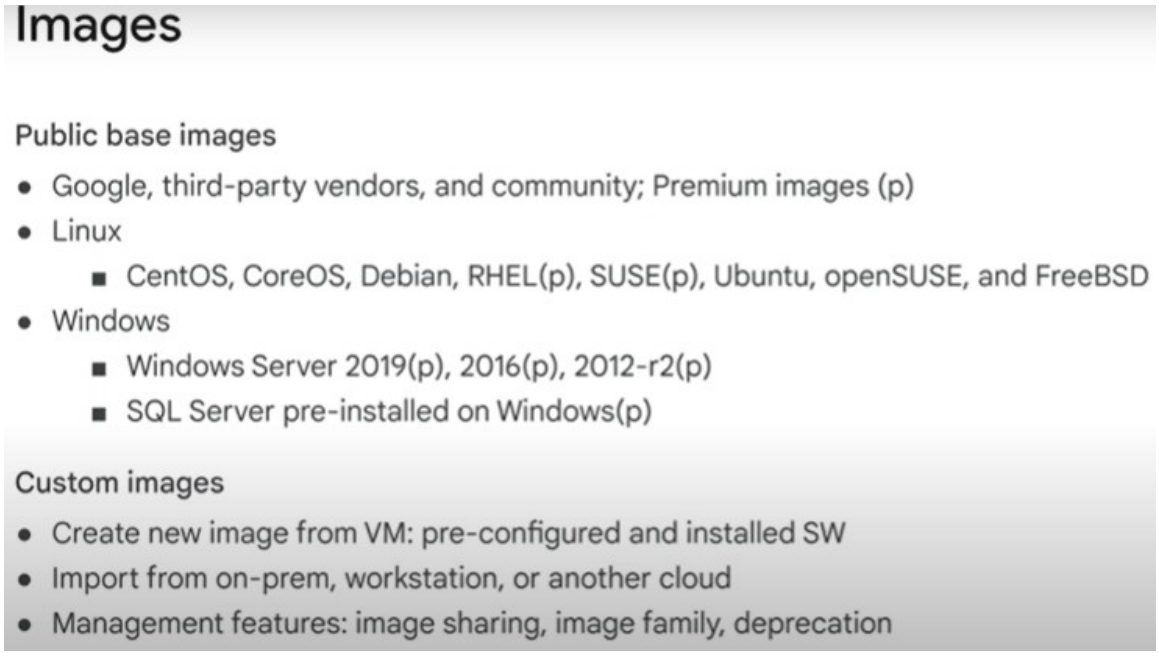
- Encrypts data while it's being processed.
- Easy to use with no changes to code or performance compromise.
- N2D Compute Engine VM running on second generation AMD Epyc processors.
- Provides high memory capacity, high throughput, and supports parallel and compute heavy workloads.

Images

What's in an image?

- Boot loader
- Operating system
- File system structure
- Software
- Customizations

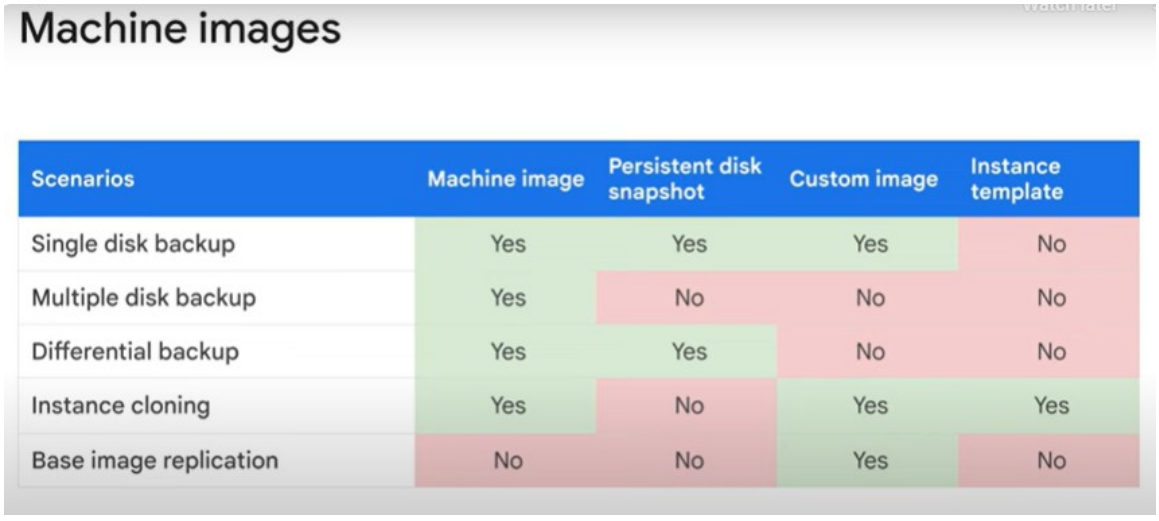




Some of these images are premium images, as indicated in parentheses with a p.

These images will have per-second charges after a 1-minute minimum, with the exception of SQL Server images, which are charged per minute after a 10-minute minimum.

A machine image is a Compute Engine resource that stores all the configuration, metadata, permissions, and data from one or more disks required to create a virtual machine (VM) instance.



Disk options

Every single VM comes with a single root persistent disk, because you're choosing a base image to have that loaded on.

Boot disk

- VM comes with a single root persistent disk.
- Image is loaded onto root disk during first boot:
  - Bootable: you can attach to a VM and boot from it.
  - Durable: can survive VM terminate.
- Some OS images are customized for Compute Engine.
- Can survive VM deletion if "Delete boot disk when instance is deleted" is disabled.

Persistent disks

Network storage appearing as a block device

- Attached to a VM through the network interface
- Durable storage: can survive VM terminate
- Bootable: you can attach to a VM and boot from it
- Snapshots: incremental backups
- Performance: Scales with size
- HDD (magnetic) or SSD (faster, solid-state) options
- Disk resizing: even running and attached!
- Can be attached in read-only mode to multiple VMs
- Zonal or Regional
  - `pd-standard`
  - `pd-ssd`
  - `pd-balanced`
  - `pd-extreme` (zonal only)
- Encryption keys:
  - Google-managed
  - Customer-managed
  - Customer-supplied

Regional persistent disks provide active-active disk replication across two zones in the same region

Regional persistent disks deliver durable storage that is synchronously replicated across zones and are **a great option for high-performance databases** and enterprise **applications that also require high availability**.

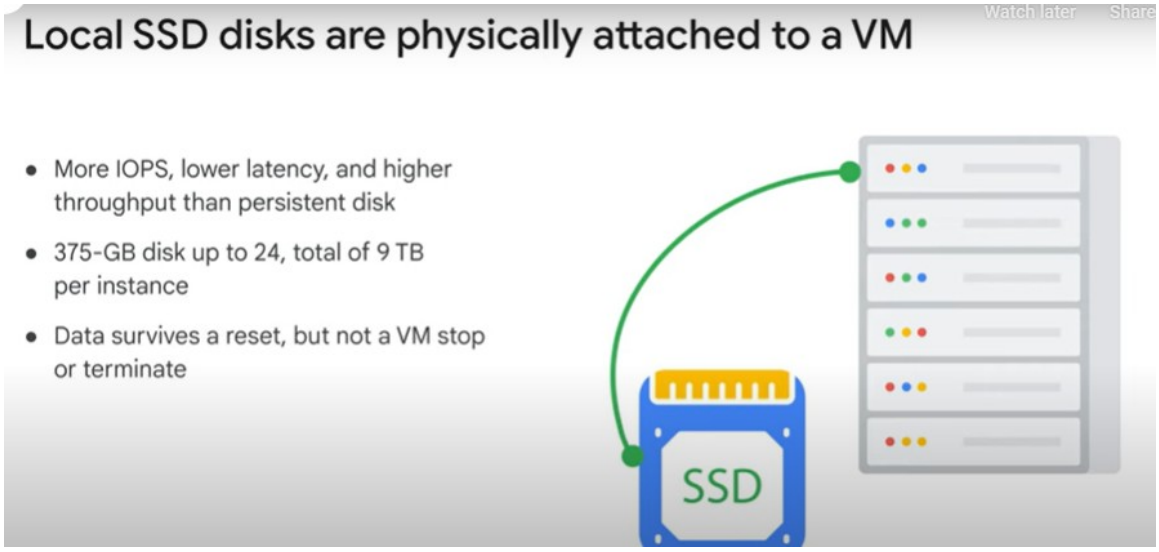
pd-satndard : Standard persistent disks are backed by standard hard disk drives and are suitable for large data processing workloads that primarily use sequential I/Os

pd-ssd - Performance SSD persistent disks are backed by solid-state drives and are suitable for enterprise applications and high-performance databases that require lower latency and more IOPS than standard persistent disks provide.

Balanced-ssd - Balanced persistent disks are also backed by solid-state drives.

They are an alternative to SSD persistent disks that balance performance and cost.

Local SSD disks



Currently, local SSDs are 375 gigabytes in size and you can attach up to to 24 local SSD partitions for a total of 9 terabytes per instance.

Data on these disks will survive a reset but not a VM stop or terminate, because these disks can't be reattached to a different VM.

RAM disk

- `tmpfs`
- Faster than local disk, slower than memory
  - Use when your application expects a file system structure and cannot directly store its data in memory
  - Fast scratch disk, or fast cache
- Very volatile; erase on stop or restart
- May need a larger machine type if RAM was sized for the application
- Consider using a persistent disk to back up RAM disk data

Summary of disk options

	Persistent disk HDD	Persistent disk SSD	Local SSD disk	RAM disk
Data redundancy	Yes	Yes	No	No
Encryption at rest	Yes	Yes	Yes	N/A
Snapshotting	Yes	Yes	No	No
Bootable	Yes	Yes	No	Not
Use case	General, bulk file storage	Very random IOPS	High IOPS and low latency	low latency and risk of data loss

Low Latency

- **Definition**:
  Latency refers to the time it takes for a storage device to complete a single operation (read or write).
    - **Low latency** means the operations are performed quickly, with minimal delay.

Maximum persistent disks

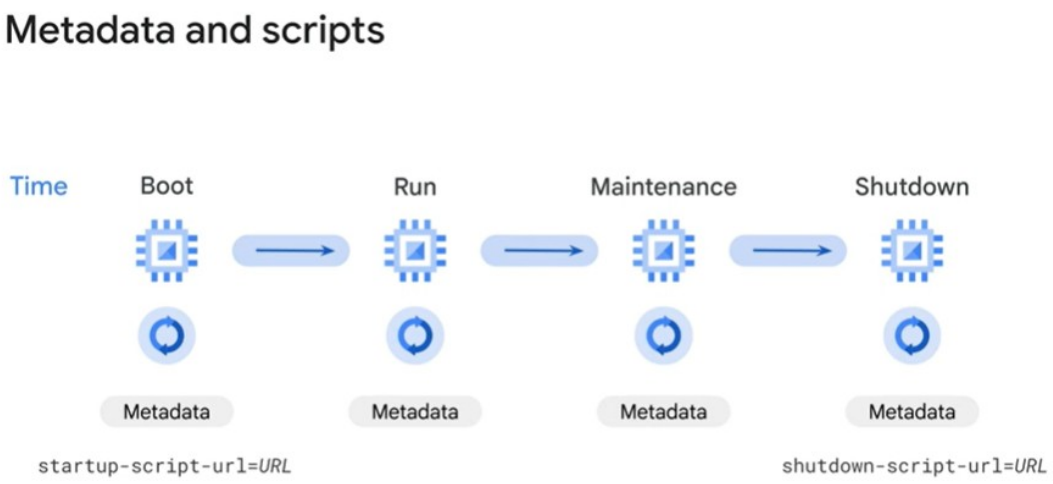
Machine type	Disk number limit
Shared-core	16
Standard	128
High-memory	
High-CPU	
Memory-optimized	
Compute-optimized	

So if you plan on having a large amount of Disk IO throughput, it will also compete with any network egress or ingress throughput.

Persistent disk management differences

Cloud Persistent Disk	Computer Hardware Disk
• Single file system is best • Resize (grow) disks • Resize file system • Built-in snapshot service • Automatic encryption	• Partitioning • Repartition disk • Reformat • Redundant disk arrays • Subvolume management and snapshots • Encrypt files before write to disk

Common Compute Engine actions



Step 1: Understand GCP Metadata Server

GCP instances can access their metadata via the **metadata server** at http://metadata.google.internal. The external IP address is accessible via the following metadata key:

bash

Copy code

/computeMetadata/v1/instance/network-interfaces/0/access-configs/0/external-ip

When making requests to the metadata server, you need to include a custom header:

makefile

Copy code

Metadata-Flavor: Google

Step 2: Write the Startup Script

Here's an example startup script that retrieves the external IP address and uses it to set up a database:

bash

Copy code

#!/bin/bash

Retrieve the external IP address of the instance

```
EXTERNAL_IP=$(curl -s -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/network-interfaces/0/access-configs/0/external-ip)

# Use the external IP to configure a database or any other setup
echo "The external IP is $EXTERNAL_IP" > /var/log/startup-script.log

# Example: Configure a database (MySQL in this case)
sudo apt update
sudo apt install -y mysql-server

# Allow connections from the external IP
sudo sed -i "s/bind-address.*/bind-address = $EXTERNAL_IP/" /etc/mysql/mysql.conf.d/mysqld.cnf

# Restart MySQL service to apply changes
sudo systemctl restart mysql

echo "Database configured to bind to $EXTERNAL_IP" >> /var/log/startup-script.log
```

This script:

1. Fetches the external IP from the metadata server.
2. Logs the IP to a file.
3. Installs MySQL.
4. Updates the MySQL configuration to bind to the external IP.
5. Restarts the MySQL service.

Step 3: Add the Startup Script to Your VM

You can add the startup script when creating a new VM or to an existing one.

Option A: During VM Creation

1. Go to the **GCP Console**.
2. Navigate to **Compute Engine** > **VM Instances** > **Create Instance**.
3. In the **Management, security, disks, networking, sole tenancy** section, expand **Management**.
4. In the **Startup script** field, paste the script.

Option B: To an Existing VM

1. Go to **Compute Engine** > **VM Instances**.
2. Click the name of the VM instance you want to modify.
3. Click **Edit**.
4. Scroll to the **Custom metadata** section.
5. Add a new key-value pair:
    - Key: startup-script
    - Value: Paste the script.
6. Save changes.

Step 4: Test the Startup Script

1. Restart the VM.
2. Verify that the startup script ran successfully by checking the logs:

bash

Copy code

sudo cat /var/log/startup-script.log

Step 5: Validate the Database Configuration

1. Confirm that MySQL is bound to the external IP:

bash

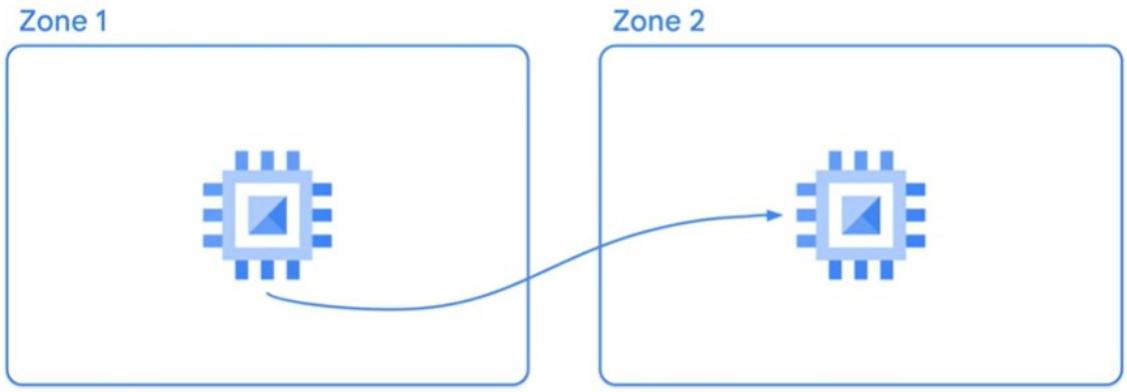
Copy code

sudo netstat -plunt | grep mysql

2. Test connectivity to the database using the external IP.

This approach ensures that the VM dynamically retrieves its external IP and uses it for setting up the database or other tasks.

Move an instance to a new zone

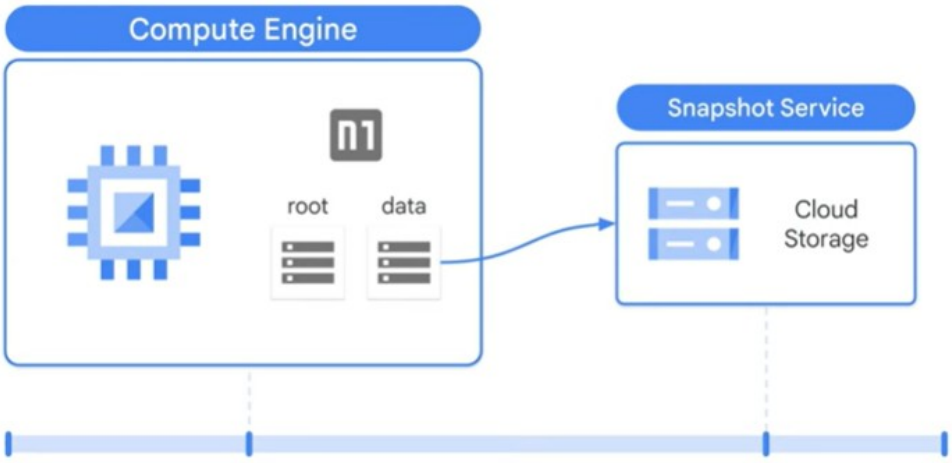


`gcloud compute instances move`

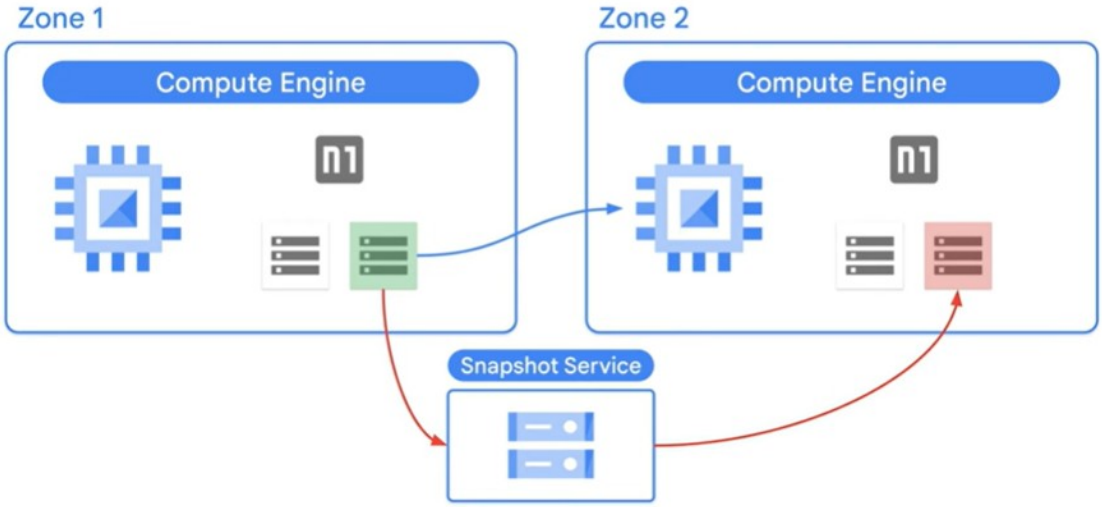
Move an instance to a new zone

- Automated process (moving within region):
  - `gcloud compute instances move`
  - Update references to VM; not automatic
- Manual process (moving between regions):
  - Snapshot all persistent disks on the source VM.
  - Create new persistent disks in destination zone restored from snapshots.
  - Create new VM in the destination zone and attach new persistent disks.
  - Assign static IP to new VM.
  - Update references to VM.
  - Delete the snapshots, original disks, and original VM.

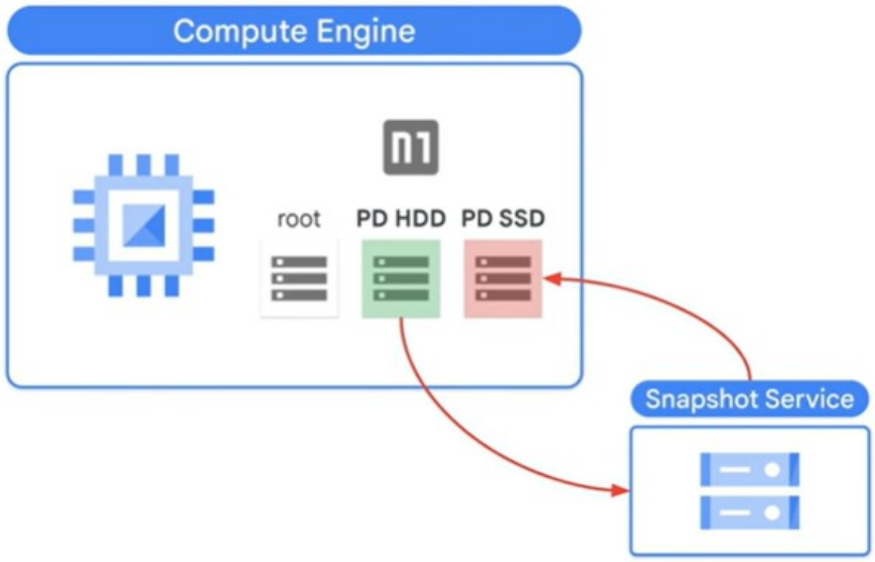
Snapshot: Backup critical data



Snapshot: Migrate data between zones



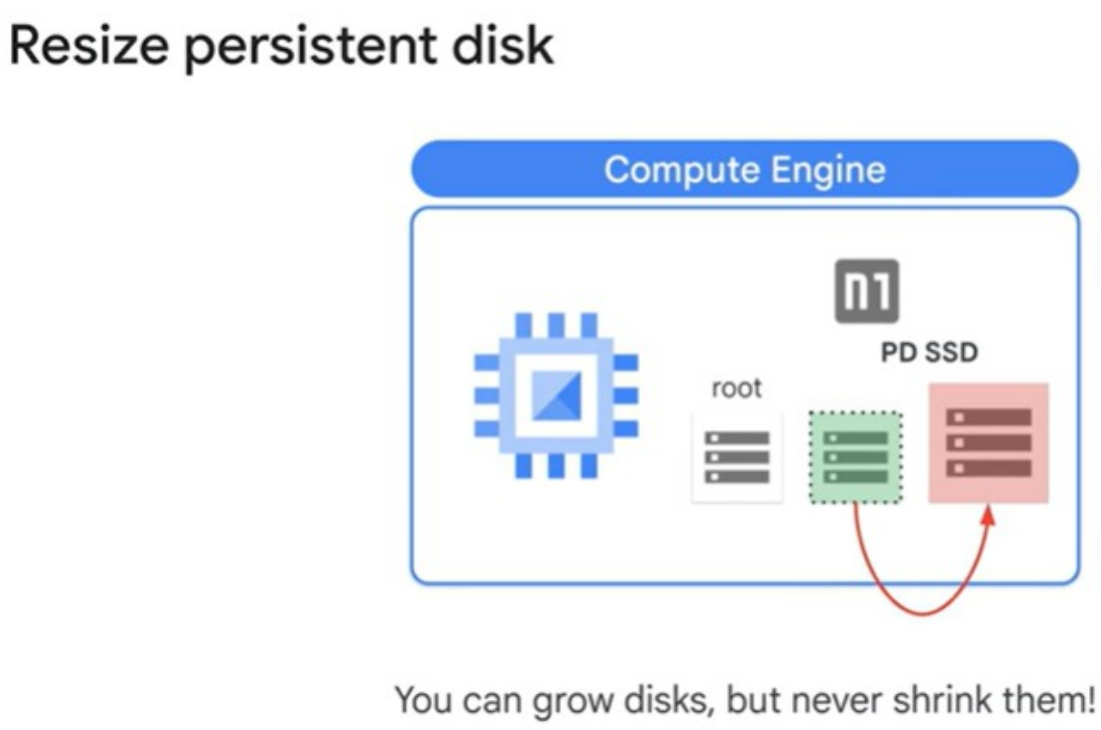
Snapshot: Transfer to SSD to improve performance



Persistent disk snapshots

- Snapshot is not available for local SSD.
- Creates an *incremental* backup to Cloud Storage.
  - Not visible in your buckets; managed by the snapshot service.
- Create scheduled snapshots.
  - Regularly and automatically back up your zonal and regional persistent disks.
- Snapshots can be restored to a new persistent disk.
  - New disk can be in another region or zone in the same project.
  - Basis of VM migration: "moving" a VM to a new zone.
    - Snapshot doesn't backup VM metadata, tags, etc.

create a persistent disk snapshot, refer to the module PDF under Course Resources.



Quiz

What are sustained use discounts?

- Discounts you receive by using preemptible VM instances
- Automatic discounts that you get for running specific Compute Engine resources for a significant portion of the billing month
- Purchase commitments for specific resources you know you will use
- Per-second billing that starts after a 1 minute minimum

The correct definition of **sustained use discounts** is:

Automatic discounts that you get for running specific Compute Engine resources for a significant portion of the billing month.

Explanation:

- **Sustained use discounts** are a feature of Google Cloud's Compute Engine pricing. They automatically apply when you run certain types of resources (e.g., virtual machine instances) for a significant part of the billing month.
- These discounts do not require any prior commitment or manual activation.

- The longer you use the resource in a billing month, the larger the discount, up to a maximum percentage.

Other options describe different pricing features but do not match sustained use discounts:

- **Preemptible VMs**: Offer steep discounts for short-term, interruptible workloads but are unrelated to sustained use discounts.
- **Purchase commitments**: Refer to committed use discounts, where you prepay or commit to use a certain amount of resources for discounts.
- **Per-second billing**: Allows flexible billing but isn't related to sustained use discounts.

IAM

Identity Role Resource

Deploy cloudRun function

```
gcloud functions deploy nodejs-pubsub-function \
  --gen2 \
  --runtime=nodejs20 \
  --region=us-east4 \
  --source=. \
  --entry-point=helloPubSub \
  --trigger-topic cf-demo \
  --stage-bucket qwiklabs-gcp-00-8be4369343fc-bucket \
  --service-account cloudfunctionsa@qwiklabs-gcp-00-8be4369343fc.iam.gserviceaccount.com \
  --allow-unauthenticated
```

Key Components

1. **`gcloud functions deploy`**
    - This command is used to deploy a Cloud Function in GCP.

2. **`nodejs-pubsub-function`**
    - The name of the Cloud Function being deployed.

3. **`--gen2`**
    - Indicates that this is a **second-generation** Cloud Function. Second-generation Cloud Functions provide improved scalability, better control over networking, and tighter integration with Google Cloud services.

4. **`--runtime=nodejs20`**
    - Specifies the runtime environment for the function. In this case, it's **Node.js 20**.

5. **`--region=us-east4`**
    - Specifies the region where the function is deployed. Here, it's **us-east4** (Northern Virginia).

6. **`--source=.`**
    - Indicates the source code location for the Cloud Function. A `.` means the current directory.

7. **`--entry-point=helloPubSub`**
    - Specifies the entry point of the function. This is the name of the function in your source code that handles the Pub/Sub event.

8. **`--trigger-topic=cf-demo`**
    - Sets the trigger for the function to be a **Pub/Sub topic** named `cf-demo`. When a message is published to this topic, the function is invoked.

9. **`--stage-bucket=qwiklabs-gcp-00-8be4369343fc-bucket`**
    - Specifies the staging bucket used during deployment. This bucket temporarily stores files and artifacts required for the deployment process.

10. **`--service-account=cloudfunctionsa@qwiklabs-gcp-00-8be4369343fc.iam.gserviceaccount.com`**
    - Specifies the **service account** under which the Cloud Function runs. This service account determines the permissions the function has when accessing other GCP resources.

11. **`--allow-unauthenticated`**
    - Allows the function to be invoked without requiring authentication. This is generally used for publicly accessible endpoints.

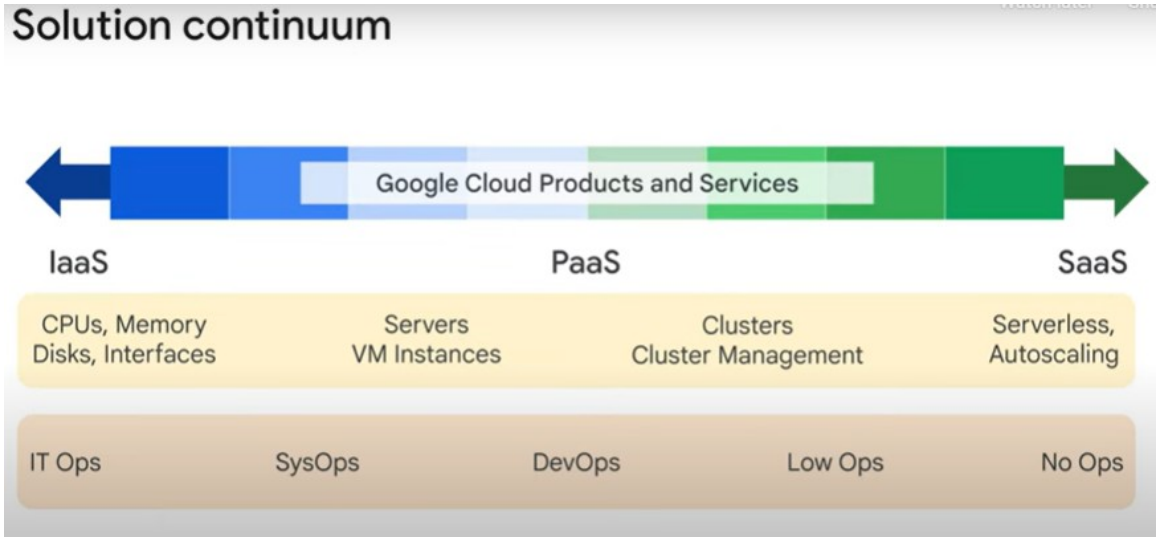
Workflow of This Deployment

1. **Trigger Setup:**
    - The function is triggered by messages published to the Pub/Sub topic `cf-demo`.

2. **Execution:**
    - When a message is published to the topic, GCP invokes the function by calling the `helloPubSub` entry point.

Core services - W3

Introduction



Cloud Run, a managed compute platform that lets you run stateless containers via web requests or Pub/Sub events.

IAM

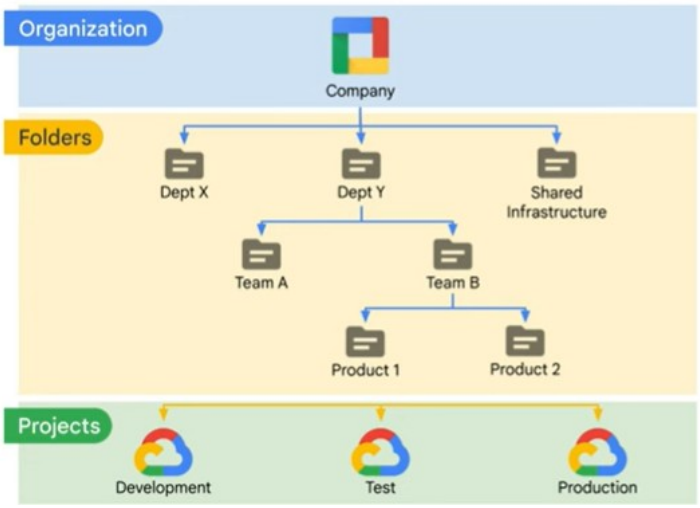
G Suite or Cloud Identity super administrators

and the GCP organization admin are key roles during the setup process and for lifecycle control, for the organization resource.

Creating and managing organizations

- Created when a **Google Workspace** or **Cloud Identity** account creates a Google Cloud Project
- **Workspace** or **Cloud Identity** super administrator:
  - Assign the **Organization admin** role to some users
  - Be the point of contact in case of recovery issues
  - Control the lifecycle of the Workspace or Cloud Identity account and Organization resource
- Organization admin:
  - Define IAM policies
  - Determine the structure of the resource hierarchy
  - Delegate responsibility over critical components such as Networking, Billing, and Resource Hierarchy through IAM roles

Folders



Additional grouping mechanism and isolation boundaries between projects:

- Different legal entities
- Departments
- Teams

Folders allow delegation of administration rights.

Roles: define the can do what on which resource part of Cloud IAM.

Create role -> name (i.e. instance operator) -> assigns permissions (i.e compute.instances.start) -> review

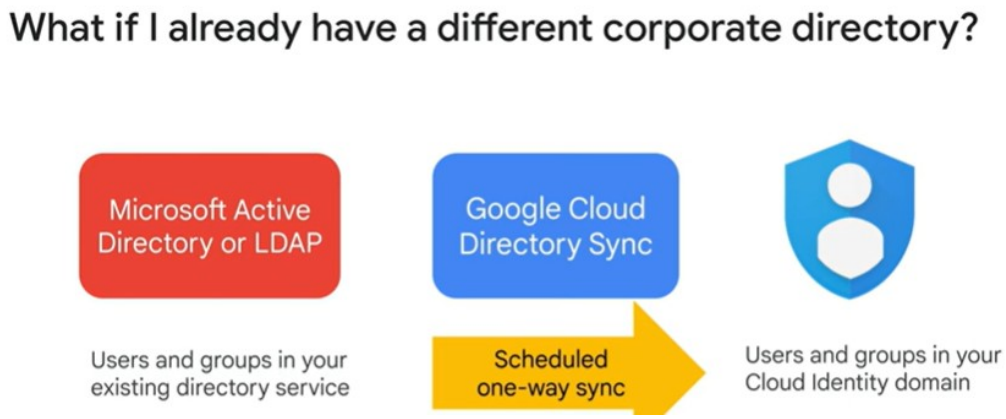
Policies:

Allow

Deny

Condition

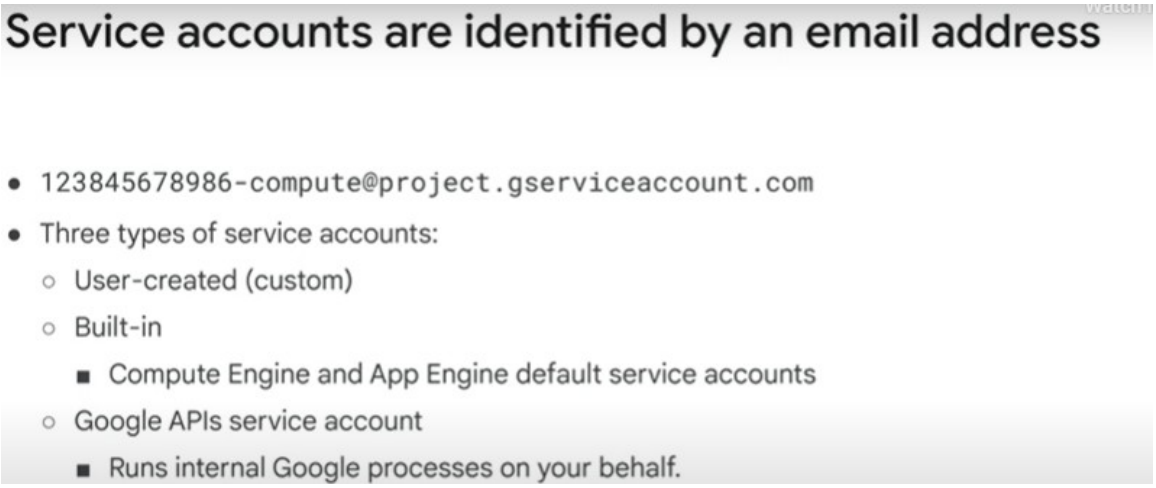
Organization policy



Service account

application that interacts with Google Cloud Storage,

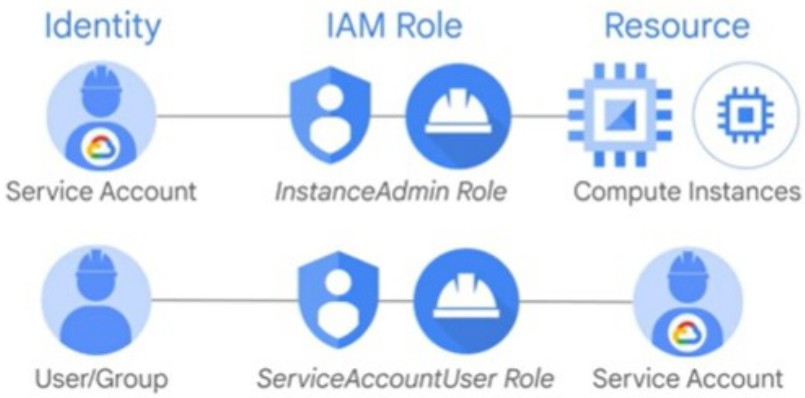
it must first authenticate to either the Google Cloud Storage XML API or JSON API.





Service account permissions

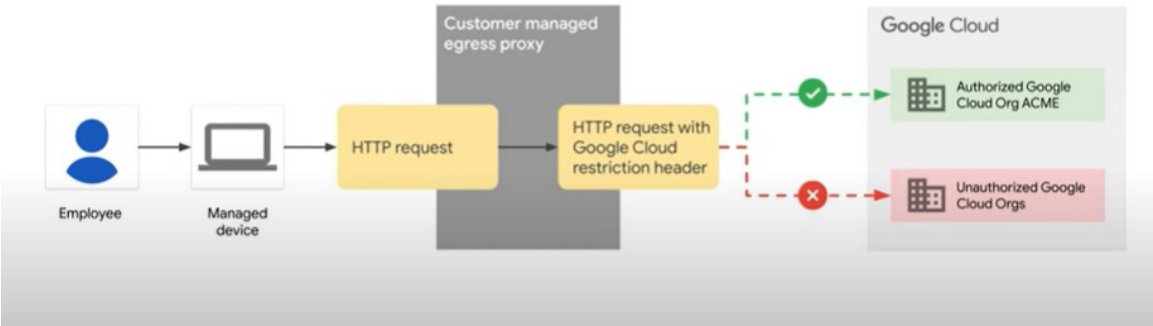
- Default service accounts: basic and predefined roles
- User-created service accounts: predefined roles
- Roles for service accounts can be assigned to groups or users



Organization retractions – important to hot spot

Why important? Cause we can restrict the machine that connect to the organization

Organization Restrictions



IAM Best practices

Leverage and understand the resource hierarchy

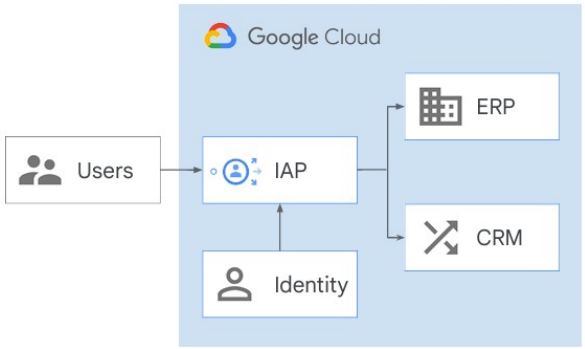
- Use projects to group resources that share the same trust boundary.
- Check the policy granted on each resource and make sure you understand the inheritance.
- Use "principles of least privilege" when granting roles.
- Audit policies in Cloud Audit Logs: setiampolicy.
- Audit membership of groups used in policies.

Identity-Aware Proxy (IAP)

Enforce access control policies for applications and resources:

- Identity-based access control
- Central authorization layer for applications accessed by HTTPS

IAM policy is applied after authentication.



Cloud IAP lets you establish a central authorization layer for applications accessed by HTTPS.

- ⇨ So you can use an application level access control model
- ⇨ instead of relying on network level firewalls.

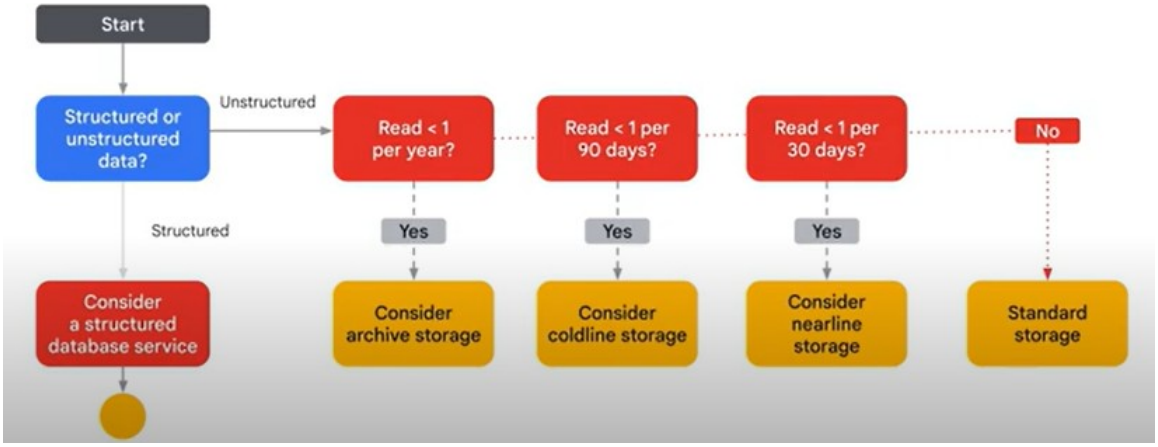
Create sevice account -> **Grant Access**. -> a domain (i.e. **altostrat.com**) -> Select a role
Compute Engine > Compute Instance Admin (v1)

Reference:

https://www.cloudskillsboost.google/course_templates/49/labs/470022

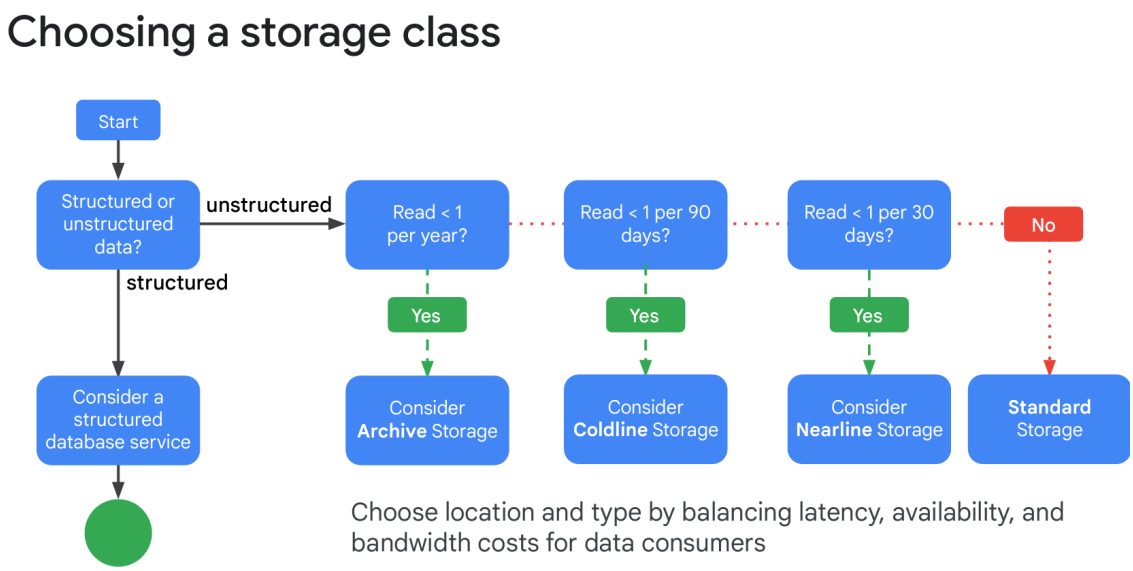
Storage and DB storage

Storage

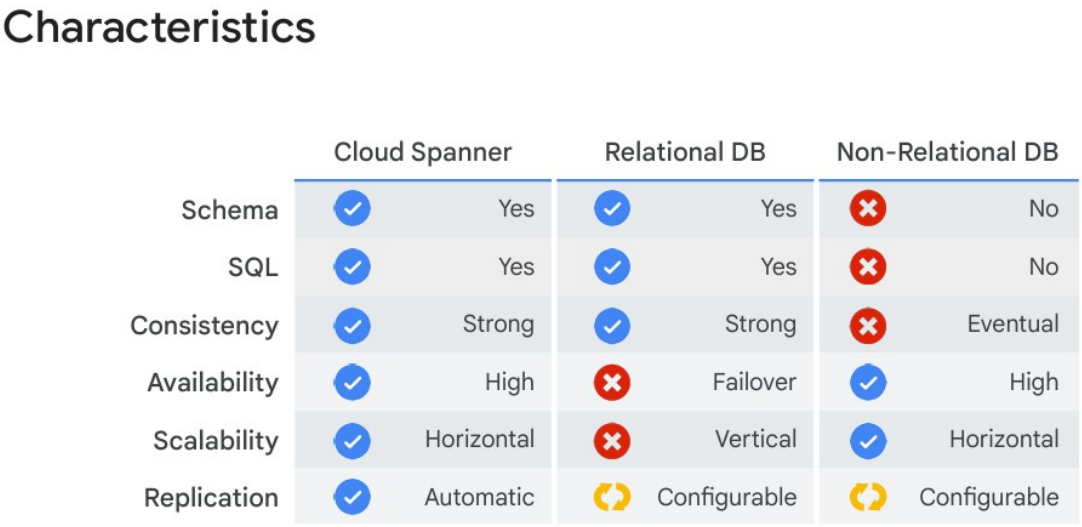


DB Storage

BigQuery service sits on the edge between data storage and data processing.



SQL Spanner



Alloy DB - PostgreSQL-compatible database

AlloyDB for PostgreSQL is a fully managed, PostgreSQL-compatible database service that's designed for demanding workloads such as hybrid transactional and analytical processing.

Firestore - highly scalable NoSQL database - for your mobile web and IOT apps

Cloud Firestore is a fast, fully managed, serverless, cloud native, NoSQL, document database that simplifies storing, synking and querying data for your mobile web and IOT apps at global scale.

ACID

ACID is a set of properties that ensure reliable processing of database transactions. The acronym stands for:

1. **Atomicity**:
    - Ensures that all operations within a transaction are treated as a single unit.
    - Either all operations are completed successfully, or none are applied.
    - In Cloud Firestore, if one operation fails, the entire transaction is rolled back, ensuring no partial changes.
2. **Consistency**:
    - Guarantees that a database remains in a valid state before and after a transaction.
    - For Firestore, this means any transaction ensures that all constraints and rules defined for the database are maintained, such as unique indexes and referential integrity.
3. **Isolation**:
    - Transactions are executed in isolation from each other.
    - In Firestore, the changes made in a transaction are not visible to other transactions or queries until the transaction is successfully committed, preventing dirty reads or race conditions.
4. **Durability**:
    - Ensures that once a transaction is committed, the changes are permanent and survive system failures.
    - Cloud Firestore writes data to multiple data centers, ensuring durability even in the case of regional failures.

Cloud Bigtable - fully managed NoSQL database with petabyte-scale and very low latency – if you don't require transactional consistency

Cloud Storage, a fully managed object store;

Filestore, a fully managed file storage service - **ACID transactions and can scale globally**

Cloud SQL, a fully managed MySQL and PostgreSQL database service **- needed to migrate a standard relational database running on a single machine in a datacenter to the cloud**

Cloud Spanner, a relational database service with transactional consistency, global scale and high availability - **ACID transactions and can scale globally**

AlloyDB, a fully managed, PostgreSQL-compatible database service;

Firestore, a fully managed NoSQL document database;

Bigtable, a fully managed NoSQL wide-column database

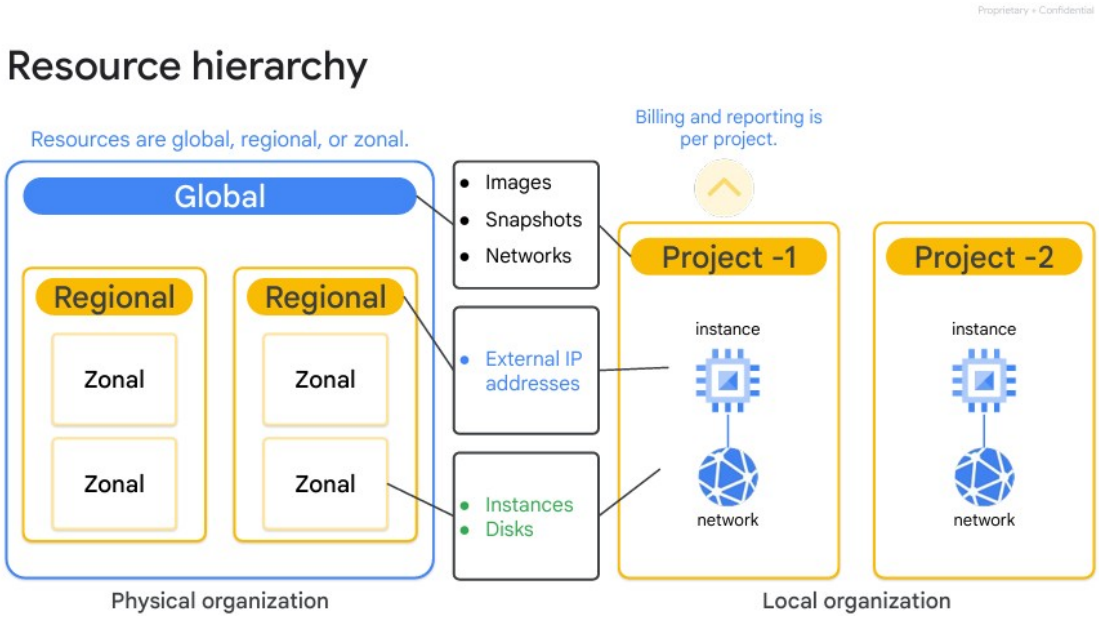
BigQuery - **provides data warehouse services for storing data but also offers an interactive SQL interface for querying the data**

Memorystore, a fully managed in-memory data store service for Redis

Resource Manager

project - associated with one billing account - accumulates the consumption of all its resources.

resource belongs to only one project



Monitoring

Trace - collects latency data from your applications and displays it in the Google Cloud console

Networking

Cloud VPN – securely connects your on-premises network to your Google Cloud VPC network

Cloud VPN gateways: HA VPN and Classic VPN

IPsec = (Internet Protocol Security)

An **IPsec VPN tunnel** in Google Cloud is a **secure communication channel** between two networks or devices over the internet. It uses the **IPsec (Internet Protocol Security)** protocol to encrypt and protect data transmitted between the two endpoints, ensuring confidentiality, integrity, and authenticity.

In the context of Google Cloud, an IPsec VPN tunnel is typically used to connect:

1. A **Google Cloud Virtual Private Cloud (VPC)** to an **on-premises network**.
2. Two separate Google Cloud VPC networks.
3. A Google Cloud VPC to a different cloud provider's network.

Key Features of IPsec VPN Tunnel in GCP

1. **Encryption and Security**:
    - Ensures data is encrypted and protected while traveling over the public internet.
    - Supports AES encryption for high levels of security.
2. **Hybrid Connectivity**:
    - Enables hybrid cloud setups by securely connecting on-premises networks with Google Cloud resources.
3. **High Availability**:
    - Supports **high-availability VPN (HA VPN)** with redundant tunnels for improved reliability.
    - Supports **classic VPN** with a single tunnel (less reliable than HA VPN).
4. **Standards-Based**:
    - Uses industry-standard protocols (IKEv1/IKEv2 for key exchange and IPsec for encryption).

Components of an IPsec VPN Tunnel

1. **VPN Gateway**:
    - A virtual device that terminates the VPN connection at each end.
    - Google Cloud provides the **Cloud VPN Gateway** for this purpose.
2. **Tunnels**:
    - The logical connection between the two VPN gateways.
    - Each VPN tunnel uses **IPsec encryption** for secure communication.
3. **Routing**:
    - Two main options for directing traffic through the VPN tunnel:
        - **Policy-based routing**: Based on specific IP address ranges (legacy approach, supported by Classic VPN).
        - **Route-based routing**: Uses dynamic (BGP) or static routes (supported by HA VPN).
4. **Peer Gateway**:
    - The gateway on the other end of the tunnel (on-premises or another cloud provider).

Use Cases for IPsec VPN Tunnels

1. **Hybrid Cloud Architecture**:
    - Connect an on-premises data center to Google Cloud for extending workloads securely.
2. **Inter-VPC Communication**:
    - Connect multiple Google Cloud VPCs in different regions or projects securely.
3. **Disaster Recovery and Backup**:
    - Use VPN tunnels to replicate data securely between on-premises systems and Google Cloud.
4. **Secure Application Access**:
    - Enable private and secure access to applications hosted in Google Cloud from external environments.

How to Set Up an IPsec VPN Tunnel in GCP

1. **Create a Cloud VPN Gateway** in a GCP region and attach it to a subnet.
2. **Configure a VPN tunnel**:
    - Provide details of the peer gateway (IP address and shared secret).
    - Choose routing (static or dynamic with BGP).
3. **Establish Connectivity**:
    - Ensure firewall rules are configured to allow IPsec traffic (ports 500 and 4500).
4. **Test the Tunnel**:
    - Verify that the tunnel is established and traffic is flowing correctly.

Classic VPN doesn't support use cases where client computers need to “dial in” to a VPN using client VPN software.

What can we do ?

Google Cloud can support such use cases through third-party solutions or alternative methods.

Native GCP VPN Solutions and Their Limitations

- **Classic VPN** and **HA VPN** are designed for **site-to-site VPN** connections and are not built to handle **remote access VPN** connections where individual client computers (like laptops or phones) need to connect directly.
- These solutions work well for connecting on-premises networks or other cloud environments to a Google Cloud VPC.

Options to Enable Client VPN Access in GCP

1. **Third-Party VPN Servers in GCP**:
    - Deploy a third-party VPN server in your Google Cloud VPC to support client devices. Examples include:
        - **OpenVPN**: Widely used and supports client access across multiple platforms.
        - **WireGuard**: Lightweight and secure VPN for modern use cases.
        - **StrongSwan**: An IPsec-based VPN server.
    - You install and configure these solutions on a Compute Engine instance.

Pros:

- Full control over configuration.
- Flexible to meet specific requirements (e.g., user authentication methods).

Cons:

- Requires manual setup, maintenance, and monitoring.
- Additional costs for Compute Engine instances and egress traffic.

---

2. **BeyondCorp Enterprise** (Zero Trust Access):
   - A **zero-trust** solution from Google that replaces the need for a traditional VPN.
   - Users access applications securely based on their identity and device context, without needing to establish a VPN tunnel.
   - Ideal for organizations looking to move away from VPNs altogether.

---

3. **Cloud Identity-Aware Proxy (IAP)**:
   - Provides secure access to internal applications hosted on Google Cloud without a VPN.
   - Uses OAuth 2.0 for authentication and supports fine-grained access control.
   - Suitable for granting specific application-level access rather than network-level access.

---

4. **Google Workspace VPN (Enterprise Tier)**:
   - If you're using Google Workspace Enterprise, there are advanced options for securing client connections using Google's built-in infrastructure.
   - Works well with BeyondCorp principles.

IKEV1 / 2

IKEv1 and IKEv2 Ciphers

IKE (Internet Key Exchange) is a protocol used to set up secure communication between devices in IPsec-based VPNs. It negotiates cryptographic keys and security policies for the VPN. There are two versions, **IKEv1** (legacy) and **IKEv2** (modern), and they both use cryptographic ciphers for encryption, integrity, and authentication.

IKE Cipher Types

1. **Encryption Ciphers**:
    - Provide confidentiality by encrypting the data exchanged between endpoints.
    - Examples: AES, DES, 3DES.
2. **Integrity Ciphers (Hashing)**:
    - Ensure data integrity by generating and verifying message digests.
    - Examples: SHA-1, SHA-2 (SHA-256, SHA-384, SHA-512).
3. **Authentication Mechanisms**:
    - Authenticate the peers using shared secrets, certificates, or pre-shared keys.
    - Examples: RSA, ECDSA, Pre-shared key (PSK).
4. **Diffie-Hellman (DH) Groups**:
    - Used during the key exchange process to securely derive shared keys over an insecure channel.
    - Examples: DH Groups 1, 2, 14, 19, 20, 21.

IKEv1 Ciphers

- **Encryption Algorithms**:
    - DES (weak, deprecated due to short key length of 56 bits).
    - 3DES (Triple DES, stronger than DES but less efficient).
    - AES-128, AES-192, AES-256 (preferred due to efficiency and security).
- **Integrity Algorithms**:
    - MD5 (weak, deprecated due to vulnerabilities).
    - SHA-1 (widely supported but less secure than SHA-2).
    - SHA-2 (SHA-256, SHA-384, SHA-512).
- **Key Exchange**:
    - Diffie-Hellman groups: 1, 2, 5 (lower groups like 1 and 2 are weak and deprecated).

Limitations:

- IKEv1 is less efficient and less secure than IKEv2.
- Does not support modern ciphers like GCM for authenticated encryption.

IKEv2 Ciphers

- **Encryption Algorithms**:
    - AES-GCM (Galois/Counter Mode, provides both encryption and integrity in one step).
    - AES-CBC (Cipher Block Chaining, widely supported).
    - ChaCha20-Poly1305 (lightweight and efficient, suitable for constrained devices).
- **Integrity Algorithms**:
    - SHA-2 family (SHA-256, SHA-384, SHA-512).
    - SHA-1 (for backward compatibility but not recommended).
- **Key Exchange**:
    - Diffie-Hellman groups: 14 (2048-bit), 19 (256-bit ECDH), 20 (384-bit ECDH), 21 (521-bit ECDH).
    - Stronger groups like 19–21 are recommended for high security.
- **Authentication Mechanisms**:
    - RSA, ECDSA, and PSK.

Advantages:

- Supports modern encryption standards.
- More efficient, with faster setup and fewer round trips than IKEv1.
- Built-in NAT traversal and enhanced resiliency.

Recommended Cipher Suites

For a secure configuration, especially with IKEv2:

- **Encryption**: AES-256-GCM or AES-128-GCM.
- **Integrity**: SHA-256 or SHA-384.
- **Key Exchange**: DH group 19 (256-bit ECDH) or group 21 (521-bit ECDH).
- **Authentication**: RSA (2048+ bits) or ECDSA (256+ bits).

Why IKEv2 is Preferred Over IKEv1

1. Better security: Supports modern encryption algorithms like AES-GCM and ChaCha20-Poly1305.
2. Efficiency: Faster negotiation with fewer round trips.
3. Resilience: Handles network disruptions better, including NAT traversal.

Cloud Router and a VPN Gateway

The choice between using a **Cloud Router** and a **VPN Gateway** depends on your networking requirements and how you want to connect different networks. Here's a detailed breakdown to clarify when to use each:

Cloud Router

Purpose: Manages dynamic routing for hybrid and multi-cloud networking.

When to Use:

1. **Dynamic Routing with BGP:**
    - You need to exchange dynamic routing information between Google Cloud and your on-premises or another cloud network.
    - Useful when your network topology changes frequently, as it allows automatic updates of routes without manual intervention.
2. **Cloud Interconnect:**
    - Required if you're using **Dedicated Interconnect** or **Partner Interconnect** to connect your on-premises network to Google Cloud.
    - Ensures routing information is dynamically updated through Border Gateway Protocol (BGP).
3. **Hybrid Cloud Architectures:**
    - When you want seamless communication between Google Cloud and on-premises networks with dynamic route updates.
4. **Peering Scenarios:**
    - Facilitates dynamic route exchange in **Network Peering** setups within Google Cloud or with other clouds.

VPN Gateway

Purpose: Provides secure, encrypted communication between Google Cloud and on-premises or other cloud environments.

When to Use:

1. **Secure Communication:**
    - When you need to establish an **IPsec VPN tunnel** for encrypted communication between Google Cloud and on-premises or other cloud environments.
    - Ideal for connecting smaller setups or as a backup to Interconnect.
2. **Static or Dynamic Routing:**
    - Can work with both static and dynamic routing (via Cloud Router) for route management.
    - If dynamic routing is required, a Cloud Router is used alongside the VPN Gateway to support BGP.
3. **Lower Bandwidth Needs:**
    - When the bandwidth requirement is relatively low compared to Interconnect solutions.
4. **Temporary Connections:**
    - Ideal for temporary or ad-hoc connections between on-premises and cloud.
5. **Failover Solution:**
    - As a backup solution for Cloud Interconnect to maintain connectivity during outages.

Comparison and Scenarios

Feature	Cloud Router	VPN Gateway
Routing	Dynamic with BGP (**Border Gateway Protocol -** automatically exchanging routing information between networks)	Static or Dynamic (if paired with BGP)
Security	No encryption (routing only)	Encrypted communication
Use Case	Route management for hybrid setups	Secure data transfer over the internet
Bandwidth	High (e.g., Interconnect scenarios)	Lower (subject to VPN limits)
Primary Role	Routing protocol (BGP) manager	Secure IPsec tunnel setup

Common Use Cases

1. **Use Cloud Router with VPN Gateway:**
    - When you need dynamic routing for your IPsec VPN connection.
    - Example: Connecting dynamically changing on-premises subnets to Google Cloud.
2. **Use Cloud Router with Interconnect:**
    - For high-bandwidth, low-latency private connectivity.
    - Example: Enterprise-grade workloads requiring predictable performance.
3. **Use VPN Gateway Alone:**
    - For simple, encrypted connectivity with static routing.
    - Example: Connecting a single office to Google Cloud.

In summary, use **Cloud Router** when routing dynamics and route updates are crucial, and use **VPN Gateway** when secure, encrypted connectivity is required. Often, they are used together in scenarios requiring dynamic and secure connections.

Comparison of Interconnect options

Connection	Provides	Capacity	Requirements	Access Type
VPN tunnel	Encrypted tunnel to VPC networks through the public internet	1.5–3 Gbps per tunnel	Remote VPN gateway	Internal IP addresses
Dedicated Interconnect	Dedicated, direct connection to VPC networks	10 Gbps or 100 Gbps per link	Connection in colocation facility	
Partner Interconnect	Dedicated bandwidth, connection to VPC network through a service provider	50 Mbps – 50 Gbps per connection	Service provider	
Cross-Cloud Interconnect	Dedicated physical connection between VPC network and network hosted by service provider	10 Gbps or 100 Gbps per connection	Primary and redundant ports (Google Cloud and remote cloud service provider)	

Comparison of Peering options

Connection	Provides	Capacity	Requirements	Access Type
Direct Peering	Dedicated, direct connection to Google's network	10 Gbps Per link	Connection in Google Cloud PoPs	Public IP addresses
Carrier Peering	Peering through service provider to Google's public network	Varies based on partner offering	Service provider	

Appendix- N Networking

Set up a Global VPC environment – with 2 VMs in each subnet and firewall rules

Reference: https://www.cloudskillsboost.google/course_templates/178/labs/470286

In this task you set up a Global VPC with two custom subnets and two VM instances running in each zone.

1. In Cloud Shell, create a VPC network called **vpc-demo**:

gcloud compute networks create vpc-demo --subnet-mode custom

Copied!

content_copy

The output should look similar to this:

Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-03-cdb29e18d20d/global/networks/vpc-demo].

NAME: vpc-demo

SUBNET_MODE: CUSTOM

BGP_ROUTING_MODE: REGIONAL

IPV4_RANGE:

GATEWAY_IPV4:

2. In Cloud Shell, create subnet **vpc-demo-subnet1** in the region **us-west1**:

```
gcloud compute networks subnets create vpc-demo-subnet1 \
--network vpc-demo --range 10.1.1.0/24 --region "us-west1"
```

Copied!

content_copy

3. Create subnet **vpc-demo-subnet2** in the region us-east1:

```
gcloud compute networks subnets create vpc-demo-subnet2 \
--network vpc-demo --range 10.2.1.0/24 --region us-east1
```

Copied!

content_copy

4. Create a firewall rule to allow all custom traffic within the network:

```
gcloud compute firewall-rules create vpc-demo-allow-custom \
 --network vpc-demo \
 --allow tcp:0-65535,udp:0-65535,icmp \
 --source-ranges 10.0.0.0/8
```

Copied!

content_copy

The output should look similar to this:

Creating firewall...working..Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-03-cdb29e18d20d/global/firewalls/vpc-demo-allow-custom].

Creating firewall...done.

NAME: vpc-demo-allow-custom

NETWORK: vpc-demo

DIRECTION: INGRESS

PRIORITY: 1000

ALLOW: tcp:0-65535,udp:0-65535,icmp

DENY:

DISABLED: False

5. Create a firewall rule to allow SSH, ICMP traffic from anywhere:

```
gcloud compute firewall-rules create vpc-demo-allow-ssh-icmp \
  --network vpc-demo \
  --allow tcp:22,icmp
```

Copied!

content_copy

6. Create a VM instance **vpc-demo-instance1** in zone **us-west1-a**:

```
gcloud compute instances create vpc-demo-instance1 --machine-type=e2-medium --zone us-west1-a --subnet vpc-demo-subnet1
```

Copied!

content_copy

The output should look similar to this:

Created [https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-03-cdb29e18d20d/zones/us-west1-a/instances/vpc-demo-instance1].

NAME: vpc-demo-instance1

ZONE: us-west1-a

MACHINE_TYPE: e2-standard-2

PREEMPTIBLE:

INTERNAL_IP: 10.1.1.2

EXTERNAL_IP: 34.71.135.218

STATUS: RUNNING

7. Create a VM instance **vpc-demo-instance2** in zone us-east1-c:

```
gcloud compute instances create vpc-demo-instance2 --machine-type=e2-medium --zone us-east1-c --subnet vpc-demo-subnet2
```

LoadBalance

internal load balancer

uses

Andromeda - Google Cloud's software-defined network virtualization stack

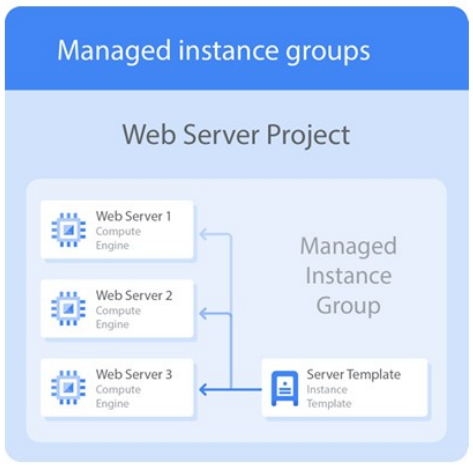
Maglev - a large, distributed software system.

for HTTP(S) traffic is a proxy-based, regional Layer 7 load balancer

Instance groups

Managed instance groups

- Deploy identical instances based on instance template
- Instance group can be resized
- Manager ensures all instances are RUNNING
- Typically used with autoscaler
- Can be single zone or regional



Regional managed instance groups (us-central1) better than zonal managed instance groups (us-central1-a) because allowing u to spread the application load across multiple zones (a/b/c)

Create manages instance group -> template -> instances

Examples:

stateless serving

batch workloads (website front end or image processing from a queue)

statefull applications.

HTTP(S) load balancing

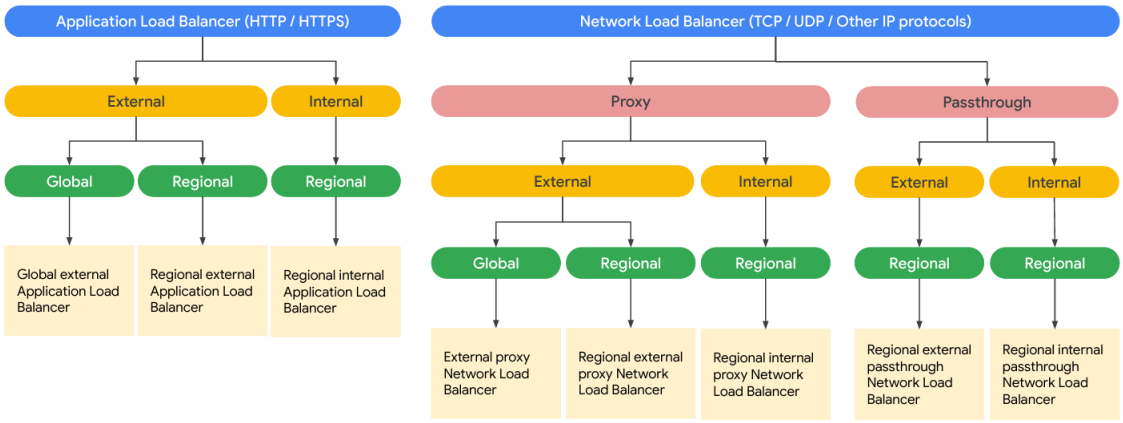
acts at Layer 7 of the OSI model (application layer, which deals with the actual content of each message, allowing for routing decisions based on the URL)

EXTERNAL HTTP LOAD BALANCER with NAT and instance groups with autoscaling

https://www.cloudskillsboost.google/course_templates/178/video/470302

load balancing services support IPv6 clients are : HTTP(S), SSL proxy, and TCP proxy

Deployment modes available for Cloud Load Balancing



Load balancer	Deployment mode	Traffic type	Network Service Tier	Load-balancing scheme
Application Load Balancers	Global external	HTTP or HTTPS	Premium	EXTERNAL_MANAGED
	Regional external	HTTP or HTTPS	Standard	EXTERNAL_MANAGED
	Classic	HTTP or HTTPS	Global in Premium Regional in Standard	EXTERNAL
	Internal Always regional	HTTP or HTTPS	Premium	INTERNAL_MANAGED
Proxy Network Load Balancers	Global external	TCP with optional SSL offload	Global in Premium Regional in Standard	EXTERNAL
	Regional external	TCP	Standard only	EXTERNAL_MANAGED
	Internal Always regional	TCP without SSL offload	Premium only	INTERNAL_MANAGED
Passthrough Network Load Balancers	External Always regional	TCP, UDP, ESP, GRE, ICMP, and ICMPv6	Premium or Standard	EXTERNAL
	Internal Always regional	TCP or UDP	Premium only	INTERNAL

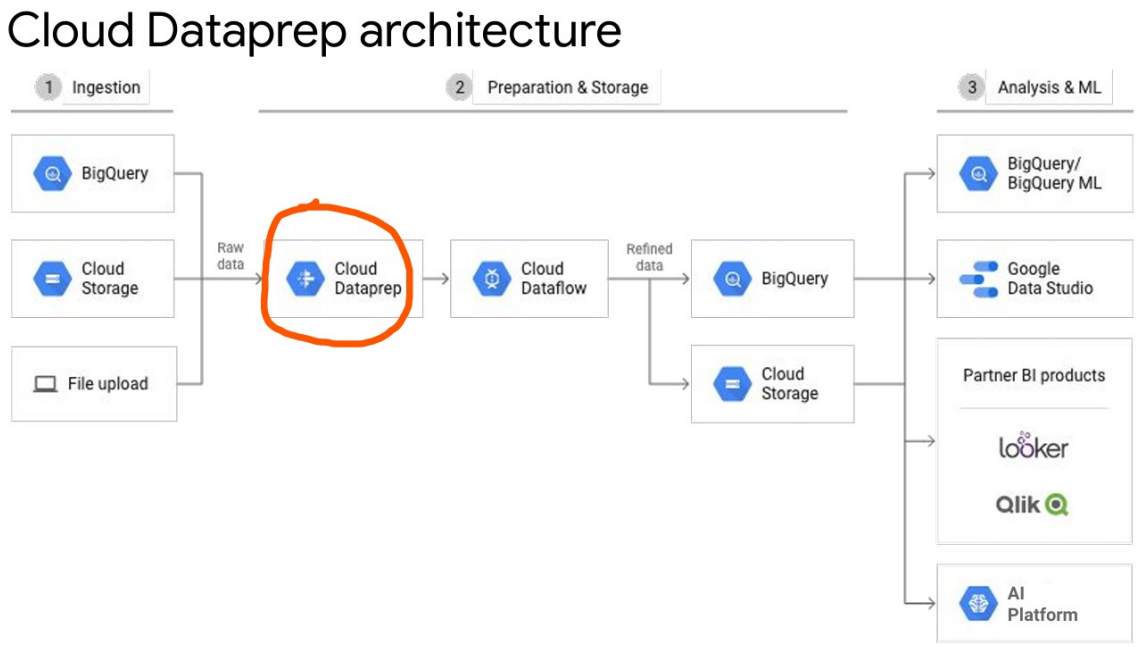
Cloud CDN - Content Delivery Network - cache HTTP(S) load-balanced content close to your users

Managed Services

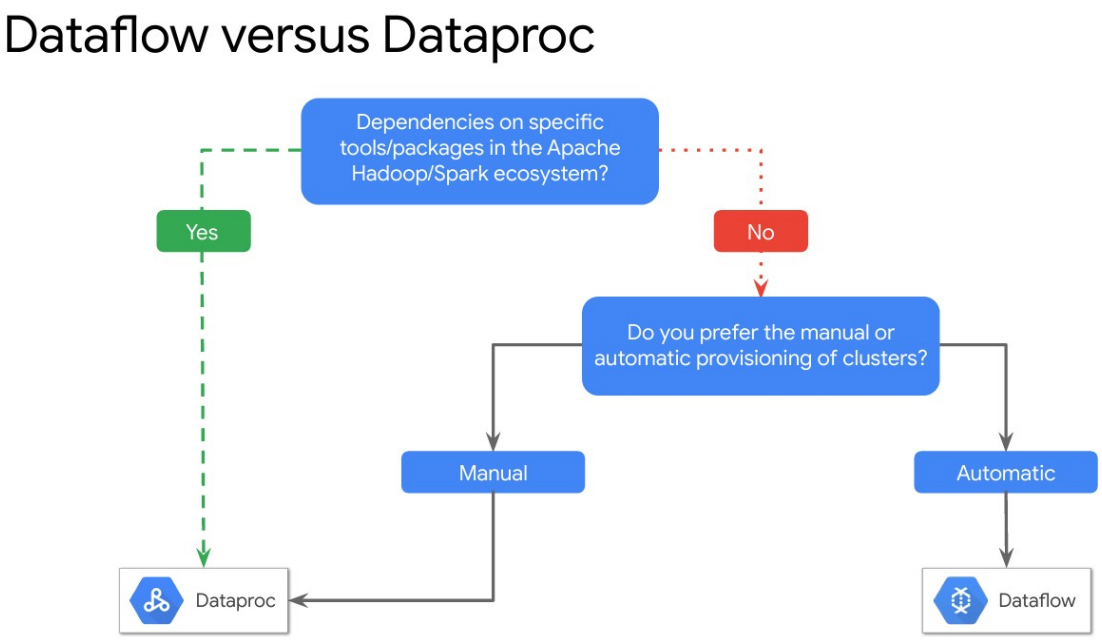
BigQuery -> like db for data analyzing

DataFlow -> like Fluentd and terraform (it provision cluster)

Dataprep -> cleaning and preparing un/structured data for analysis reporting / ML



Dataproc -> like manually deploying cluster - fully managed cloud service for running Apache Spark and Apache Hadoop clusters



Architecture – W6

what the software is supposed to do, who your users are, and why this is important.

Do you want a relational database, a NoSQL database, or a data warehouse?

compute platform.

deploy your apps to virtual machines, a Kubernetes cluster, or an automated platform, like App Engine

Availability, durability, cost, and disaster recovery

The prerequisite for this course is

either

the Architecting with Google Compute Engine

the Architecting with Google Kubernetes Engine course.

SRE :: SLI (Service Level Indicator) and SLO (Service Level Objective)

key concepts used to define, measure, and maintain service reliability.

---

1. SLI (Service Level Indicator)

- **Definition:** A quantitative measure of service performance.
- **Purpose:** It helps track how well a service is meeting expectations.
- **Example:**
    - **Availability SLI:** Percentage of successful HTTP requests (e.g., 99.95% of requests return HTTP 200).
    - **Latency SLI:** Percentage of requests completed within 100ms.
    - **Error Rate SLI:** Percentage of failed requests.

SLIs are collected using **Cloud Monitoring, Cloud Logging, and Cloud Trace** in GCP.

---

2. SLO (Service Level Objective)

- **Definition:** A target value or range for an SLI.
- **Purpose:** Sets a reliability goal that the service should maintain.
- **Example:**

- **Availability SLO:** "The API must be available 99.95% of the time over a rolling 30-day period."
- **Latency SLO:** "95% of requests must have a response time under 100ms."

SLOs help balance **feature development vs. reliability**. If an SLO is violated, teams should prioritize fixing reliability issues.

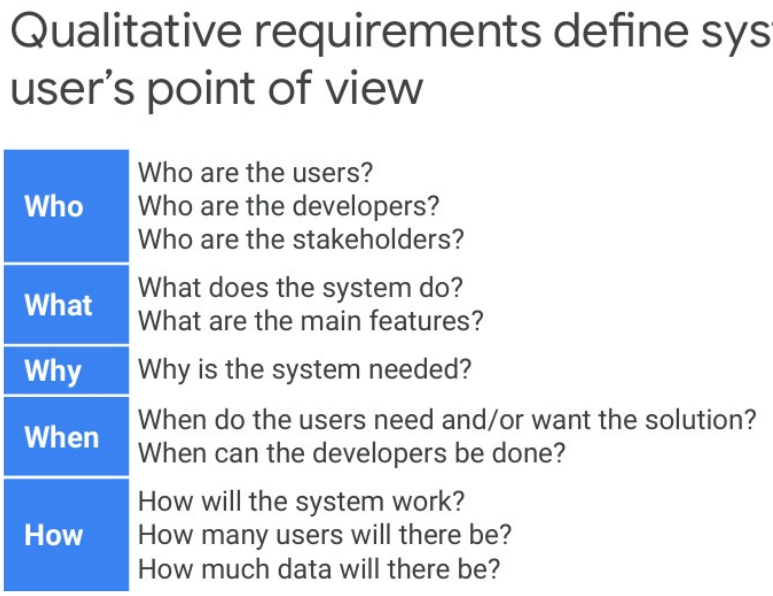
How GCP Helps with SLI and SLO?

GCP provides **Cloud Operations Suite (formerly Stackdriver)** to monitor SLIs and enforce SLOs:

- **Cloud Monitoring:** Collects metrics for SLIs.
- **Cloud Logging:** Stores logs for debugging.
- **Cloud Trace:** Measures latency.
- **Cloud Service Health API:** Tracks Google Cloud services' health.

By defining **SLIs and SLOs in Service Monitoring**, DevOps teams in GCP can **track service health, prevent outages, and ensure high availability**.

Roles -> persona



Persona:

● In a real-world application, go find your users and talk to them.

●Personas tell a story of who they are.

●Personas are not a list of job functions.

● For each role, there could be many personas

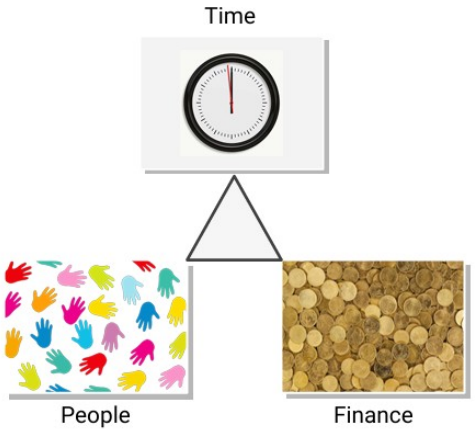
Evaluate user stories with the INVEST criteria

- **I**ndependent
- **N**egotiable
- **V**aluable
- **E**stimatable
- **S**mall
- **T**estable

SLOs, SLIs, and SLAs

manage a service

Quantitative requirements are things that are measurable



Given the constraints:

- Time
- Finance
- People

What can be achieved:

- How many users are there?
- How much data is there?
- What are the rewards and risks?
- Which features can be launched?

Manage service - data gathered from the services

For user-facing systems

-> was a request responded to? (availability)

-> how long did it take to respond? (latency)

-> how many requests can be handled? (throughput)

For data storage systems

-> how long does it take to read and write data? (latency)

-> Is the data there when we need it? (availability)

-> if there is a failure, do we lose any data (durability)

System Type	Means to get SLI	SLO
User-facing	Availability - was a request responded to Example: API percentage available of the time over a rolling 30-day period	99
	Latency - how long did it take to respond Example: percentage of the requests that must have a response time under 100ms	95
	Throughput - how many requests can be handled	
Data storage	Availability - Is the data there when we need it	
	Latency - how long does it take to read and write data	
	Durability - if there is a failure, do we lose any data	

SLOs must be achievable and relevant

SLI	SLO	
HTTP POST photo uploads complete within 100ms aggregated per minute	**99%**	✖ If our users are using mobile phones, maybe this is overkill.
	80%	✔ This might be good enough.
Available as measured with an uptime check every 10 seconds aggregated per minute	**100%**	✖ Sounds good, but not practical.
	99.999%	✖ Possible, but maybe too expensive.
	99%	✔ Maybe good enough and easier and more cost-effective.

What can be achieved:

- How many users are there?
- How much data is there?
- What are the rewards and risks?
- Which features can be launched

KPI's

In business, common KPIs include:

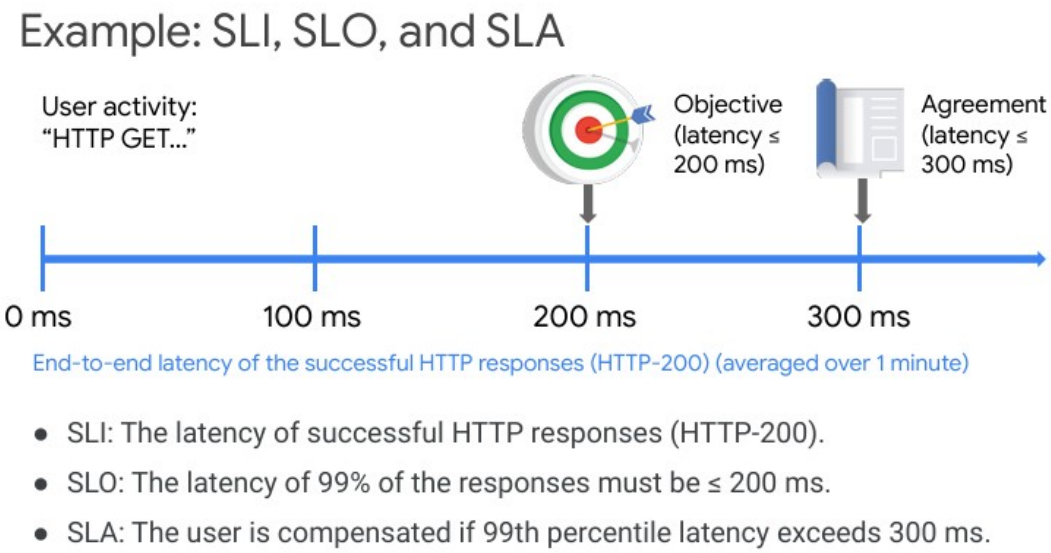
- Return on investment (ROI)
- Earnings before interest and taxes

(EBIT)

- Employee turnover
- Customer churn

In software, common KPIs include:

- Page views
- User registrations
- Clickthroughs
- Checkouts



- SLI: The latency of successful HTTP responses (HTTP-200).
- SLO: The latency of 99% of the responses must be ≤ 200 ms.
- SLA: The user is compensated if 99th percentile latency exceeds 300 ms.

Microservice Design and Architecture

Session affinity

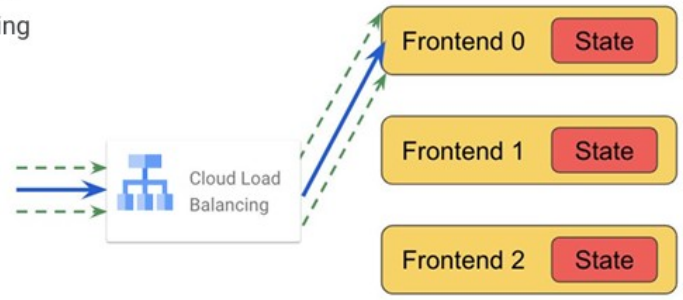
subsequent client requests have to be sent to the same server that the initial request was made to -> configuration of the load balancers to use sticky sessions -> which in Google Cloud is referred to as session affinity.

Best practices

Avoid storing shared state in-memory on your servers

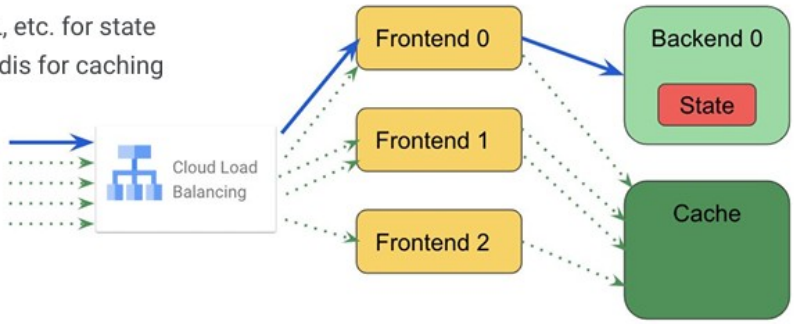
sticky session = in google: Session affinity

- Requires sticky sessions to be set up in the load balancer
- Hinders elastic autoscaling

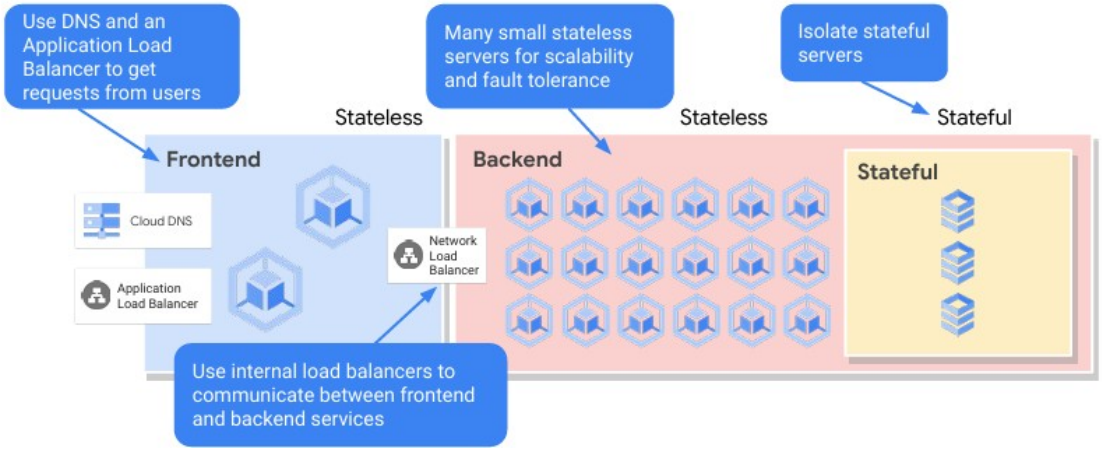


Store state using backend storage services shared by the frontend server

- Cache state data for faster access
- Take advantage of Google Cloud–managed data services
  - Firestore, Cloud SQL, etc. for state
  - Memorystore for Redis for caching



A general solution for large-scale cloud-based systems



The 12 factors

01

Codebase

One codebase tracked in revision control, many deploys

- Use a version control system like Git.
- Each app has one code repo and vice versa.

02

Dependencies

Explicitly declare and isolate dependencies

- Use a package manager like Maven, Pip, NPM to install dependencies.
- Declare dependencies in your code base.

03

Config

Store config in the environment

- Don't put secrets, connection strings, endpoints, etc., in source code.
- Store those as environment variables.

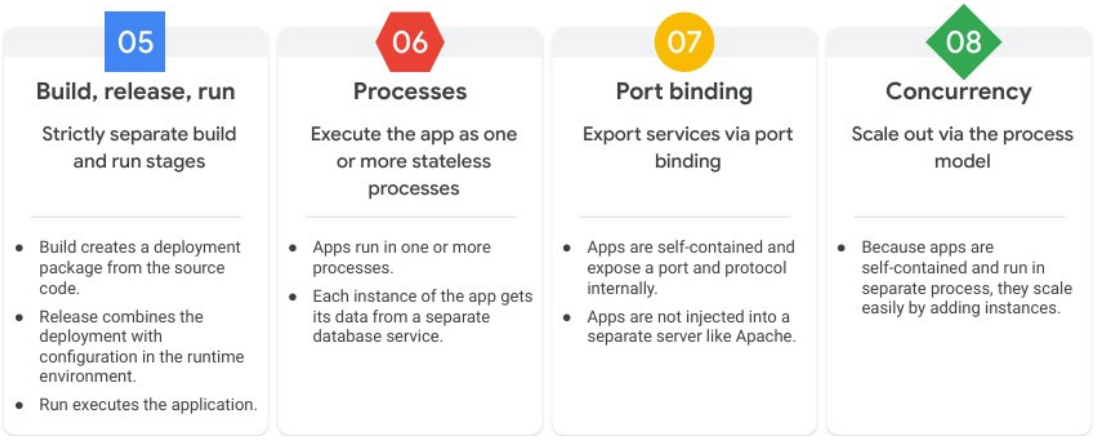
04

Backing Services

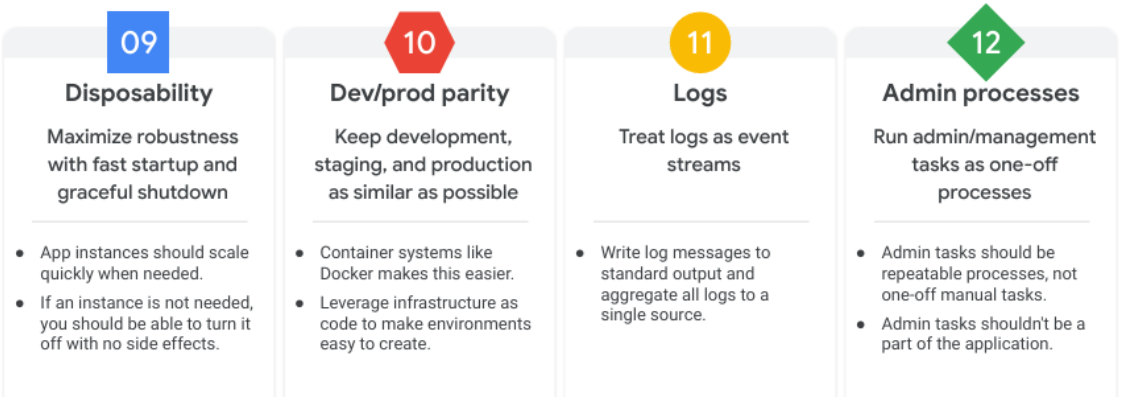
Treat backing services as attached resources

- Databases, caches, queues, and other services are accessed via URLs.
- Should be easy to swap one implementation for another.

The 12 factors (continued)



The 12 factors (continued)



API's

It's important to design consistent APIs for services

- Each Google Cloud service exposes a REST API

  Functions are in the form:

  service.collection.verb

  Parameters are passed either in the URL or in the request body in JSON format

- For example, the Compute Engine API has…

  A service endpoint at: https://compute.googleapis.com

  Collections include instances, instanceGroups, instanceTemplates, etc.

Verbs include insert, list, get, etc.

- So, to see all your instances, make a GET request to:

https://compute.googleapis.com/compute/v1/projects/{project}/zones/{zone}/instances

It is important to design consistent APIs for services

[API design guide | Cloud API Design Guide | Google Cloud]

Google Cloud provides three tools for managing APIs: Cloud Endpoints, Apigee, and API Gateway

They provide tools for:

- User authentication
- Monitoring
- Securing APIs
- Etc.

They support OpenAPI and gRPC



Cloud Endpoint: API management on any Google Cloud backend.
Apigee: for enterprises,
with deployment options for on-cloud, on-premises, or hybrid.
API Gateway: secure access to your backend services
through a well-defined REST API
that is consistent across all of your services.

For a UI service, we might return HTML.

For backend service might return JSON or XML.

Choosing storage Solution

Google Cloud–managed storage and database portfolio



Proprietary + Confidential

Google Cloud storage and database portfolio

Relational			File	NoSQL		Object	Block	Warehouse	In-memory
Cloud SQL	Spanner	AlloyDB	Filestore	Firestore	Bigtable	Cloud Storage	Persistent Disk	BigQuery	Memorystore
Good for: Web frameworks	**Good for:** RDBMS+scale, HA, HTAP	**Good for:** Hybrid transactional and analytical processing	**Good for:** Network Attached Storage (NAS)	**Good for:** Hierarchical, mobile, web	**Good for:** Heavy read + write, events	**Good for:** Binary object data	**Good for:** Applications requiring high performance, scalability and availability	**Good for:** Enterprise data warehouse	**Good for:** Caching for Web/Mobile apps
Such as: CMS, eCommerce	**Such as:** User metadata, Ad/Fin/MarTech	**Such as:** Machine learning, Generative AI	**Such as:** Latency sensitive workloads	**Such as:** User profiles, Game State	**Such as:** AdTech, financial, IoT	**Such as:** Images, media serving, backups	**Such as:** Data warehousing, Big data analytics	**Such as:** Analytics, dashboards	**Such as:** Game state, user sessions
Scales to 64 TB MySQL PostgreSQL SQL Server	**Scales infinitely Regional or multi-regional**	**Secure, AI-powered performance**	**Fully managed, enterprise grade file storage service**	**Completely managed Document database**	**Scales infinitely Wide-column NoSQL**	**Completely managed Infinitely scalable**	**Powerful and versatile storage service**	**Completely Managed SQL analysis**	**Managed Redis DB**
Fixed schema	Fixed schema	Fixed schema	Schemaless	Schemaless	Schemaless	Schemaless	Schemaless	Fixed schema	Schemaless

If I want

FINANCE (structured data) -> Spanner

WEB (structured data) -> Cloud SQL

AI & ML (structured data) -> Alloy DB

NAS -> FileStore

FIN (NO STRUCTURE) -> BigTable

Images (google images) -> Cloud Storage

Big Data Analytics -> Persistent Disk

Datawarehouse -> Big Query

Cache -> MemoryStore DB

CloudSQL can be implemented in ONLY in one (1) REGION

For a **relational database with extremely high availability (99.999%)** that **must run uninterrupted even during a regional outage**, the best option is:

✅ **Google Cloud Spanner**

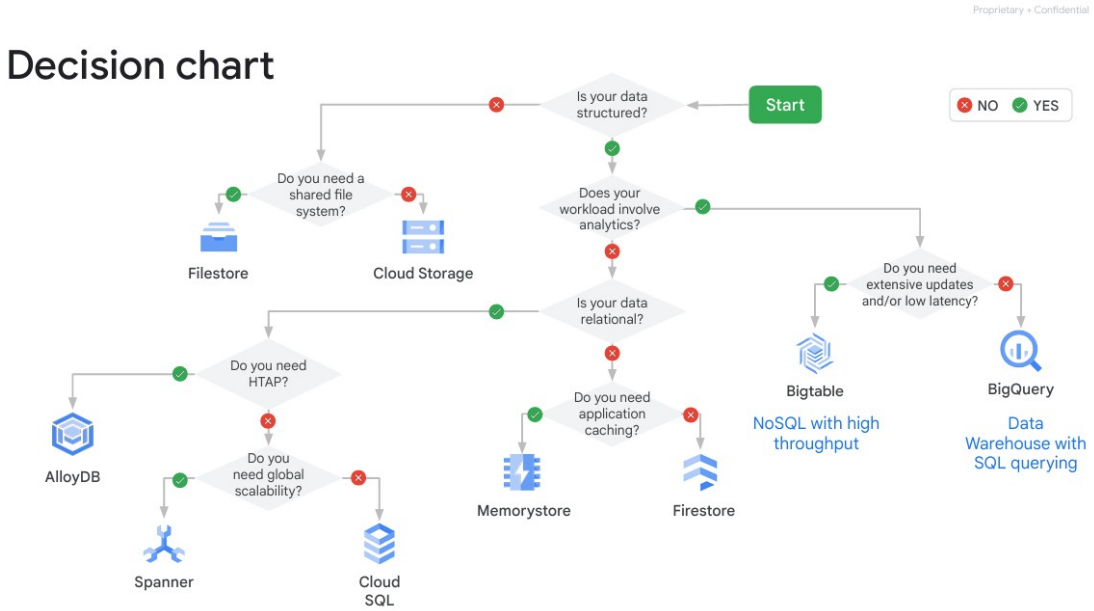
- **Global distribution** → Can replicate across multiple regions.
- **99.999% SLA (Five 9's Availability)** → Ensures uptime even during regional failures.
- **Strong consistency** → ACID-compliant transactions.
- **Auto-sharding & scaling** → Handles massive workloads without downtime.

web application storage architecture

Firestore – NoSQL storage such as product data (being able to synchronize with client applications)

cloudSQL - structured relational data such as customer data and sales data

cloudstorage – images (backend will retrieve data)



PRODUCT to use Firestore (like mongodb)

Product can be from many manufactors , each has its own attribute , (chees can be with lowerfat attribute, while other will write just the fat percentage), so , better to use like a document

Hence, product better to be kept as firestore DB and not structured db

Key Differences: Firestore vs. Cloud SQL

Feature	Firestore (NoSQL, Document DB)	Cloud SQL (SQL, Relational DB)
Data Model	Document-based (JSON-like)	Relational (Tables, Rows, Columns)
Scalability	Auto-scales globally	Vertical & Horizontal Scaling
Schema	Schema-less, flexible	Strict schema, predefined
Querying	Limited queries, no joins	SQL queries, complex joins
Transactions	Limited multi-document transactions	ACID-compliant transactions
Performance	Fast for key-value lookups	Optimized for structured queries
Real-time Sync	Yes, built-in	No, needs polling/webhooks
Replication	Multi-region, auto-managed	Regional & manual replication
Use Case	Web & mobile apps, flexible schema (ike product)	Enterprise, structured apps (like registered customer)

Questions:

1. You are a global financial services company with users all over the world. You need a database service that can provide low latency worldwide with strong consistency. Which service might you choose?

    Firestore

    BigQuery

    Spanner - correct

    Cloud SQL

Spanner - A key feature of Spanner is scale for relational data with strong consistency, and it is globally distributed to provide low latency. The high availability and automatic replication are also strong features for financial services.

2. You need to store user preferences, product information, and reviews for a website you are building. There won't be a huge amount of data. What would be a simple, cost-effective, managed solution?

    BigQuery.

Firestore - correct

Cloud SQL

Spanner

Firestore provides automatic scale ACID transactions and live synchronization and is integrated with Google Cloud and Firebase. It also has a free tier.

3. You want to analyze sales trends. To help achieve this, you want to combine data from your on-premises Oracle database with Google Analytics data and your web server logs. Where might you store the data so it is both easy to query and cost-effective?
    1. Spanner
    2. Firestore
    3. BigQuery - correct
    4. Cloud SQL

    BigQuery is a data warehouse used for data analytics, and so is built for this type of use case. It provides the infrastructure to ingest data from many different sources, which is a requirement too. The cost model of paying for storage and then only for queries run is attractive too.
4. Currently, you are using Firestore to store information about products, reviews, and user sessions. You'd like to speed up data access in a simple, cost-effective way. What would you recommend?
    1. Move the data to Spanner.
    2. Cache the data using Memorystore. - correct
    3. Move the data to BigQuery.
    4. Move the data to Bigtable.

e best option for speeding up data access in a **simple, cost-effective way** while using **Firestore** is:

✅ **Cache the data using Memorystore**

- **Memorystore (Redis or Memcached)** provides an **in-memory cache** to **reduce Firestore read latencies**.
- Firestore **bills per read operation**, so caching frequent queries significantly reduces costs.
- **Low-latency access** (~sub-millisecond) for commonly accessed data.
- **No schema changes** or migration required—just add caching to existing Firestore queries.

Why Not the Other Options?

1. ❌ **Move the data to Spanner**

- **Google Spanner** is a globally distributed, high-availability database but is **expensive** and best for **high-volume OLTP workloads**.
- Firestore is already scalable; Spanner is **overkill for simple caching**.

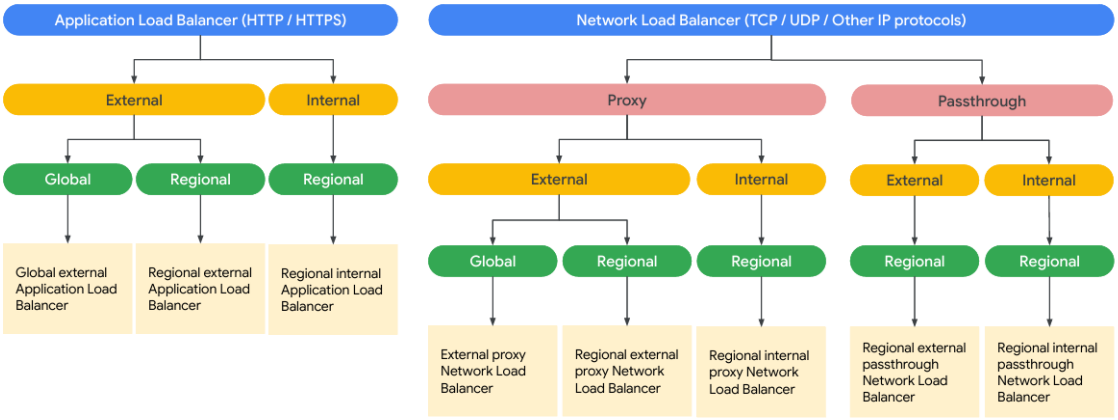
2. ❌ **Move the data to BigQuery**
   - **BigQuery is for analytics (OLAP), not fast transactional lookups**.
   - Optimized for **batch querying** and aggregations, not low-latency real-time queries.
3. ❌ **Move the data to Bigtable**
   - **Bigtable is best for massive, time-series data (IoT, logs)**.
   - **Not a direct replacement** for Firestore; complex to migrate.

Final Recommendation: Use Memorystore (Redis)

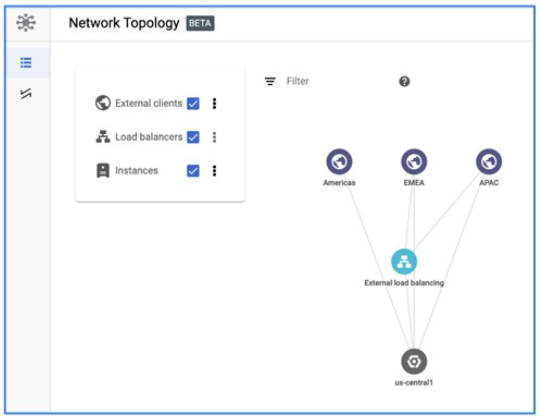
1. **Cache frequently read Firestore data** in **Memorystore (Redis)**.
2. **Reduce Firestore reads**, improving both **performance** and **cost-efficiency**.
3. **TTL (Time-to-Live) caching** ensures data remains fresh without manual invalidation.
4. **Minimal changes required**—just update the app to check **Redis cache first**, then fallback to Firestore.

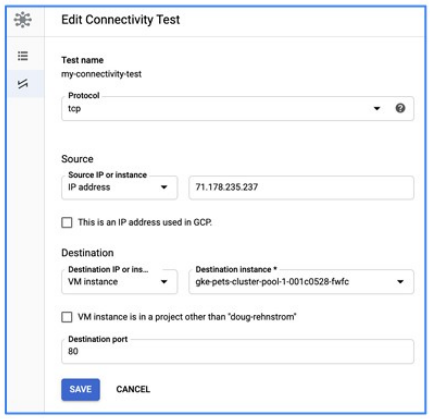
W6 – Network

Deployment modes available for Cloud Load Balancing



Network Intelligence Center can be used to visualize network topology and test network connectivity





Summary of Google Cloud load balancers

Load balancer	Deployment mode	Traffic type	Network Service Tier	Load-balancing scheme
Application Load Balancers	Global external	HTTP or HTTPS	Premium	EXTERNAL_MANAGED
	Regional external	HTTP or HTTPS	Standard	EXTERNAL_MANAGED
	Classic	HTTP or HTTPS	Global in Premium Regional in Standard	EXTERNAL
	Internal Always regional	HTTP or HTTPS	Premium	INTERNAL_MANAGED
Proxy Network Load Balancers	Global external	TCP with optional SSL offload	Global in Premium Regional in Standard	EXTERNAL
	Regional external	TCP	Standard only	EXTERNAL_MANAGED
	Internal Always regional	TCP without SSL offload	Premium only	INTERNAL_MANAGED
Passthrough Network Load Balancers	External Always regional	TCP, UDP, ESP, GRE, ICMP, and ICMPv6	Premium or Standard	EXTERNAL
	Internal Always regional	TCP or UDP	Premium only	INTERNAL

Network questions

You want a secure, private connection between your network and a Google Cloud network. There is not a lot of volume, but the connection needs to be extremely reliable. Which configuration below would you choose? VPN Cloud Interconnect VPN with high availability and Cloud Router. VPC peering

ChatGPT said:
ChatGPT

The best choice for a **secure, private, reliable** connection between your **on-premises network and Google Cloud** with **low volume but high reliability** is:

✅ VPN with High Availability and Cloud Router

- **High-availability VPN (HA VPN)** ensures **99.99% uptime**, providing **reliability and redundancy**.
- **Cloud Router** dynamically exchanges BGP routes, ensuring optimal path selection and failover.
- Ideal for **low-to-medium bandwidth needs** with **strong encryption (IPSec VPN)**.

Why Not the Other Options?

1. ❌ **VPN (Standard)**
   - Standard **VPN is not as reliable** as HA VPN.
   - **Single tunnel** means no automatic failover.
   - Less suitable for mission-critical workloads.

2. ❌ **Cloud Interconnect**
   - Provides **higher bandwidth (10 Gbps or more)** but **requires expensive physical fiber connections**.
   - **Not cost-effective** for low-volume traffic.

3. ❌ **VPC Peering**
   - Used for **connecting two VPCs within Google Cloud**, **not** for connecting to an on-premises network.
   - **Does not provide a private hybrid connection**.

Final Recommendation:

Use **VPN with High Availability + Cloud Router** for: ✅ **Reliable connectivity** (99.99% uptime).
✅ **Automatic failover** in case of tunnel failure.
✅ **Private, secure traffic** with **IPSec encryption**.
✅ **Dynamic routing** using **Cloud Router**.

Would you like guidance on setting up **HA VPN + Cloud Router**? 🚀

4o

You said:

You have a contract with a service provider to manage your Google VPC networks. You want to connect a network they own to your VPC. Both networks are in Google Cloud. Which Connection option should you choose? VPN with high availability and Cloud Router. VPC peering Cloud Interconnect VPN

ChatGPT said:

ChatGPT

The best choice for **connecting a service provider's network to your VPC within Google Cloud** is:

✅ VPC Peering

- **VPC Peering allows direct, private connectivity** between two VPCs in **Google Cloud**.
- **Low-latency, high-bandwidth** connection with **no additional cost for data transfer within the same region**.
- **Simple to configure** and does **not require external VPNs or physical links**.
- Traffic stays within Google's internal network, ensuring **high performance and security**.

Why Not the Other Options?

1. ❌ **VPN with High Availability and Cloud Router**
   - **Used for hybrid connections** (connecting an on-prem network to GCP), not for **two GCP VPCs**.
   - Adds **unnecessary encryption overhead** for internal traffic.

2. ❌ **Cloud Interconnect**
   - **Used for on-premises to Google Cloud** connections, requiring a **physical link**.
   - **Not needed when both networks are inside GCP**.

3. ❌ **VPN**
   - **Unnecessary complexity** for VPC-to-VPC communication.
   - **Higher latency** due to encryption overhead.

Final Recommendation: Use VPC Peering

✅ **Secure, private, and high-performance** connection.
✅ **No bandwidth bottlenecks** (as with VPN).
✅ **Low-cost** (no extra egress charges within the same region).
✅ **Simplifies management** by using Google Cloud's internal networking.

Would you like assistance in **configuring VPC Peering**? 🚀

4o

You said:

You are a large bank deploying an online banking service to Google Cloud. The service needs high volume access to mainframe data on-premises. Which connectivity option would likely be most suitable? VPN Cloud Interconnect HTTPS Peering

Best Choice: ✅ Cloud Interconnect

For a **large bank** deploying an **online banking service** that requires **high-volume access** to **on-premises mainframe data**, the most suitable option is **Cloud Interconnect**.

Why Cloud Interconnect?

- **Dedicated, High-Bandwidth Connection**: Supports **10 Gbps or 100 Gbps** links, making it ideal for high-volume banking transactions.
- **Low Latency & High Reliability**: Provides a **direct, private connection** between your data center and Google Cloud, ensuring minimal delays.

- **More Secure than VPN**: Since the traffic does not travel over the public internet, it's more **secure and compliant** with banking regulations (e.g., PCI DSS).
- **Hybrid Cloud Flexibility**: Perfect for banks that need **real-time access to on-prem mainframe data** from cloud-based applications.

Why Not the Other Options?

1. ❌ **VPN**
   - **Limited bandwidth** (~3 Gbps per tunnel with HA VPN).
   - Higher **latency** due to internet routing.
   - Not ideal for **high-volume banking transactions**.

2. ❌ **HTTPS**
   - HTTPS is an **application-layer security protocol**, not a **network connectivity solution**.
   - Does not provide **direct on-prem to cloud connectivity**.

3. ❌ **Peering**
   - **VPC Peering** is for **connecting two GCP networks**, not for **on-prem to cloud connectivity**.
   - **Direct Peering** (Google's peering service) requires a **public IP** and still relies on **the internet**.

Final Recommendation:

Use **Cloud Interconnect** for: ✅ **Reliable, low-latency hybrid banking workloads**.
✅ **High security & compliance** for banking regulations.
✅ **Efficient, high-volume mainframe data access**.
✅ **Scalability for future banking service expansion**.

Would you like help with **setting up Cloud Interconnect** for your bank? 🚀

4o

You said:

ou are deploying a large-scale web application with users all over the world and a lot of static content. Which load balancer configuration would likely be the most suitable? Network Load Balancer with SSL configured. close Network Load Balancer with SSL configured and the CDN enabled. Application Load Balancer with SSL configured and the CDN enabled. Application Load Balancer with SSL configured. This answer is not correct. The traffic type is not UDP.

ChatGPT said:

ChatGPT

Best Choice: ✅ Application Load Balancer with SSL configured and the CDN enabled

For a **large-scale web application** with **global users and a lot of static content**, the most suitable option is:

✅ Application Load Balancer (HTTP(S)) + SSL + Cloud CDN

- **Application Load Balancer (ALB)** is designed for **HTTP(S) traffic**, which is typical for web applications.
- **Cloud CDN** caches static content at **Google's global edge locations**, reducing latency and offloading traffic from backend servers.
- **SSL/TLS termination** improves security and performance by handling encryption at the load balancer level.

Why Not the Other Options?

1. ❌ **Network Load Balancer with SSL configured**
   - **Network Load Balancers (NLBs) are for TCP/UDP traffic**, not HTTP(S).
   - **Does not support Cloud CDN**, so static content delivery would be slow.
   - Not optimized for web applications.

2. ❌ **Network Load Balancer with SSL configured and the CDN enabled**
   - **Same issue as above**: NLB is for lower-level TCP/UDP traffic, not **layer 7 (HTTP/S)**.
   - **Cloud CDN does not integrate with Network Load Balancer**.

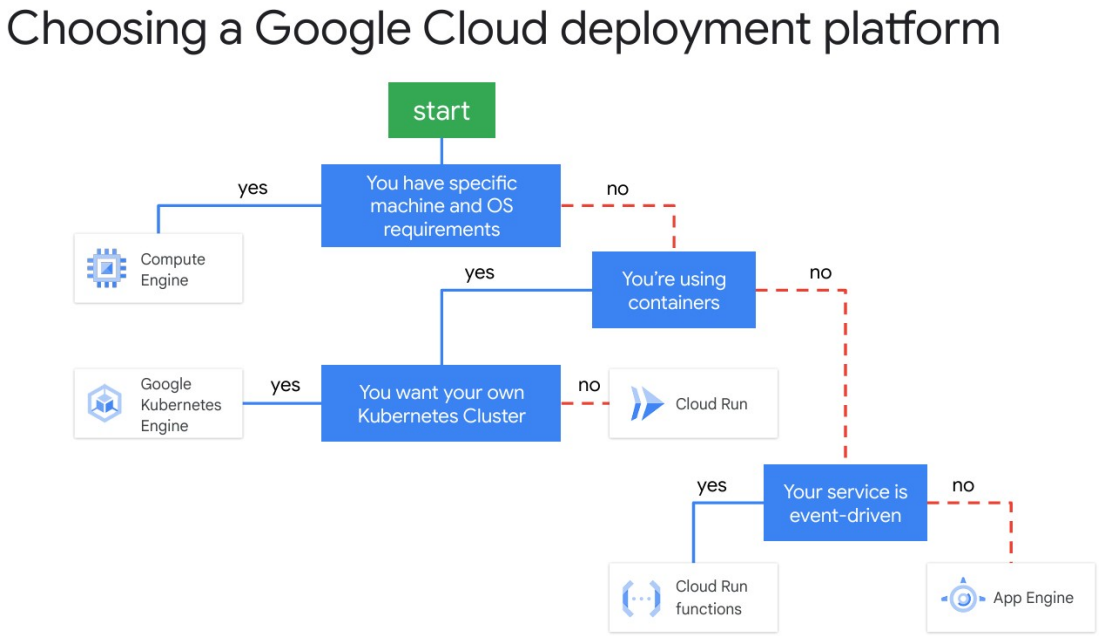
3. ❌ **Application Load Balancer with SSL configured (No CDN)**
   - While **ALB + SSL** ensures secure and scalable application delivery, **without Cloud CDN, static content** (images, CSS, JavaScript, etc.) must be served directly from origin servers, **increasing latency and backend load**.

Final Recommendation:

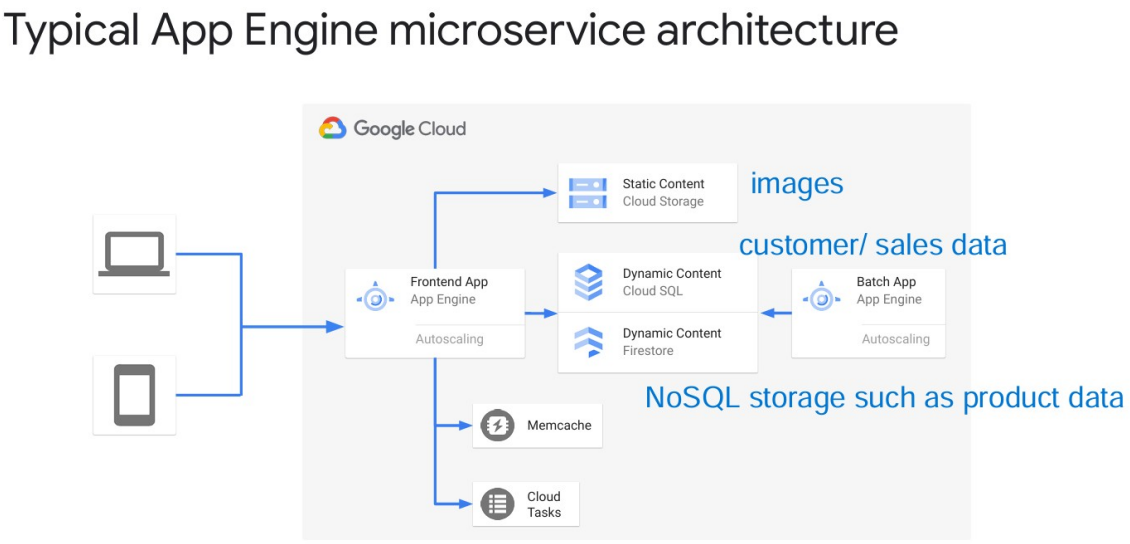
Use **Application Load Balancer (ALB) with SSL and Cloud CDN** for: ✅ **Global scalability & load balancing** for HTTP(S) traffic.
✅ **Faster content delivery via CDN edge locations**.
✅ **Secure connections with managed SSL certificates**.
✅ **Lower backend costs by caching static assets**.

Would you like help setting up **ALB + Cloud CDN** on Google Cloud?

Deploying applications

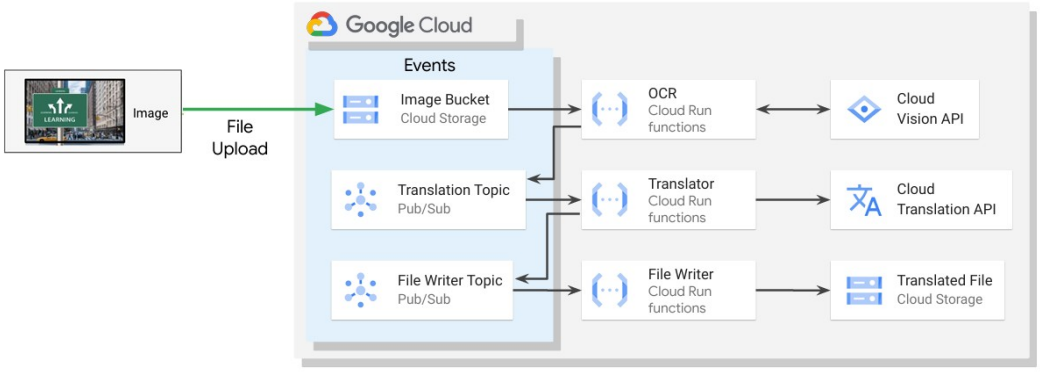


App engine



Cloud Run functions is great way to create loosely coupled, event-driven microservices

- Can be triggered by changes in a storage bucket, Pub/Sub messages, or web requests
- Completely managed, scalable, and inexpensive



App Engine vs. Compute Engine: Key Differences

Both **Google App Engine (GAE)** and **Google Compute Engine (GCE)** are services that allow you to deploy applications, but they serve **different purposes**.

Feature	**App Engine (GAE)**	**Compute Engine (GCE)**
Type	Fully managed PaaS (Platform-as-a-Service)	IaaS (Infrastructure-as-a-Service)
Management	Google manages everything (serverless)	You manage the VM, OS, and configurations
Use Case	Web apps, APIs, serverless workloads	Custom applications, VMs, databases, legacy apps
Scaling	Auto-scales based on demand	Manual or auto-scaling for VMs
OS Control	No control over OS, runtime only	Full control over OS (Windows, Linux)
Customization	Limited to supported languages	Any software, any OS
Pricing	Pay-per-use (per request)	Pay for VM uptime
Best For	Web apps, REST APIs, microservices	Windows/Linux workloads, .NET apps, DBs

Designing Reliable Systems

concepts

Concept	Definition	Example GCP Services	Example Use Case
Availability	Ensures uptime & access	Load Balancer, Cloud SQL HA, GKE	Web app running in multiple regions
Durability	Ensures data is never lost	Cloud Storage, Snapshots, BigQuery	Backing up user files securely
Scalability	Handles growing traffic efficiently	Autoscaler, GKE, Cloud Run	E-commerce site handling Black Friday traffic

CloudSQL can be implemented in ONLY 1 REGION !!!!!

Security

Separation of duties

Prevention of conflicts of interests

Detection of control failure

For example, security breaches and information theft:

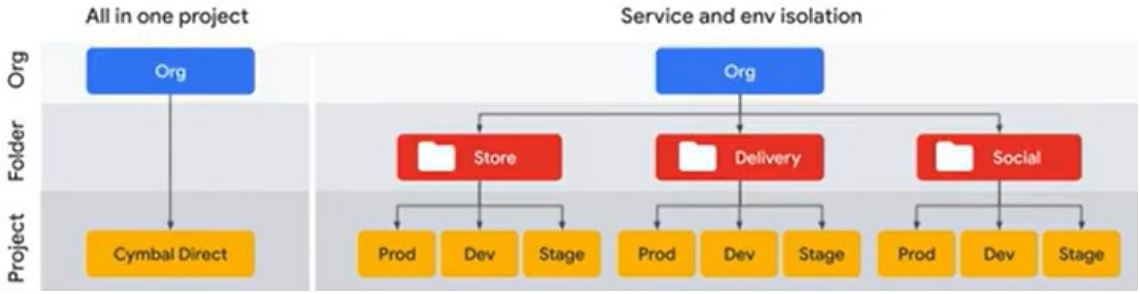
no one person can change or delete data without being detected.

Tools:

- Integrated firewall in your VPC network.
- Cloud Armor
- hardened versions of Linux
- secure boot

Projects

- Projects limit access.
- Projects limit the scope of damage.
- Projects are in your resource hierarchy.



Separation of duties

Separation of duties means:

- No one person can change or delete data without being detected.
- No one person can steal sensitive data.
- No one person is in charge of designing, implementing, and reporting on sensitive systems.

For example, the people who write the code shouldn't deploy the code, and those who deploy the code shouldn't be able to change it.

- Use multiple projects to separate duties.
- Different people can be given different rights in different projects.
- Use folders to help organize projects.

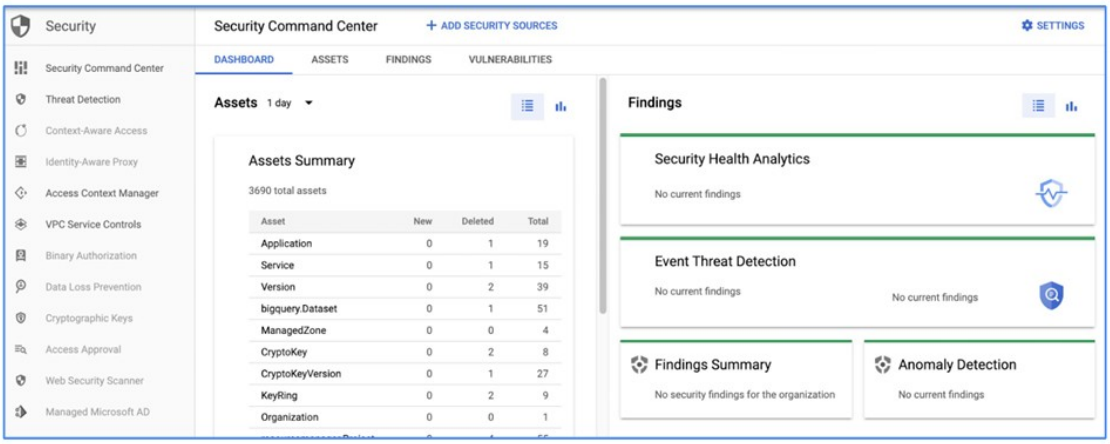
Regularly audit the Google Cloud logs to discover attacks

All Google Cloud services write to audit logs:

- Admin logs
- Data access logs
- VPC Flow logs
- Firewall logs
- System logs



Security Command Center provides access to organizational and project security configuration



Grant people access: Project -> Member -> Roles

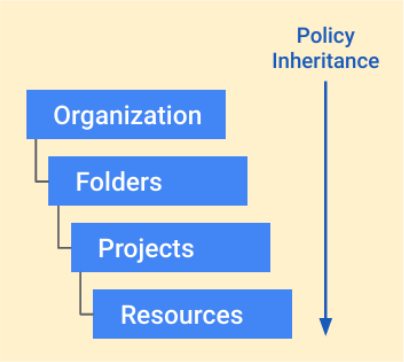
Use Identity and Access Management policies to secure environments and manage resources

Grant roles to Google groups rather than individuals

- Groups can be more granular than job roles.
- Use multiple groups for better control (such as *view only*).

Roles

- Prefer pre-defined roles over custom roles.
- Grant roles at the smallest scope needed (least privilege).
- Limit use of “owner” and "editor" roles.
- Consider hierarchy inheritance when assigning roles.



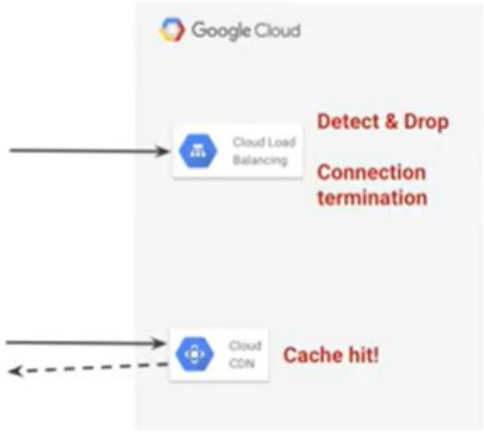
Cloud Identity-Aware Proxy (Cloud IAP)

is a **zero-trust security service** that controls **user access to applications and services** running on Google Cloud.

IAP sits in front of your application and ensures that only **authenticated and authorized** users can access it, regardless of whether it's deployed on **App Engine, Compute Engine, GKE, or Cloud Run**.

Leverage Google Cloud network services for DDoS protection

- Global load balancers detect attacks and drop them.
- Enabling the CDN will protect backend resources.





Google Cloud Armor - DDoS protection and Web Application Firewall (WAF) service

Google Cloud Armor is a **DDoS protection and Web Application Firewall (WAF) service** that helps secure applications from **cyber threats, malicious traffic, and large-scale attacks**.

🔻 It works with **Google Cloud HTTP(S) Load Balancers** to **filter and block unwanted traffic** before it reaches your backend services.

◆ Key Features of Google Cloud Armor

✅ **DDoS Protection** → Defends against **Layer 3 (network) & Layer 7 (application) attacks**.
✅ **Web Application Firewall (WAF)** → Blocks **SQL injection, XSS, and OWASP Top 10 threats**.
✅ **IP-Based Access Control** → Allows or denies access based on **IP address or geolocation**.
✅ **Adaptive Protection** → Uses **machine learning** to detect and mitigate **zero-day threats**.
✅ **Custom Security Policies** → Define **rules** to allow, deny, or rate-limit requests.
✅ **Real-Time Logging & Monitoring** → Integrates with **Cloud Logging & Security Command Center**.

Cloud Armor vs. Other Security Solutions

Feature	Google Cloud Armor	Firewall Rules	Cloud IAP
DDoS Protection	✅ Yes	❌ No	❌ No
WAF for Web Apps	✅ Yes	❌ No	❌ No
IP/Geo-based Rules	✅ Yes	✅ Yes	❌ No
User Identity Control	❌ No	❌ No	✅ Yes
Integrates with HTTP(S) Load Balancer	✅ Yes	❌ No	✅ Yes

Google Cloud Armor supports layer 7 web application firewall (WAF) rules

- Predefined rules for preventing common attacks like SQL injection and cross-site scripting
- Flexible rules language allows you to allow or deny traffic using request headers, geographic location, ip addresses, cookies, etc.
- Examples:

```
inIpRange(origin.ip, '9.9.9.0/24')
request.headers['cookie'].contains('80=BLAH')
origin.region_code == 'AU'
inIpRange(origin.ip, '1.2.3.4/32') &&
request.headers['user-agent'].contains('WordPress')
evaluatePreconfiguredExpr('xss-canary')
```

Encryption

DEK & KEK

Google Cloud follows a **two-layer encryption model** to protect your data:

1️⃣ **Data Encryption Key (DEK)** 🔑 → Encrypts the actual data using **AES-256** (symmetric encryption).
2️⃣ **Key Encryption Key (KEK)** 🔐 → Encrypts the **DEK itself** for additional security.

How Each Feature Helps Against DDoS Attacks

Feature	How It Helps Mitigate DDoS?
✅ **Google Cloud Armor**	**Primary DDoS defense** → Detects and blocks large-scale attack traffic (L3/L4 DDoS, OWASP Top 10 threats).
✅ **Global External Application Load Balancer**	Absorbs high traffic loads and distributes requests **across multiple backend instances**.
✅ **Cloud CDN**	Reduces origin server load by **caching static content**, preventing exhaustion attacks on your backend.

You don't want programmers to have access to production resources. What's the easiest way to do this in Google Cloud?

Create a firewall rule that blocks developer access to production servers and databases.

checkCreate development and production projects, and don't give developers access to production.

Use different service accounts for production and development resources with your project.

Set up private access and Identity-Aware Proxy.

This is the correct answer, because the simplest way is to have separate projects and not give developers access to the production project.

check

2.

What Google Cloud service can you use to enforce the principle of least privilege when using Google Cloud?

Encryption keys

checkIAM members and roles

SSL certificates

Firewall rules

This is the correct answer, because the principle of least privilege requires user permissions that are just enough to do what they need, and no more. IAM provides this level of control.

check

3.

Which Google Cloud features could help reduce the risk of DDoS attacks?

Global external Application Load Balancer

checkAll of these

Google Cloud Armor

Cloud CDN

This answer is correct, because a global external Application Load Balancer mitigates and absorbs many layer 4 below attacks such as SYN flood, IP fragment floods, and port exhaustion. Cloud CDN caches cacheable content at points of presence close to users. In the event of a DDoS attack for cacheable content, the requests are sent to points of presence, not to your servers/infrastructures, thus increasing the likelihood of the attack being absorbed. Google Cloud Armor is built for DDos mitigation, working with Cloud Load Balancing to detect DDoS attacks.

check

4.

What do you have to do to enable encryption when using Cloud Storage?

Enable encryption and upload a key.

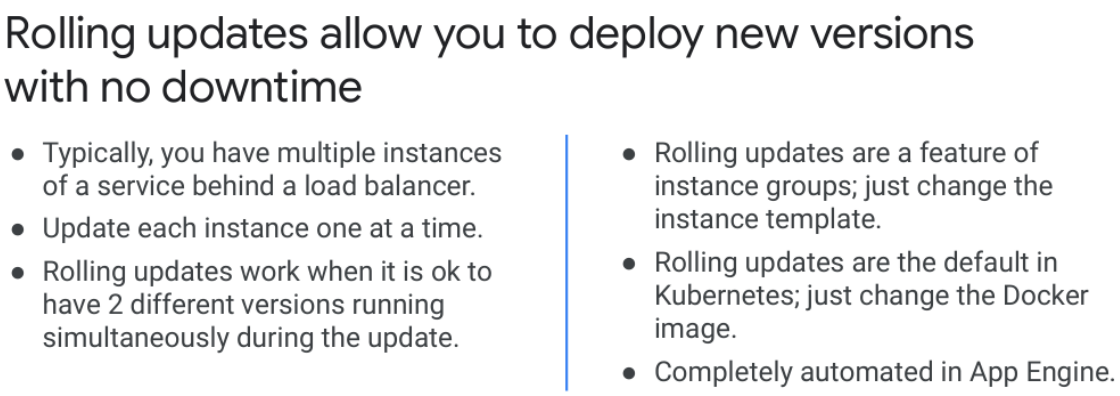
Check Nothing as encryption is enabled by default.

Create an encryption key using Cloud Key Management Service, and select it when creating a Cloud Storage bucket.

Simply enable encryption when configuring a bucket.

This answer is correct, because Cloud Storage always encrypts data on the server side before it is written to disk. For server side encryption there are options of customer supplied or customer managed encryption keys, but these are only usually used for compliance reasons and are not necessary.

Maintenance and Monitoring



Canary releases can be used prior to a rolling update to reduce the risk

- The current service version continues to run.
- Deploy an instance of the new version and give it a portion of requests.
- Monitor for errors.

- In Compute Engine, you can create a new instance group and add it as an additional backend in in your load balancer.
- In Kubernetes, create a new pod with the same labels as the existing pods; the service will automatically route a portion of requests to it.
- In App Engine, use the Traffic Splitting feature.

Use a blue/green deployment when you don't want multiple versions of a service running simultaneously

- The blue deployment is the current version.
- Create an entirely new environment (the green).
- Once the green deployment is tested, migrate client requests to it.
- If failures occur, switch it back.

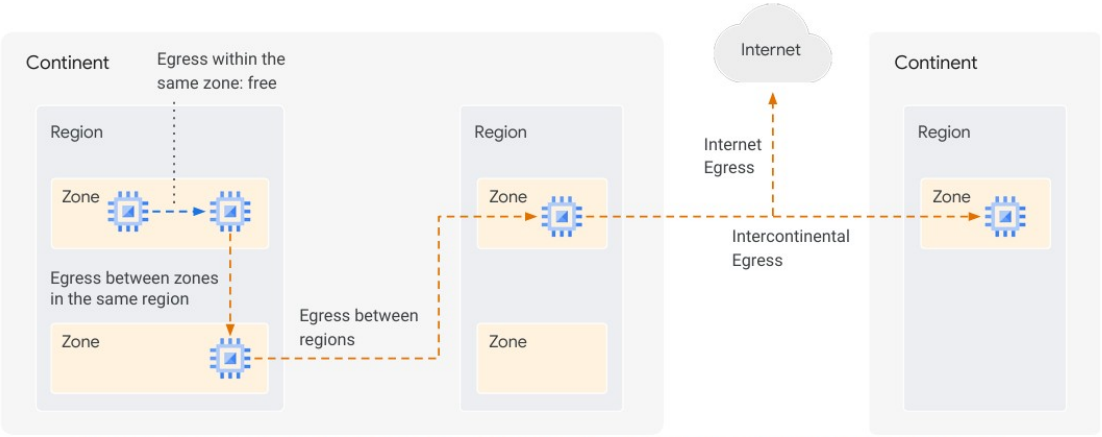
- In Compute Engine, you can use DNS to migrate requests from one load balancer to another.
- In Kubernetes, configure your service to route to the new pods using labels.
  - Simple configuration change
- In App Engine, use the Traffic Splitting feature.

Optimizing disk cost

- Don’t over-allocate disk space.
- Determine what performance characteristics your applications require:
  - I/O Pattern: small reads and writes or large reads and writes
  - Configure your instances to optimize storage performance.
- Depending on I/O requirements, consider Standard over SSD disks.

Monthly capacity	Standard PD	SSD PD
10 GB	$0.40	$1.70
1 TB	$40	$170
16 TB	$655.36	$5,570.56

To optimize network costs, keep machines close to your data

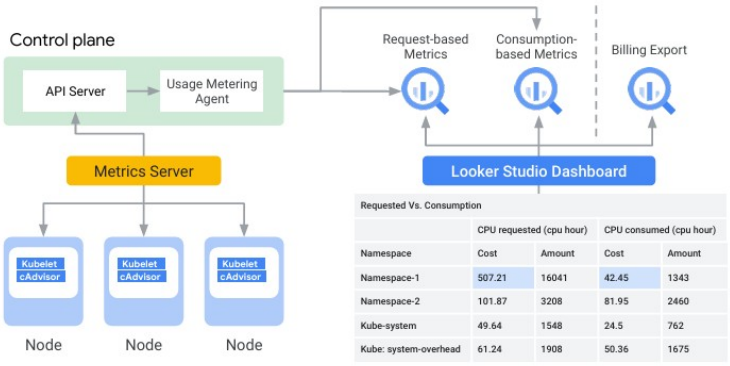


Egress between zones in the same region is charged and all internet egress is charged.

Google Cloud

GKE usage metering can prevent over-provisioning Kubernetes clusters

Compares requested resources with consumed resources.



Terraform – W7

Lab: https://www.cloudskillsboost.google/course_templates/636/labs/464836

- Import existing infrastructure into your Terraform configuration.
- Build and reference your own Terraform modules.
- Add a remote backend to your configuration.
- Use and implement a module from the Terraform Registry.
- Re-provision, destroy, and update infrastructure.
- Test connectivity between the resources you've created.

Initial Configuration files

```
main.tf
variables.tf
modules/
└── instances
    ├── instances.tf
    ├── outputs.tf
    └── variables.tf
└── storage
    ├── storage.tf
    ├── outputs.tf
```

```
└── variables.tf
```

Create structure – run commands:

variables.tf

```
variable "project_id" {
  description = "The ID of the project to create the bucket in."
  type        = string
  default     = ""
}
variable "region" {
  description = "The region of the project."
  type        = string
  default     = "us-central1"
}
variable "zone" {
  description = "The zone of the project."
  type        = string
  default     = "us-central1-f"
}
```

main.tf – Adding provider

```
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
       version = ">= 4.64.0, < 6.0.0"    }
  }
  required_version = ">= 1.0.0"
}

provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}
```

Import infrastructure –

create in console 2 VM's - **tf-instance-1** and **tf-instance-2**

import these 2 vms to terraform module **– here**

Add the module reference into the main.tf

Create structure

```
cd ~
mkdir -p ~/modules/instances
mkdir -p ~/modules/storage
cd ~/modules/instances
touch instances.tf  outputs.tf variables.tf
cd ~/modules/storage
touch storage.tf  outputs.tf variables.tf
```

Adding in main.tf (project root)

```
module "instances" {
 source = "./modules/instances"
 zone = var.zone
}
module " storage" {
 source = "./modules/ storage"

 name      = var.name
 project_id = var.project_id
 region   = var.region
 zone = var.zone
}
```

Initiate after modules:

```
Run command: terraform init
```

Import 2 instances to existing modules

Update template with the existing VM's

Google reference here

Add resources in file: ~/modules/instances/instances.tf

```
resource "google_compute_instance" "tf-instance-1" {
 name         = "tf-instance-1"
 machine_type = "e2-micro"
 zone = var.zone

 tags = ["generated", "terraform"]

 boot_disk {
  initialize_params {
   image = "debian-11-bullseye-v20250123"
   labels = {
    my_label = "value"
   }
  }
 }

 network_interface {
  network = "default"
 }

 metadata_startup_script = <<-EOT
    #!/bin/bash
  EOT
 allow_stopping_for_update = true

}
```

Run command: terraform import to the State

```
terraform import module.instances.google_compute_instance.tf-instance-1
projects/qwiklabs-gcp-04-95e26805f4df/zones/us-east1-d/instances/tf-instance-1

module.instances.google_compute_instance.tf-instance-1: Importing from ID
"projects/qwiklabs-gcp-04-95e26805f4df/zones/us-east1-d/instances/tf-instance-1"...

module.instances.google_compute_instance.tf-instance-1: Import prepared!

  Prepared google_compute_instance for import

module.instances.google_compute_instance.tf-instance-1: Refreshing state... [id=projects/qwiklabs-gcp-04-
95e26805f4df/zones/us-east1-d/instances/tf-instance-1]


Import successful!


The resources that were imported are shown above. These resources are now in

your Terraform state and will henceforth be managed by Terraform.


student_04_0b3586da56c8@cloudshell:~ (qwiklabs-gcp-04-95e26805f4df)$ terraform import
module.instances.google_compute_instance.tf-instance-2 projects/qwiklabs-gcp-04-95e26805f4df/z

ones/us-east1-d/instances/tf-instance-2

module.instances.google_compute_instance.tf-instance-2: Importing from ID
"projects/qwiklabs-gcp-04-95e26805f4df/zones/us-east1-d/instances/tf-instance-2"...

module.instances.google_compute_instance.tf-instance-2: Import prepared!

  Prepared google_compute_instance for import

module.instances.google_compute_instance.tf-instance-2: Refreshing state... [id=projects/qwiklabs-gcp-04-
95e26805f4df/zones/us-east1-d/instances/tf-instance-2]


Import successful!
```

The resources that were imported are shown above. These resources are now in

your Terraform state and will henceforth be managed by Terraform.

Summarize: Action

1 Run terraform import to add the VM to Terraform state.

2 Run terraform plan to detect differences.

3 Use gcloud compute instances describe to get missing attributes.

4 Manually update instances.tf with the correct attributes.

5 Run terraform apply to align Terraform with the imported VM.

Task 3. Configure a remote backend

Add bucket resource in storage.tf – reference: here

In storage/main.tf – use attributes:

- location = "US"
- force_destroy = true
- uniform_bucket_level_access = true

```
cat modules/storage/storage.tf
resource "google_storage_bucket" "bucket-for-state" {
 name       = var.storage_name
 location    = "US"
 uniform_bucket_level_access = true
}student_01_6892095d335f@cloudshell:~ (qwiklabs-gcp-01-7fd624ea2f41)$ cat modules/storage/variables.tf
variable "storage_name" {
 description = "Storage name"
 type       = string
}student_01_6892095d335f@cloudshell:~ (qwiklabs-gcp-01-7fd624ea2f41)$ tail variables.tf
variable "zone" {
 description = "The zone of the project."
 type       = string
 default     = "us-east4-a"
}
variable "storage_name" {
 description = "Storage name"
 type       = string
 default     = "tf-bucket-070479"
}
```

Configure this storage bucket as the remote backend inside the main.tf file. Be sure to use the **prefix** terraform/state

Add in main.tf:

```
terraform {
```

```
  backend "gcs" {
    bucket  = "tf-bucket-070479"
    prefix  = "terraform/state"
  }
}
# terraform {
#  backend "local" {
#   path = "terraform/state/terraform.tfstate"
#  }
# }
```

Adding output:

In file ~/modules/instances/outputs.tf

```
output "instance_id" {
  description = "The ID of the created VM instance"
  value       = google_compute_instance.tf-instance-1.id
}

output "instance_name" {
  description = "The name of the VM instance"
  value       = google_compute_instance.tf-instance-1.name
}

output "instance_zone" {
  description = "The zone where the VM instance is deployed"
  value       = google_compute_instance.tf-instance-1.zone
}

output "instance_external_ip" {
  description = "The external IP address of the VM instance"
  value       = google_compute_instance.tf-instance-1.network_interface[0].access_config[0].nat_ip
}
```

```
output "instance_internal_ip" {
 description = "The internal IP address of the VM instance"
 value       = google_compute_instance.tf-instance-1.network_interface[0].network_ip
}
```

Reference Outputs in main.tf Root Module

```
module "instances" {
 source  = "./modules/instances"
 region  = var.region
 zone    = var.zone
}

module "storage" {
 source     = "./modules/storage"
 project_id = var.project_id
}

output "vm_name" {
 value = module.instances.instance_name
}

output "vm_ip" {
 value = module.instances.instance_external_ip
}

output "bucket_url" {
 value = module.storage.bucket_url
}
```

Use a module from the Registry – Create Firewall and attach to the network

```
cat main.tf
terraform {
 required_providers {
```

```
    google = {
      source  = "hashicorp/google"
       version = ">= 4.64.0, < 6.0.0"    }
  }
  required_version = ">= 1.0.0"

  backend "gcs" {
    bucket = "tf-bucket-675385"  # Replace with your actual GCS bucket name
    prefix = "terraform/state"     # Required prefix for grading
  }

}

provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}




module "instances" {

  source = "./modules/instances"

}

module "storage" {

  source = "./modules/storage"

}

module "network" {
```

```
source  = "terraform-google-modules/network/google"

version = "7.5.0"



project_id   = var.project_id

network_name = "tf-vpc-528386"

routing_mode = "GLOBAL"



subnets = [

 {

   subnet_name   = "subnet-01"

   subnet_ip     = "10.10.10.0/24"

   subnet_region = "us-east1"

 },

 {

   subnet_name   = "subnet-02"

   subnet_ip     = "10.10.20.0/24"

   subnet_region = "us-east1"

 }
```

```
 ]

}



resource "google_compute_firewall" "tf-firewall" {

 name    = "tf-firewall"

 network = "tf-vpc-528386"



 allow {

   protocol = "tcp"

   ports    = ["80"]

 }



 source_ranges = ["0.0.0.0/0"]

}
student_04_0b3586da56c8@cloudshell:~ (qwiklabs-gcp-04-95e26805f4df)$



 cat modules/instances/instances.tf
resource "google_compute_instance" "tf-instance-1" {
 name         = "tf-instance-1"
```

```
  machine_type = "e2-standard-2"
  zone = var.zone

  tags = ["generated", "terraform"]

  boot_disk {
    initialize_params {
      image = "debian-11-bullseye-v20250123"
      labels = {
        my_label = "value"
      }
    }
  }

  network_interface {
    network    = "tf-vpc-528386"
    subnetwork = "subnet-01" # Connect to subnet-01
  }

  metadata_startup_script = <<-EOT
      #!/bin/bash
    EOT
  allow_stopping_for_update = true

}
resource "google_compute_instance" "tf-instance-2" {
  name         = "tf-instance-2"
  machine_type = "e2-standard-2"
  zone = var.zone

  tags = ["generated", "terraform"]

  boot_disk {
    initialize_params {
      image = "debian-11-bullseye-v20250123"
```

```
    labels = {
      my_label = "value"
    }
  }
 }

 network_interface {
  network    = "tf-vpc-528386"
  subnetwork = "subnet-02"
 }


 metadata_startup_script = <<-EOT
     #!/bin/bash
  EOT
 allow_stopping_for_update = true

}


student_04_0b3586da56c8@cloudshell:~ (qwiklabs-gcp-04-95e26805f4df)$

 cat modules/storage/storage.tf
resource "google_storage_bucket" "bucket-for-state" {
 name       = var.storage_name
 location   = "US"
 uniform_bucket_level_access = true
}

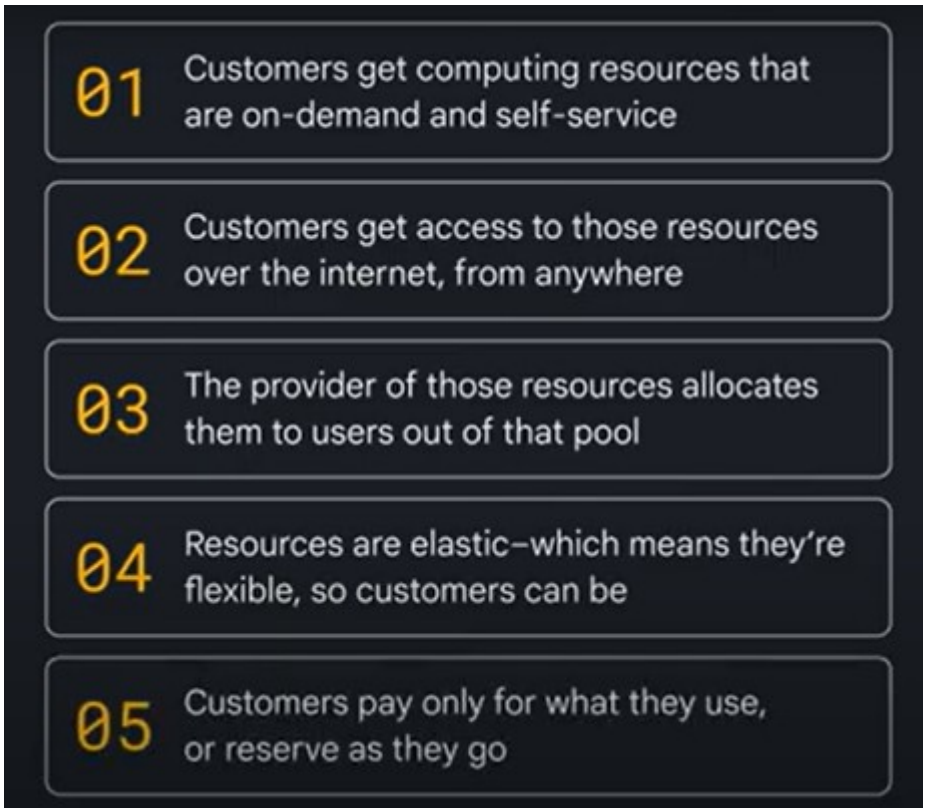

student_04_0b3586da56c8@cloudshell:~ (qwiklabs-gcp-04-95e26805f4df)$ cat modules/instances/variables.tf
variable "project_id" {
 description = "The ID of the project to create the bucket in."
 type       = string
 default    = "qwiklabs-gcp-04-95e26805f4df"
```

```
}
variable "region" {
  description = "The region of the project."
  type        = string
  default     = "us-east1"
}
variable "zone" {
  description = "The zone of the project."
  type        = string
  default     = "us-east1-d"
}
```

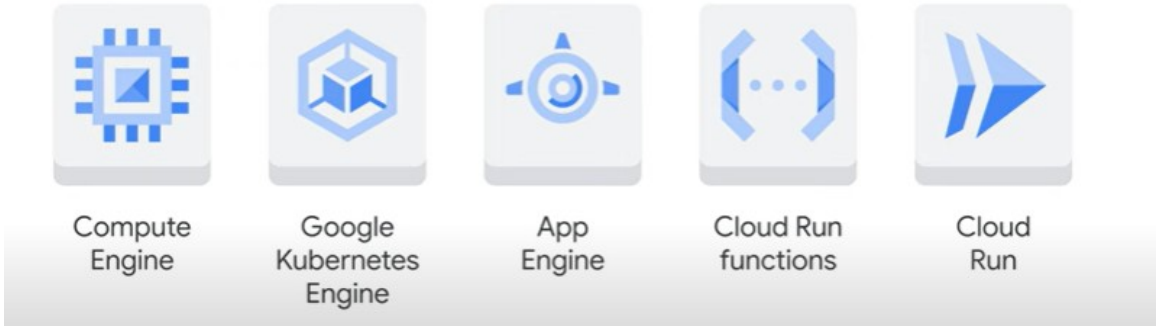
```
resource "google_storage_bucket" "terraform_state" {
  name                        = "tf-bucket-675385"
  location                    = var.location
  force_destroy               = var.force_destroy
  uniform_bucket_level_access = var.uniform_bucket_level_access
}
```

W8 – EKS

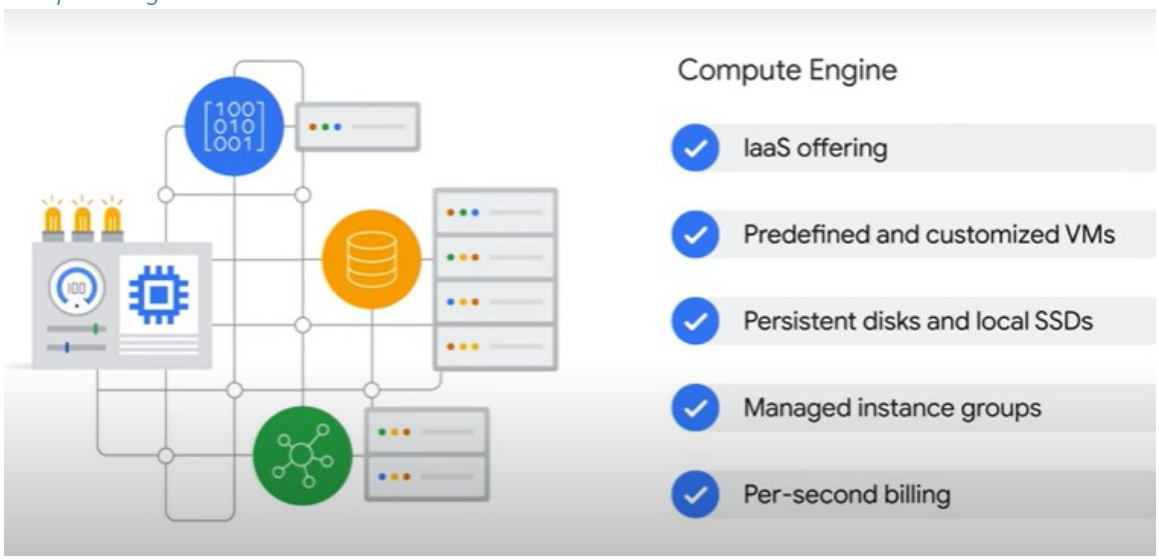
What is cloud



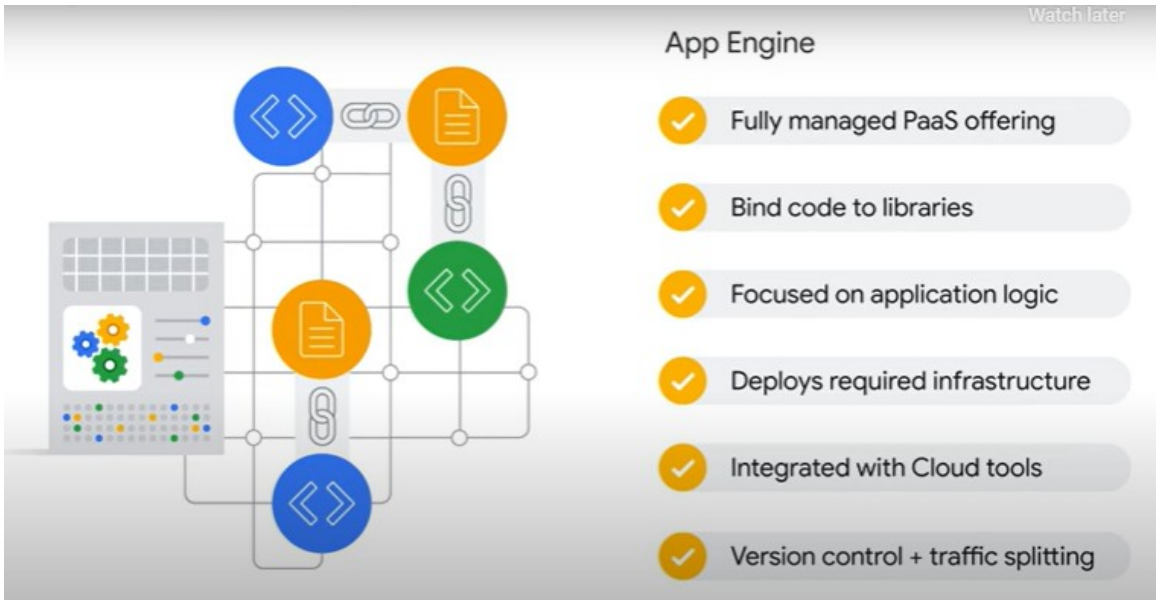
Computing services



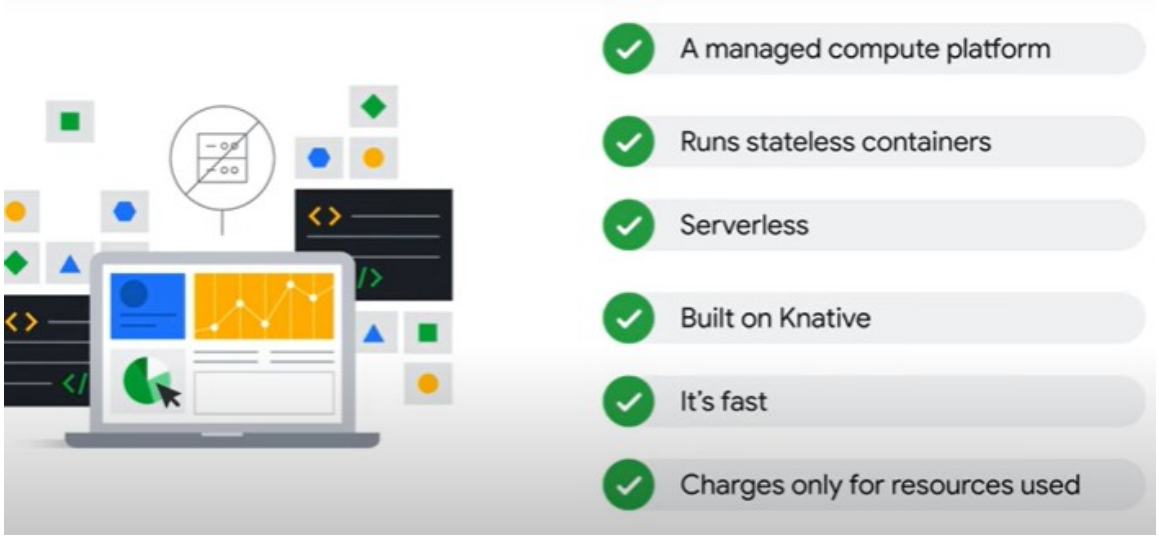
Compute Engine



App Engine – PaaS mainly for web app



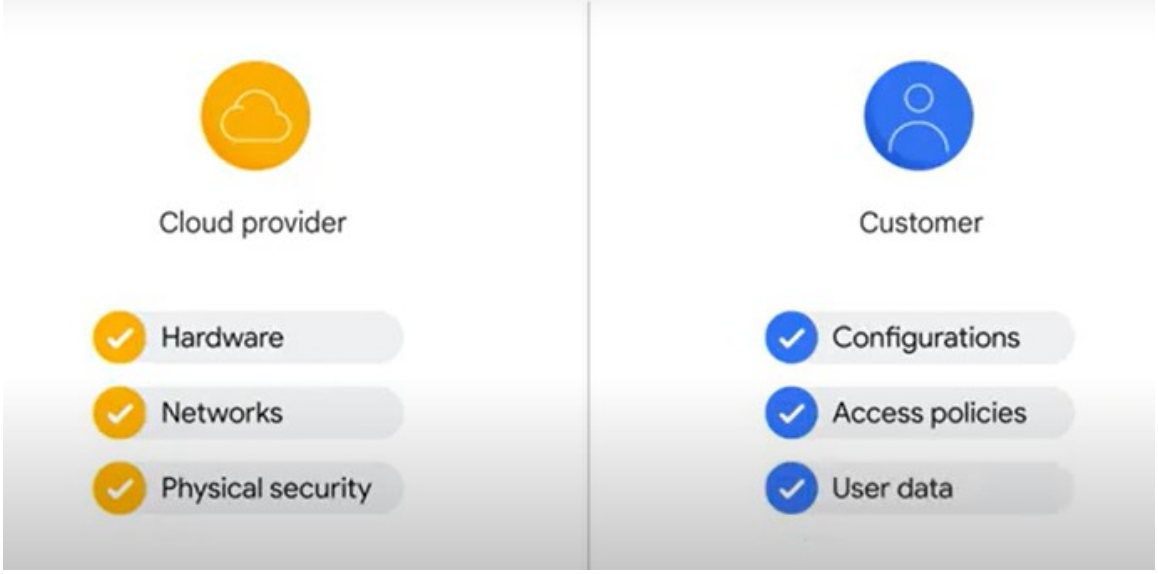
Cloud Run – container based



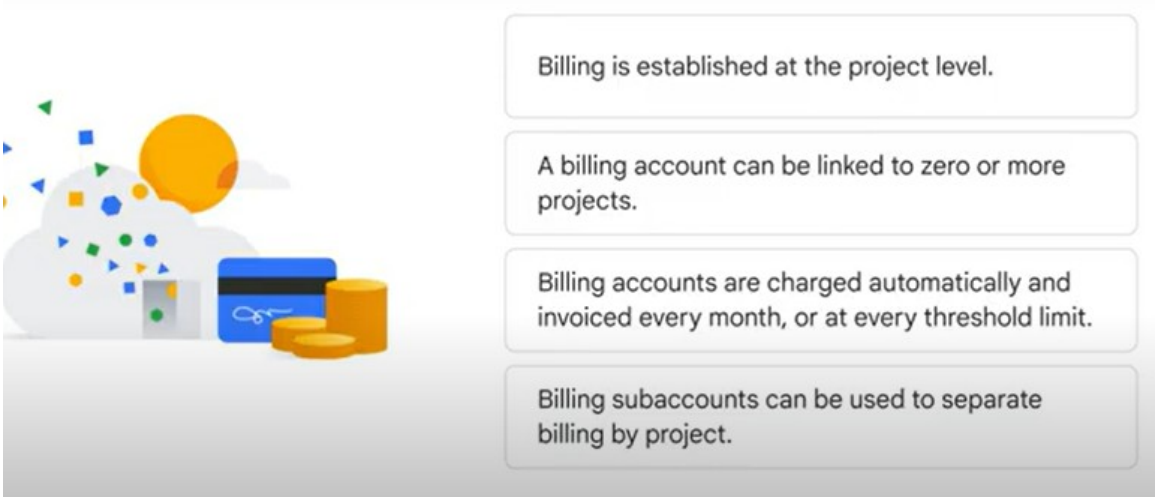
Feature	Cloud Run – Container based	App Engine – PaaS mainly for web dev
Deployment Model	Containerized workloads (Docker containers)	PaaS (Platform-as-a-Service) for web apps
Runtime Environment	Any runtime, as long as it runs in a container	Standard (predefined runtimes) & Flexible (custom runtimes)
Scaling	Scales to zero, automatically scales up based on traffic	Standard: scales to zero, automatic scaling; Flexible: VM-based, slower scaling
Traffic Splitting	Supports traffic splitting	Supports traffic splitting
Customization	Full control over dependencies	Limited in Standard, more flexible in Flexible
Networking	Fully private networking via VPC connectors	Standard: restricted networking, Flexible: VPC integration
Pricing	Pay-per-use (per request and execution time)	Standard: Pay-per-use, Flexible: VM-based pricing
Use Case	Best for APIs, microservices, event-driven workloads	Best for web applications with automatic scaling
Vendor Lock-in	Low (standard containers can run anywhere)	Higher (Standard runs only on GCP)
DevOps	Works well with CI/CD pipelines	Limited in Standard, better in Flexible

Feature	Cloud Run – Container based	App Engine – PaaS mainly for web dev
Integration	(Cloud Build, GitHub Actions, Jenkins)	

Security Responsibilities



Billing

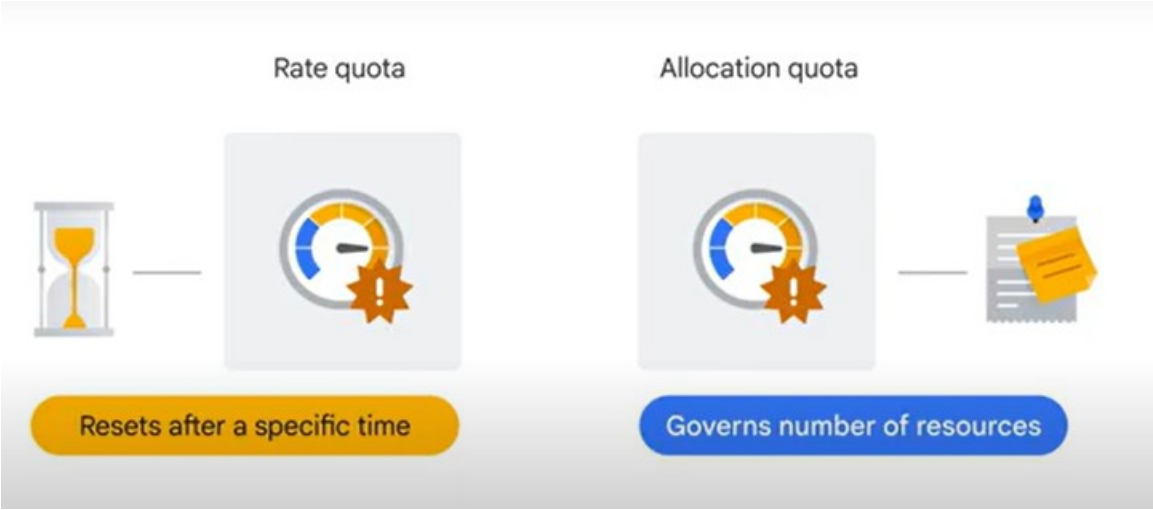


Budget : helps you **monitor** and **control** spending once you're using Google Cloud.

a fixed limit, or it can be tied to another metric; for example, a percentage of the previous month's spend.

Quote helps you **predict** costs before deploying resources.

Quote



K8S – GKE

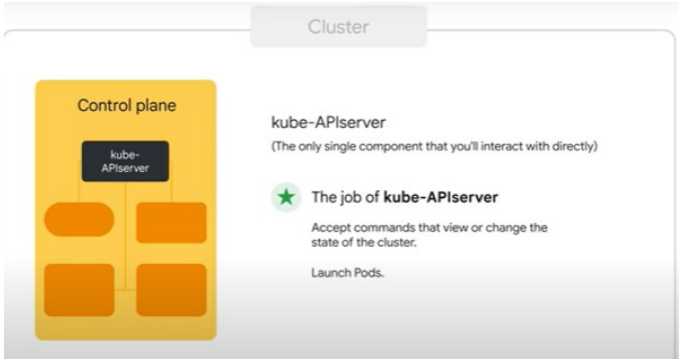
K8S object:

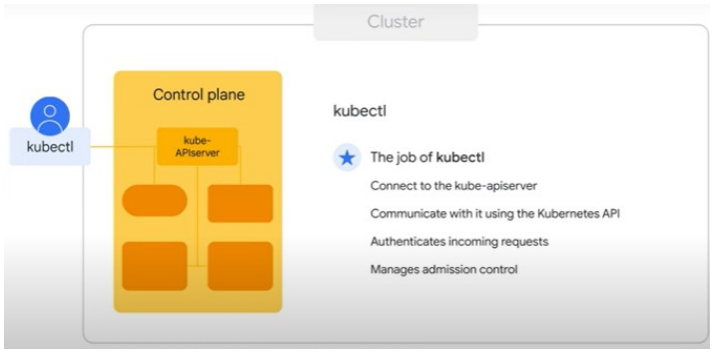
A Kubernetes object is defined as a persistent entity that represents the state of something running in a cluster: its desired state and its current state.

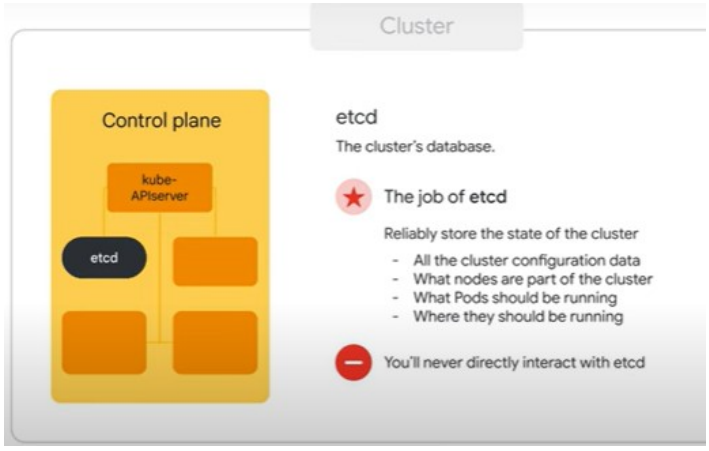
Various kinds of objects represent:

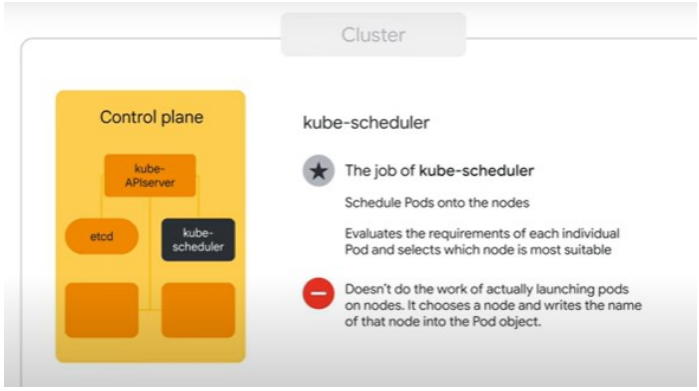
containerized applications, the resources available to them, and the policies that affect their behavior.

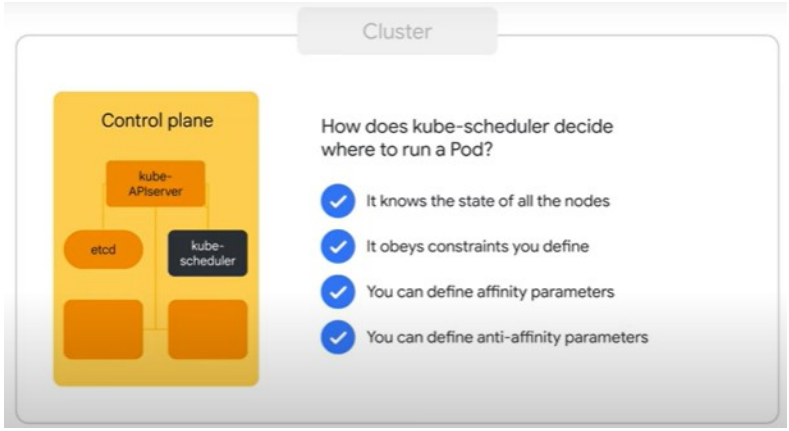


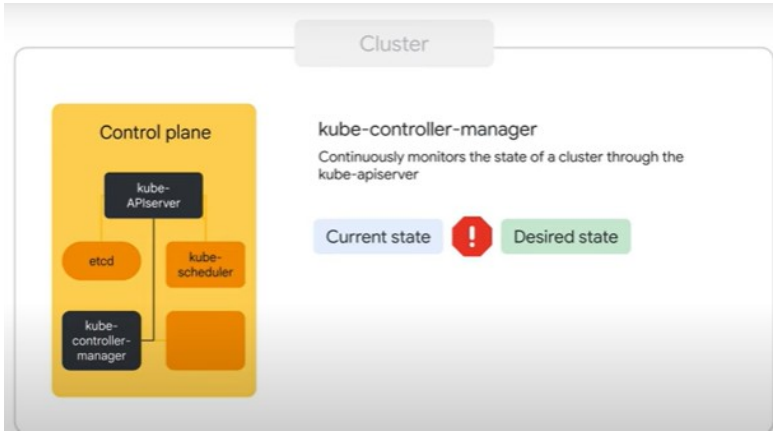

Cluster
Control plane
kube-APIserver
kube-APIserver
(The only single component that you'll interact with directly)
The job of kube-APIserver
Accept commands that view or change the state of the cluster.
Launch Pods.

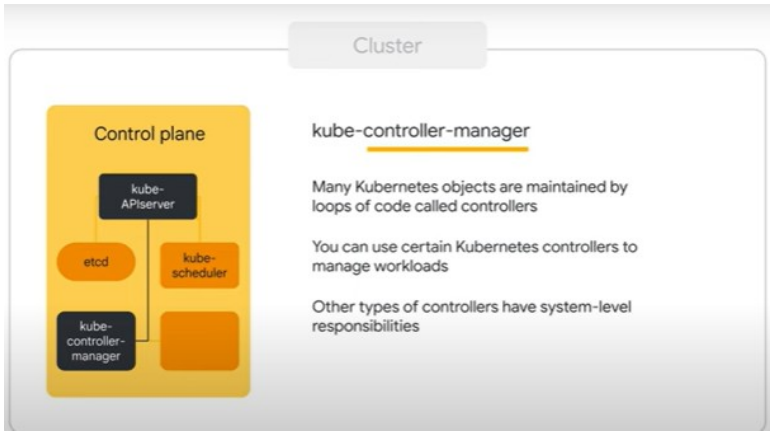

Cluster
kubectl
Control plane
kube-APIserver
kubectl
The job of kubectl
Connect to the kube-apiserver
Communicate with it using the Kubernetes API
Authenticates incoming requests
Manages admission control

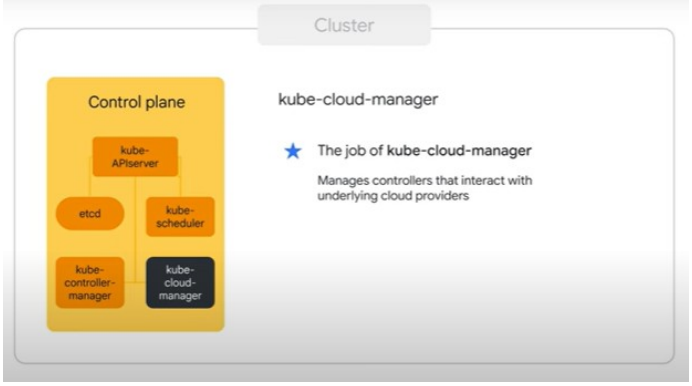

Cluster
Control plane
kube-APIserver
etcd
etcd
The cluster's database.
The job of etcd
Reliably store the state of the cluster
- All the cluster configuration data
- What nodes are part of the cluster
- What Pods should be running
- Where they should be running
You'll never directly interact with etcd


Cluster
Control plane
kube-APIserver
etcd
kube-scheduler
kube-scheduler
The job of kube-scheduler
Schedule Pods onto the nodes
Evaluates the requirements of each individual Pod and selects which node is most suitable
Doesn't do the work of actually launching pods on nodes. It chooses a node and writes the name of that node into the Pod object.

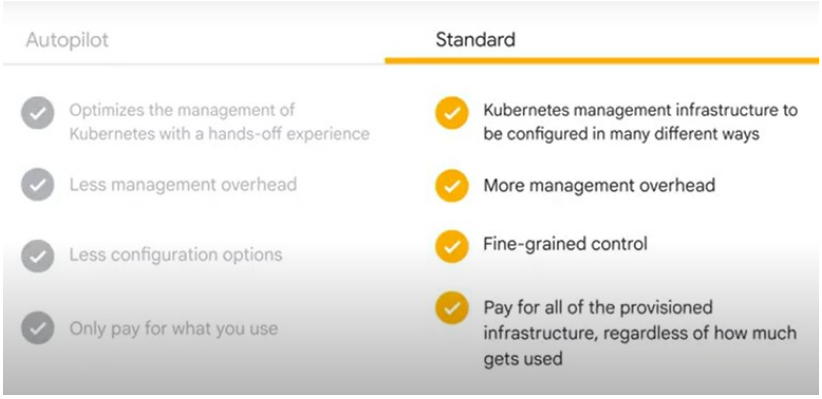

Cluster
Control plane
kube-APIserver
etcd
kube-scheduler
How does kube-scheduler decide where to run a Pod?
It knows the state of all the nodes
It obeys constraints you define
You can define affinity parameters
You can define anti-affinity parameters


Cluster
Control plane
kube-APIserver
etcd
kube-scheduler
kube-controller-manager
kube-controller-manager
Continuously monitors the state of a cluster through the kube-apiserver
Current state
Desired state


Cluster
Control plane
kube-APIserver
etcd
kube-scheduler
kube-controller-manager
kube-controller-manager
Many Kubernetes objects are maintained by loops of code called controllers
You can use certain Kubernetes controllers to manage workloads
Other types of controllers have system-level responsibilities

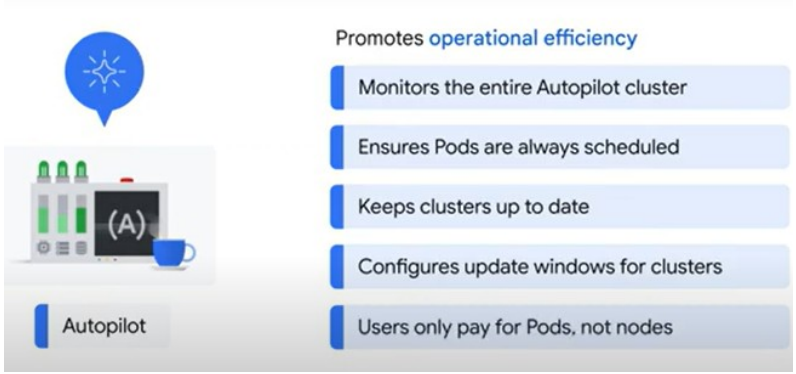


GKE Autopilot



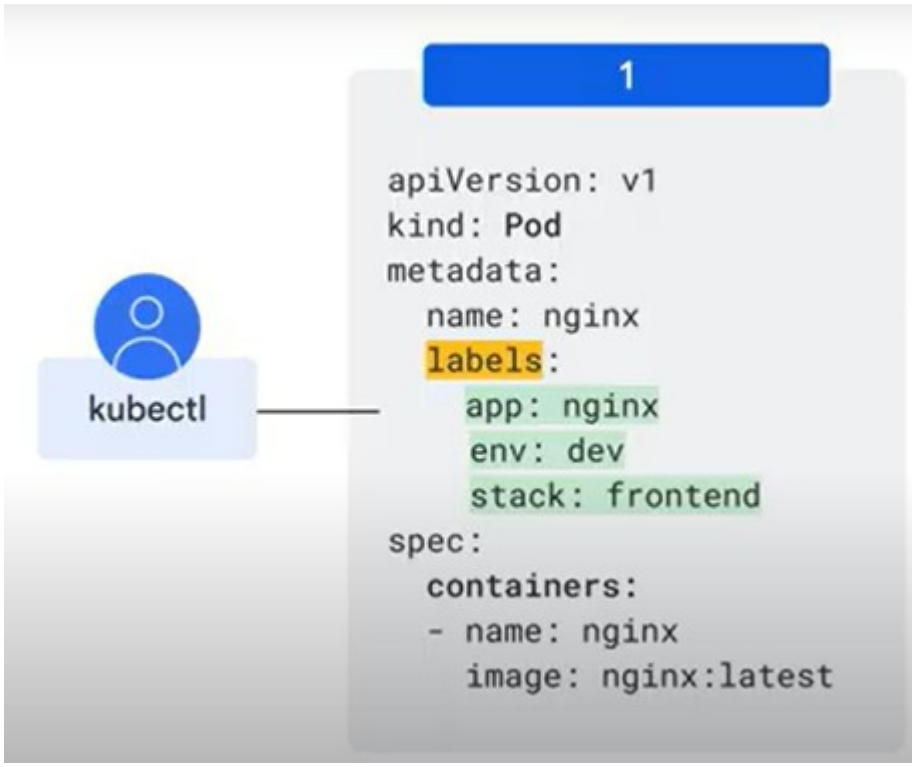


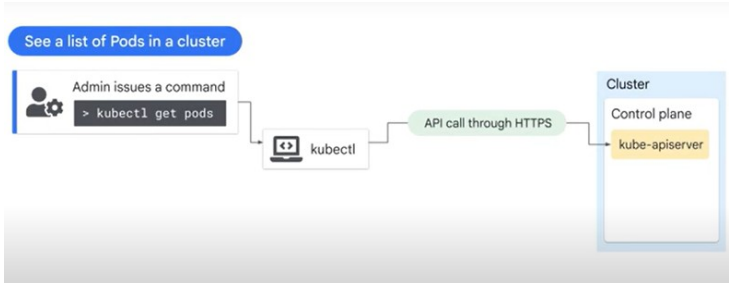


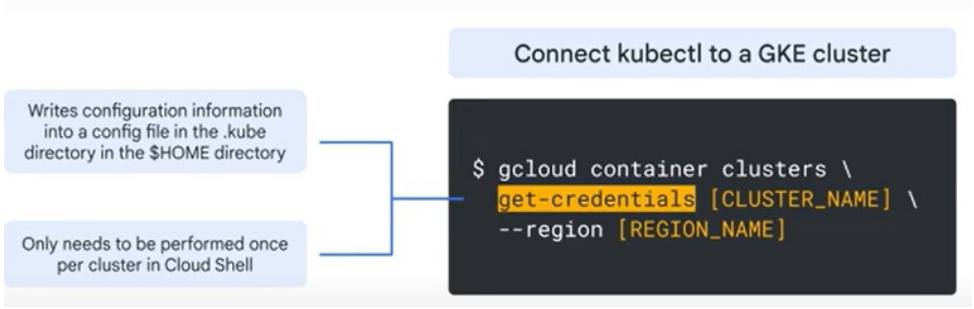

Promotes operational efficiency
Monitors the entire Autopilot cluster
Ensures Pods are always scheduled
Keeps clusters up to date
Configures update windows for clusters
Users only pay for Pods, not nodes
Autopilot

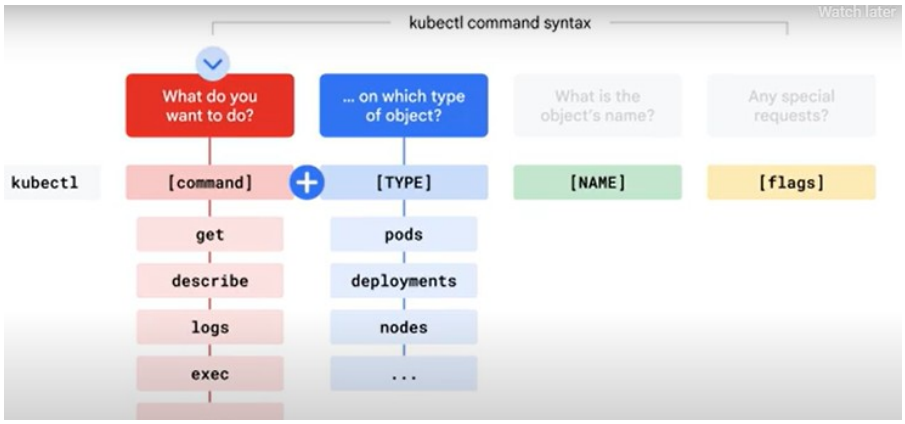

Configuration options are more restrictive
Autopilot clusters have restrictions on access to node objects
No SSH
No privilege escalation
Limitations on node affinity and host access
Autopilot

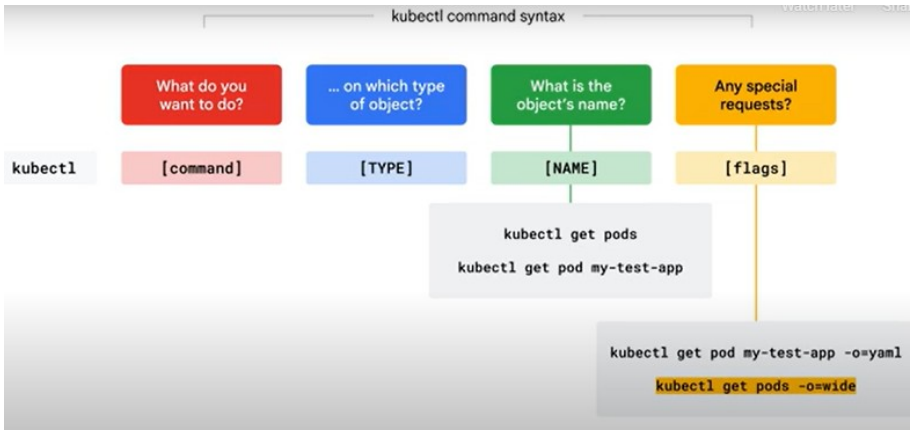
PODS etc.


1
apiVersion: v1
kind: Pod
metadata:
name: nginx
labels:
app: nginx
env: dev
stack: frontend
spec:
containers:
- name: nginx
image: nginx:latest
kubectl

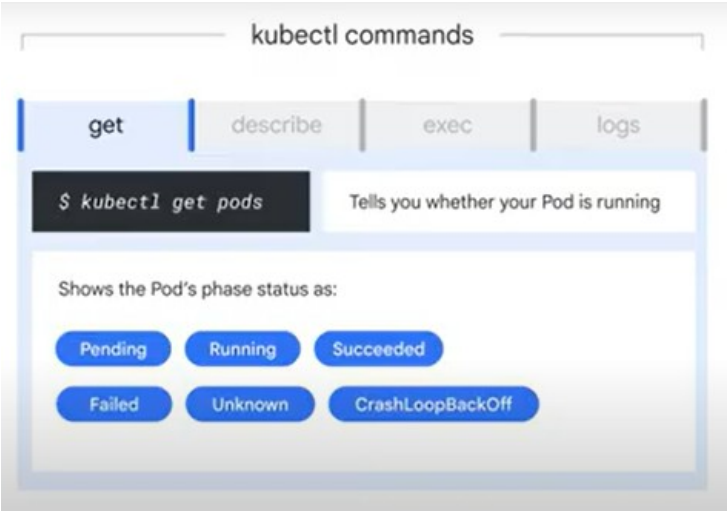

See a list of Pods in a cluster
Admin issues a command
> kubectl get pods
kubectl
API call through HTTPS
Cluster
Control plane
kube-apiserver

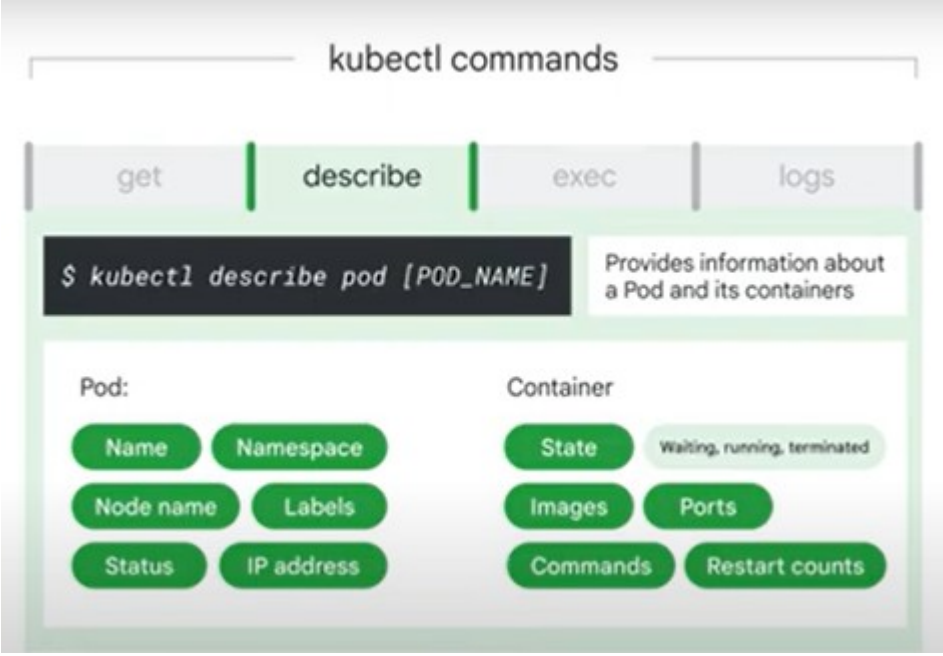

Connect kubectl to a GKE cluster
Writes configuration information into a config file in the .kube directory in the $HOME directory
Only needs to be performed once per cluster in Cloud Shell
$ gcloud container clusters \
get-credentials [CLUSTER_NAME] \
--region [REGION_NAME]

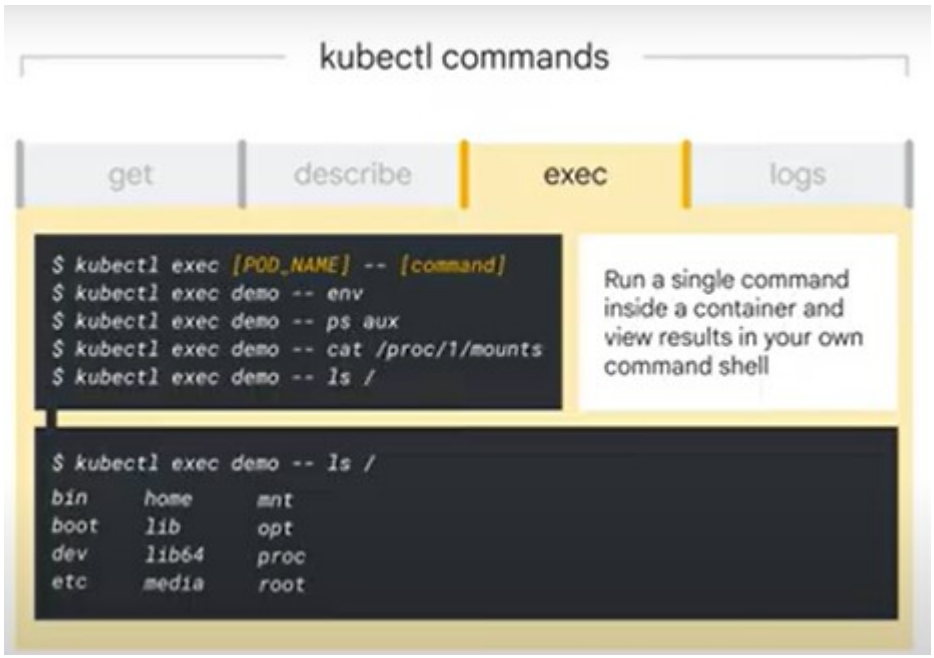

kubectl command syntax
What do you want to do?
... on which type of object?
What is the object's name?
Any special requests?
kubectl
[command]
[TYPE]
[NAME]
[flags]
get
describe
logs
exec
pods
deployments
nodes
...

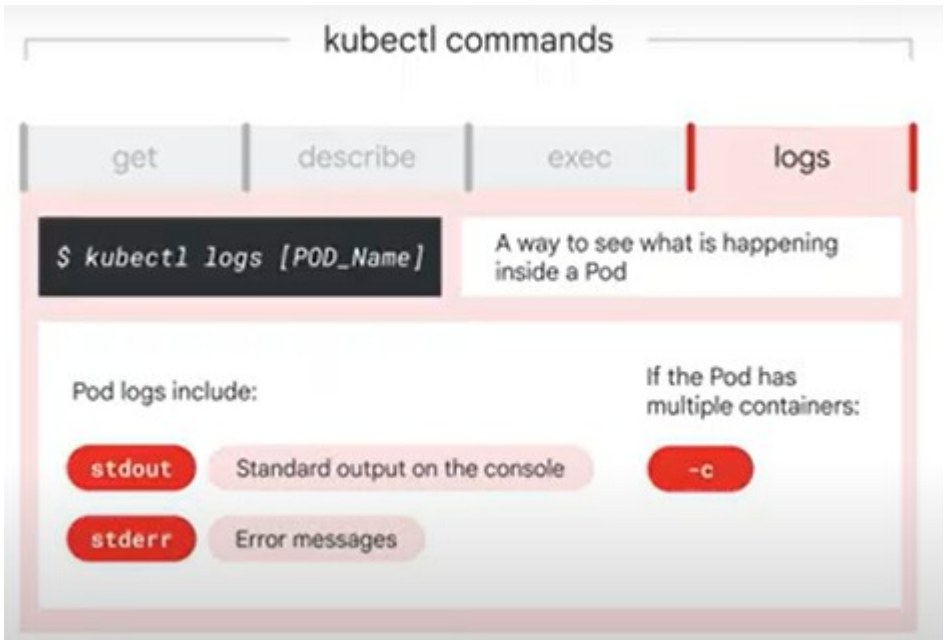

kubectl command syntax
What do you want to do?
... on which type of object?
What is the object's name?
Any special requests?
kubectl
[command]
[TYPE]
[NAME]
[flags]
kubectl get pods
kubectl get pod my-test-app
kubectl get pod my-test-app -o=yaml
kubectl get pods -o=wide

Introspection - how to debug problems when an application is running






kubectl commands
get
describe
exec
logs
$ kubectl exec [POD_NAME] -- [command]
$ kubectl exec demo -- env
$ kubectl exec demo -- ps aux
$ kubectl exec demo -- cat /proc/1/mounts
$ kubectl exec demo -- ls /
Run a single command inside a container and view results in your own command shell
$ kubectl exec demo -- ls /
bin home mnt
boot lib opt
dev lib64 proc
etc media root


kubectl commands
get
describe
exec
logs
$ kubectl logs [POD_Name]
A way to see what is happening inside a Pod
Pod logs include:
stdout
Standard output on the console
stderr
Error messages
If the Pod has multiple containers:
-c

Logging and Monitoring

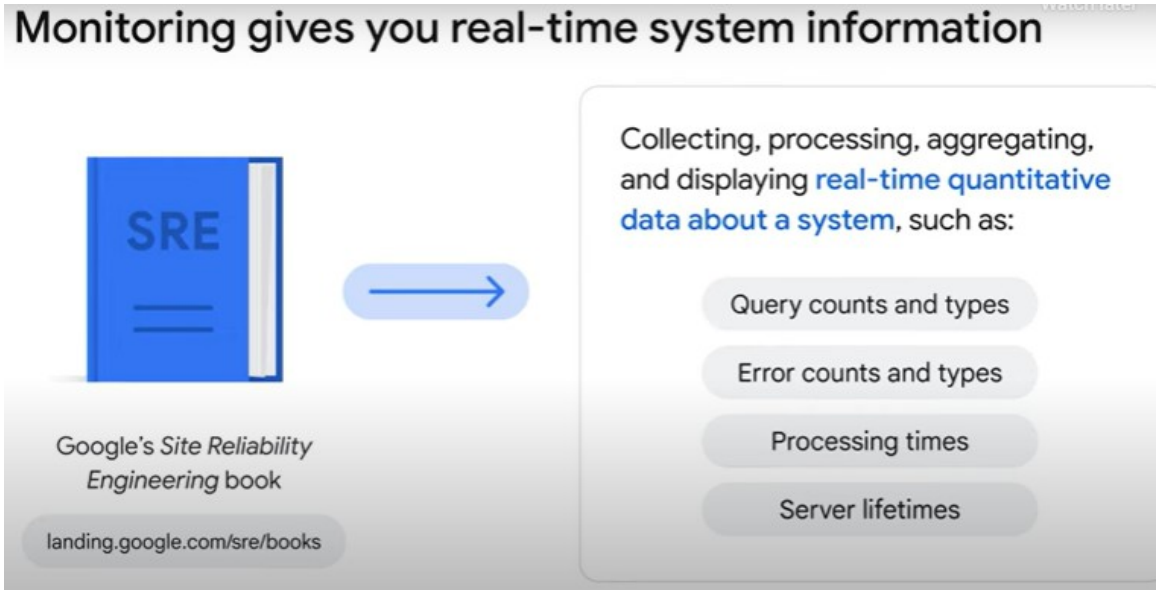
3 parts

- Cloud Logging
- Cloud Monitoring
- APM

Cloud Logging collects and stores all of your application logs

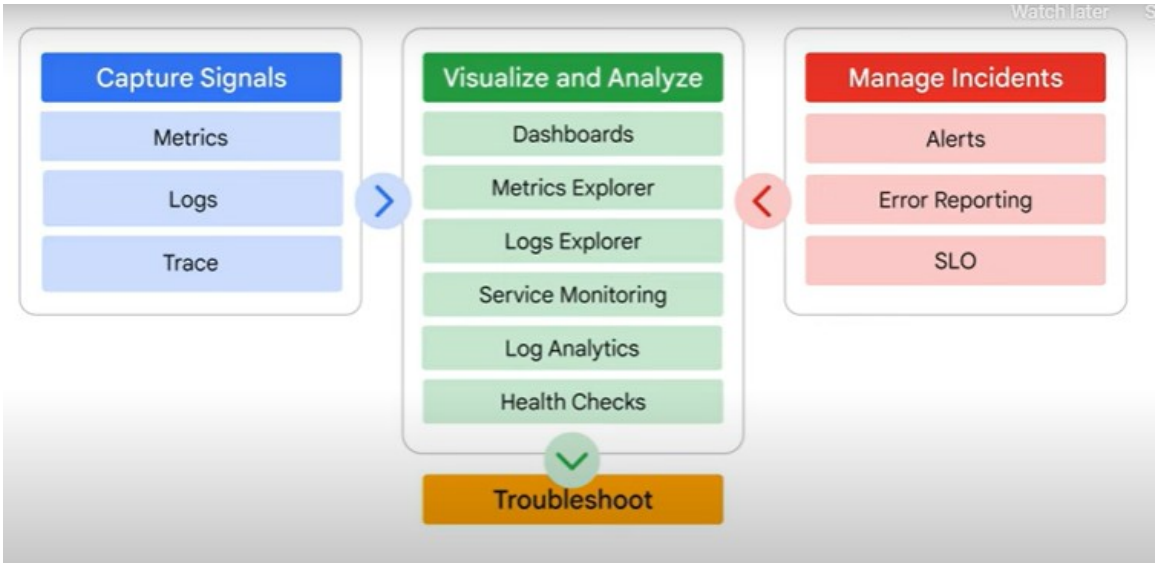
Cloud Monitoring collects metrics and traces -> track applications performance & identify bottlenecks.

APM provides a unified view of your application's performance -> quickly identify & fix problems.



4 signals:

- Latency - measures how long it takes a particular part of a system to return a result (affect UI)
- Traffic - how many requests are reaching your system
- Saturation - indicator of current system demand (how full the service is) -
- Errors

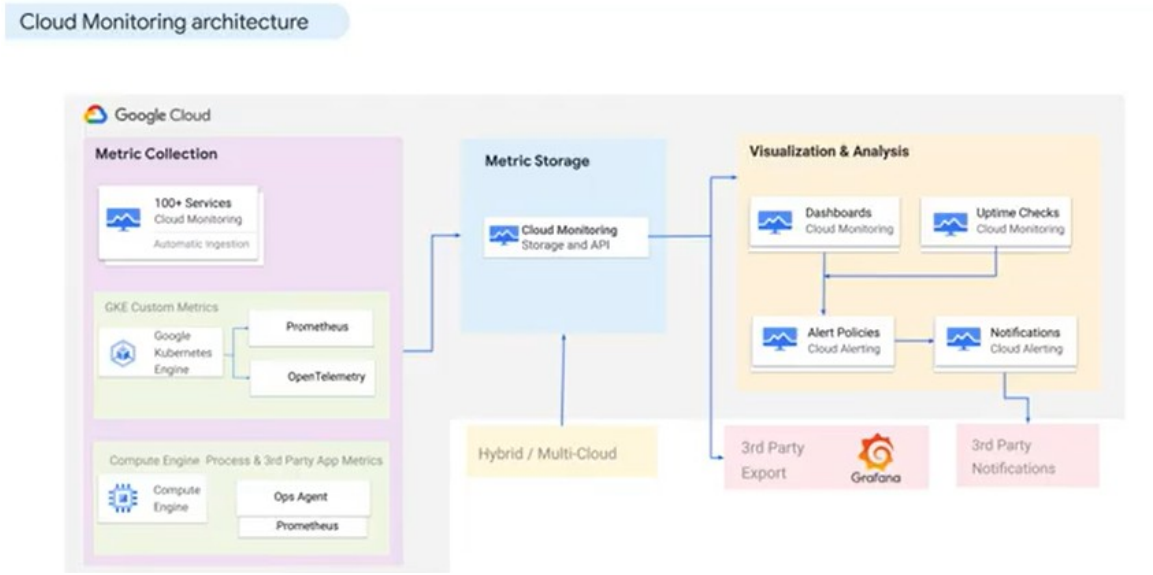


Tools

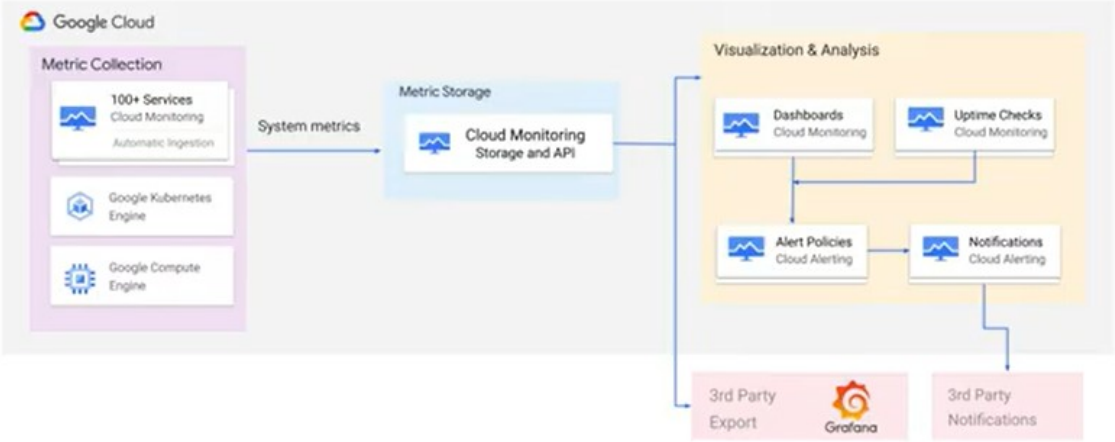
Trace: Best for latency issues and tracking request flows, but not directly for debugging data corruption.

Error Reporting: Detects and aggregates errors, but garbage data may not trigger an exception.

Profiler: Used for performance bottlenecks and CPU/memory profiling, not debugging functional issues.

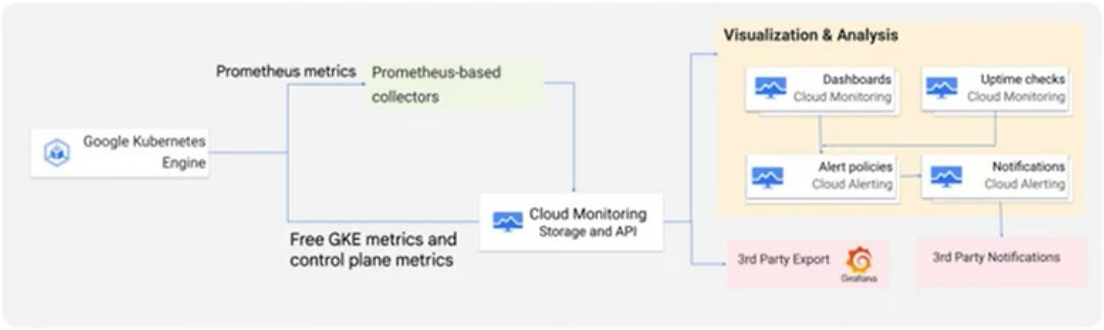


Platform monitoring



Google Cloud Monitoring is the recommended solution for Platform monitoring.

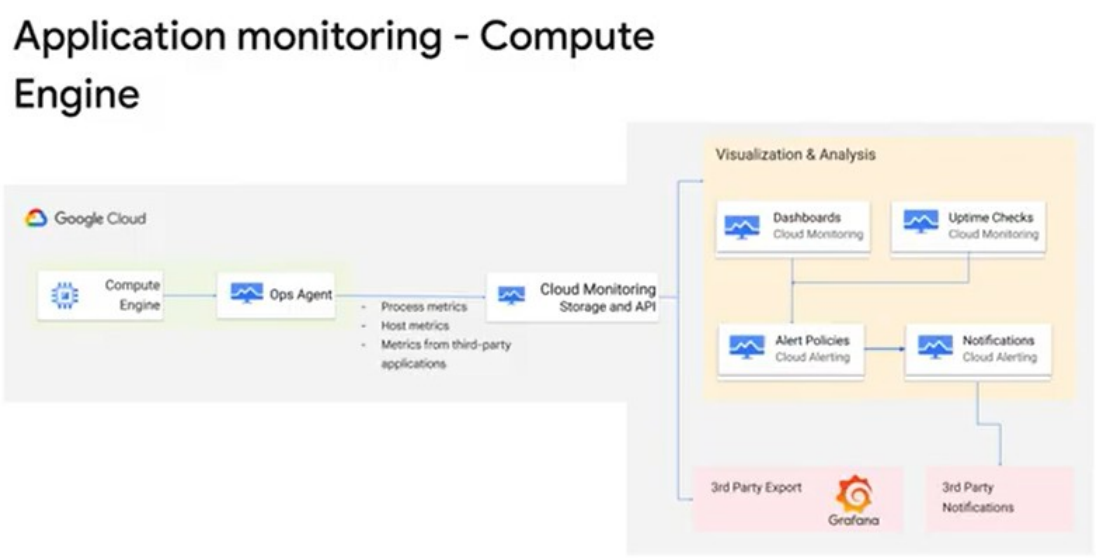
Application monitoring - GKE



GKE includes integration with:

- Cloud Logging and Cloud Monitoring
- Google Cloud Managed Service for Prometheus

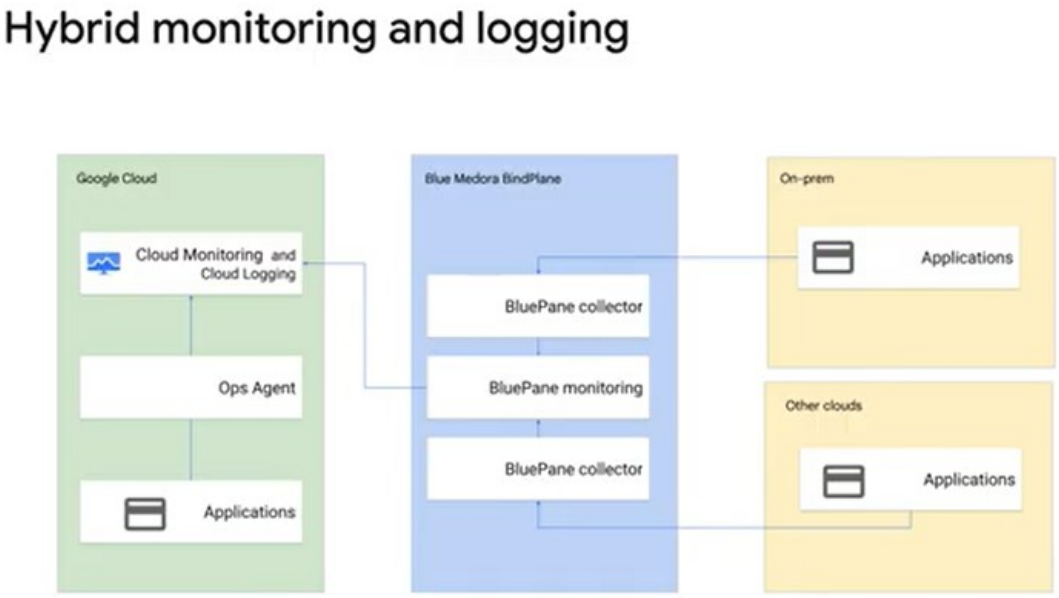
For application workloads in GKE, we recommend that customers use Google Managed Prometheus.



For applications or workloads deployed in Compute Engine, customers should use the Ops Agent

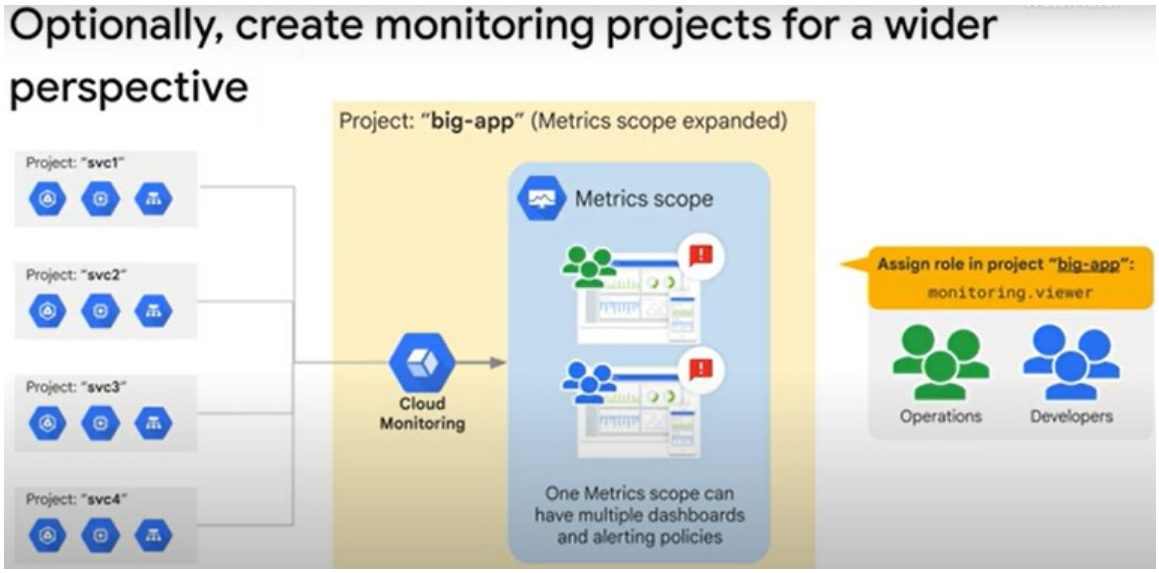
Other 3rd parties like NewRelic OR DataDog

they can collect system metrics using native API-based integrations With Google's partner BindPlane by Blue Medora,



There are no additional licensing costs for using BindPlane

Cloud monitoring



Best practice for production deployments is

create a dedicated project to host monitoring configuration data

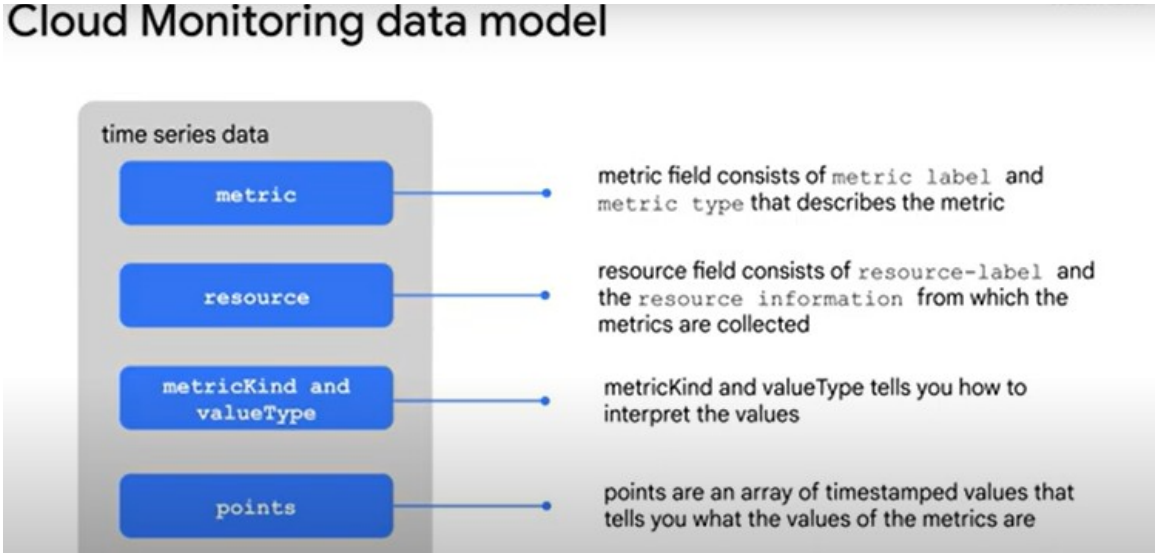
use its metrics scope to set up monitoring for the projects that have resources.

Advantage:

- ⇨ should a project not be necessary anymore and get deleted, the monitoring configuration for all the other projects won't be impacted.
- ⇨ provides a single pane of glass

Disadvantage:

- ⇨ Anyone with IAM permissions to access Cloud Monitoring will be able to see metrics for all environments.
- ⇨ Does not preserve separation of teams in production.



SLI = Service Level Indicator: monitoring metrics that measure one aspect of a service's reliability. - Metric

SLO: combines SLI with a target reliability. (Does it matter to the user?) - Thresholds

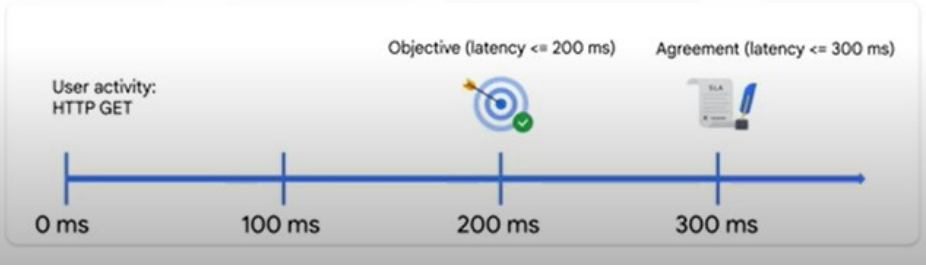
SLA: minimum levels of service that you promise to provide to your customers and what happens when you break that promise. - contract



Error budget = Perfection - SLO

- **SLIs** are the things you measure
- **SLOs** represent an achievable target

If the SLO is "90% of requests must return in 200 ms," then the error budget is 100% - 90% = 10%.



Evaluating alerts

Precision	Recall	Detection time	Reset time
Relevant alerts = Relevant alerts + irrelevant alerts.	Relevant alerts = Relevant alerts + missed alerts.	How long it takes the system to notice an alert condition.	How long alerts fire after an issue is resolved.
Adversely affected by false positives.	Striving for precision might cause events to be missed.	Long detection times can negatively affect the error budget.	Continued alerts on repaired systems can lead to confusion.
	Recall is adversely affected by missed alerts.	Raising alerts too fast may result in poor precision.	

Use multiple conditions for better precision and recall

- Many variables can affect a good alerting strategy:
  - Amount of traffic
  - Error budget
  - Peak and slow periods
- You can define multiple conditions in an alerting policy to try to get better precision, recall, detection time, and rest time.
- You can also define multiple alerts through multiple channels.

Windows-based versus request-based SLOs

- Request-based SLOs = good request : total requests
- Window-based SLO = total number of good request: total number of bad requests

Precision in Alerting - how many of the alerts actually indicate real problems

- **Definition:** Precision measures how many of the alerts **actually indicate real problems** (i.e., true positives).
- **Formula:**

  Precision= True Positives (alerts detected)

  True Positives + False Positives (relevant and irrelevant alerts)

  Impact:

  - **High Precision (Good):** Most alerts are actionable; engineers trust the system.
  - **Low Precision (Bad):** Many false alarms; engineers start ignoring alerts (alert fatigue).

Error Budget

- **Definition:** An error budget is the allowable amount of downtime or failures a service can have within a specific period while still meeting its **Service Level Objectives (SLOs)**.
- **Formula:**

  Error Budget = 1−SLO Target

  If your SLO is **99.9% availability**, the **error budget** is **0.1% downtime per month** (~43.2 minutes).

- **Purpose:**
  - It helps balance reliability and feature development.
  - If the service **exceeds** its error budget, development must slow down until stability improves.
  - If the service is **well within** its error budget, teams can take more risks (e.g., deploy new features).

Burn Rate

- **Definition:** Burn rate measures how quickly your service consumes its error budget.
- **Formula:**

$$\text{Burn Rate} = \frac{\text{Allowed Error Budget Rate}}{\text{Error Rate}}$$

  - **Burn Rate = 1** → Consuming the budget exactly on pace.
  - **Burn Rate > 1** → Consuming the budget too fast.
  - **Burn Rate < 1** → Consuming the budget slowly (safe zone).

Recall - captures actual incidents

measures how well an alerting system **captures actual incidents**, ensuring that real problems are detected. It is defined as:

$$\text{Recall} = \frac{\text{True Positives (alerts detected)}}{\text{True Positives + False Negatives (relevant alerts and missed alerts)}}$$

Where:

- **True Positives (TP):** Correctly detected incidents.
- **False Negatives (FN):** Real incidents that were missed.

Thus, recall focuses on how many **real issues** are caught versus those that go unnoticed.

Precision vs. Recall Trade-off

Metric	Focus	Impact of Low Value	How to Improve
Precision	Avoiding false alerts	Too many **false positives** (alert fatigue)	Adjust **thresholds**, use **multi-condition alerts**
Recall	Catching all real incidents	Too many **false negatives** (missed incidents)	**Lower alert thresholds**, use **redundant monitoring**

- **High Precision, Low Recall** → Fewer alerts, but may miss incidents.
- **High Recall, Low Precision** → Many alerts, but lots of false alarms.

Questions on monitor

eck

1. In the statement “Maintain an error rate of less than 0.3% for the billing system”, what is an SLI?

   Less than 0.3%

   0.3%

   checkError rate - correct

   Billing system

2. In evaluating your alerting policies, which below best describes precision?

   The proportion of events detected that were significant. - Correct

   How long it takes to send notifications in various conditions.

   How long alerts take to be acknowledged.

   How long alerts fire after an issue is resolved.

3. Explain error budget.

   How long alerts fire after an issue is resolved.

   How long it takes to send notifications in various conditions.

   The proportion of alerts detected that were relevant to the sum of relevant and irrelevant alerts.

   The proportion of alerts detected that were relevant to the sum of relevant alerts and missed alerts. - Correct

LOGS

Types

Platform logs –

Example: VPC Flow Logs record a sample of network flows sent from / received by VM's.

Component logs –

Example: GKE provides software components

that users can run on their own VM or in their own data center

Security logs - "who did what, where, and when."

Cloud Audit Logs - information about administrative activities

and accesses within your Google Cloud resource

User-written logs - written to Cloud Logging by using Ops Agent (on the VM)

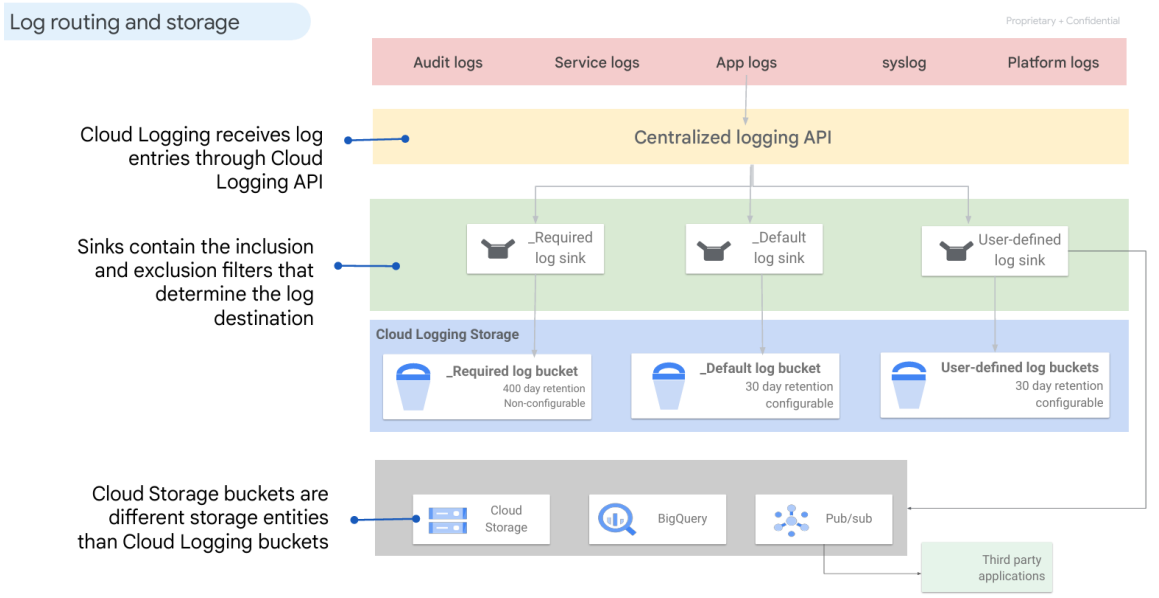
or Cloud Logging API

or Cloud Logging client libraries

Examples:

GKE or Container-Optimized OS, the agents automatically collect logs from stdout and stderr

VMs - the agents collect logs from known file locations or logging services like the Windows Event Log, journald, or syslogd.



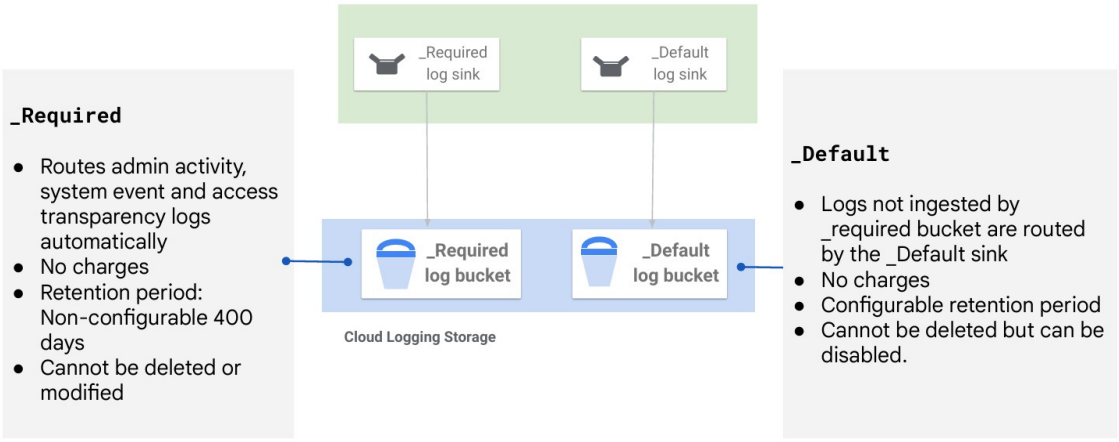
On each gcp project,

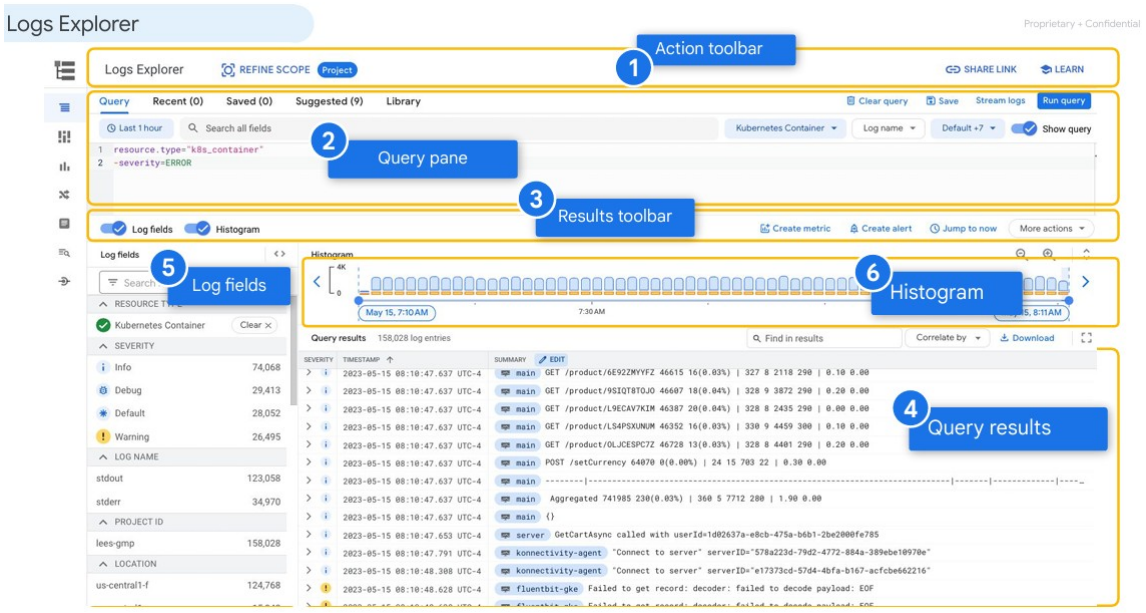
Logging automatically creates two logs buckets:

_Required

_Default,

corresponding log sinks with the same names.





Finding entries quickly

- Search on an indexed field

```
httpRequest.status, logName, operation.id, resource.type, timestamp, severity
```

- Apply constraints on **`resource.type`** and **`resource.labels`** field

```
resource.type = "gke_cluster"
resource.labels.namespace = "my-cool-namespace"
```

- Be specific on which logs you're searching

```
logName="projects/benkelly-test/logs/apache-access"
```

- Limit the time range that you're searching

```
timestamp >= "2018-08-08T10:00:00Z" AND timestamp <= "2018-08-08T10:10:00Z"
```

Logs-metrics

two types of log-based metrics:

- System-defined log-based metrics

  Created by gcp, System-defined log-based metrics are calculated only from logs that have

  been ingested by Logging. If a log has been explicitly

  by Cloud Logging, it isn't included in these metrics.

  excluded from ingestion

- User-defined log-based metrics

  created by you to track things in your Google

  Cloud project that are of particular interest to you.

  Example, log-based metric to count the number of log entries that match a

given filter

Logs-based metrics are suitable in different cases

Count the occurrences	Observe trends in your data	Visualize extracted data
Count the occurrences of a message, like a warning or error, in your logs and receive a notification when the number of occurrences crosses a threshold.	Observe trends in your data, like latency values in your logs, and receive a notification if the values change in an unacceptable way.	Create charts to display the numeric data extracted from your logs.

Key access control roles

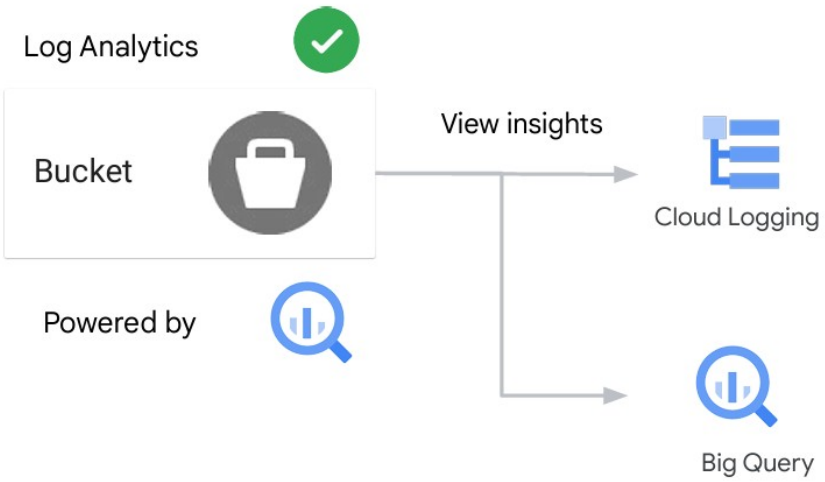
- **Logs Configuration Writer**
  - List, create, get, update, and delete log-based metrics.
- **Logs Viewer**
  - View existing logs.
- **Monitoring Viewer**
  - Read the time series in log-based metrics.
- **Logging Admin, Editor, and Owner**
  - These are broad-level roles that can create log-based metrics.

Log-based metric types



Log Analytics

Perform analytics on log data using Log Analytics



How different is analytics-enabled bucket log data from logs routed to BigQuery

- Log data in BigQuery is managed by Cloud Logging.
- BigQuery ingestion and storage costs are included in your Logging costs.
- Data residency and lifecycle are managed by Cloud Logging.

Log Analytics use cases

DevOps

Reduce MTTR by using advanced analytical capabilities to diagnose issues.

"Help me quickly troubleshoot an issue by looking at the top count of requests grouped by response type and severity".

Security

Better investigate security-related attacks with queries over large volumes of security logs.

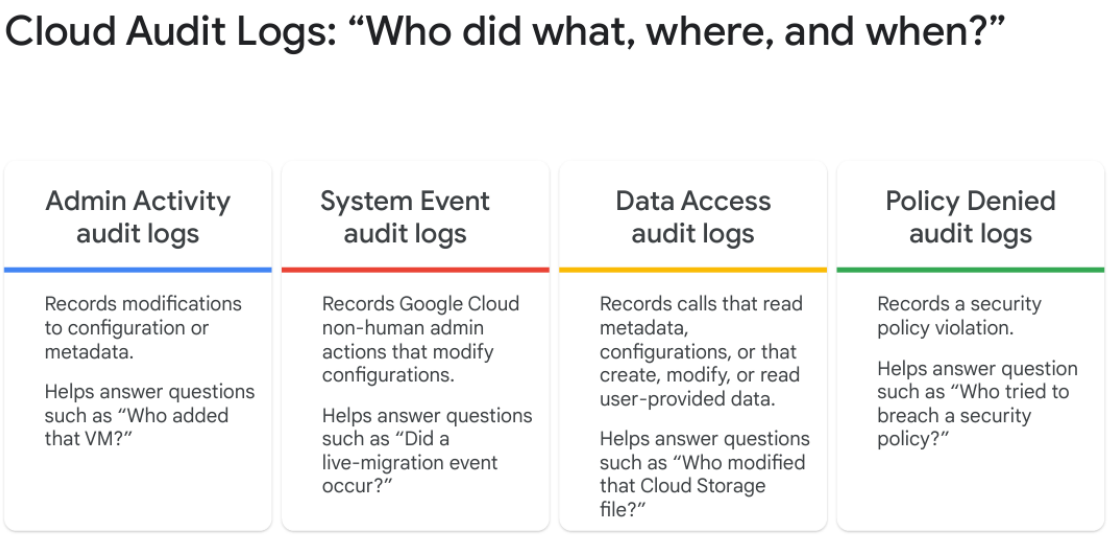
"Help me find the all the audit logs associated with a specific user over the past month".

IT or Network Operations

Provide Better network insight and management through advanced log aggregation capabilities

"Help me identify network issues for GKE instances using VPC and firewall rules".

Audit Logs - Who did what, where, and when



Admin:

log entries for API calls or other administrative actions

retained for 400 days, no charge

IAM role: Logging/Logs Viewer or Project/Viewer.

Example: create VM

System Event audit logs:

Google Cloud administrative actions that modify the configuration of resources.

enerated by Google systems; they are **NOT** driven by direct user action.

always on, retained for 400 days, no charge.

IAM roles: Logging/Logs Viewer or Project/Viewer.

Data Access audit logs:

API calls that read the configuration or metadata of resources.

user-driven API calls that create, modify, or read user-provided resource data.

don’t record the data-access operations on resources that are publicly shared

don’t record the data-access operations on resources that can be accessed without logging into Google Cloud.

disabled by default (except for BigQuery),

when enabled, default retention is 30 days.

IAM roles Logging/Private Logs Viewer or Project/Owner.

Policy Denies Audit logs:

When a security policy is violated,

Records when access to a user or service account is denied by Google Cloud service

Generated by default, Google Cloud project is charged for the logs storage.

can't disable Policy Denied audit logs.

can use exclusion filters to prevent these logs from being ingested and stored in Cloud Logging

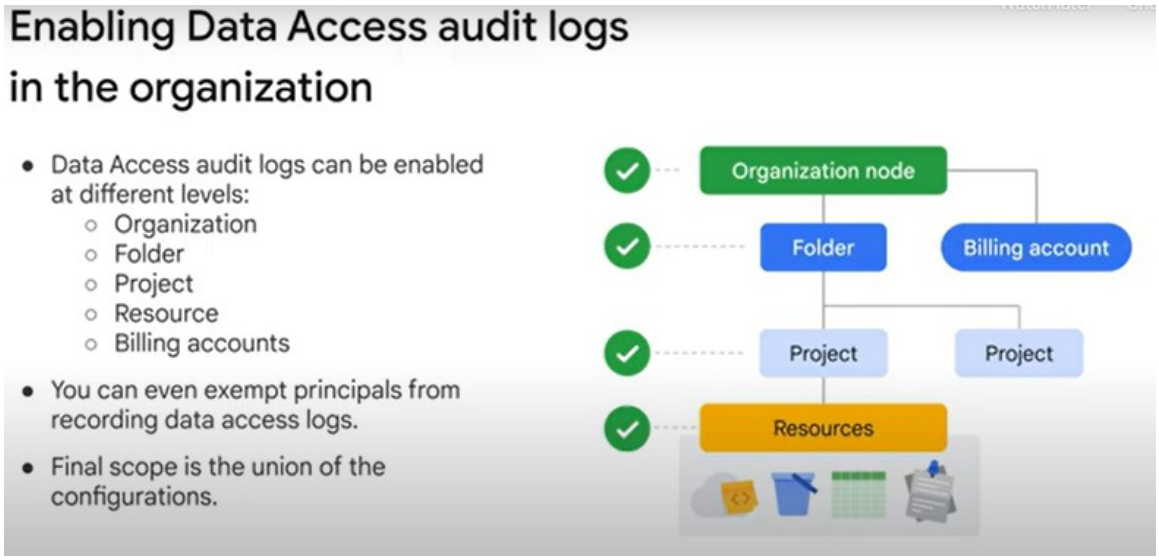
Access Transparency logs

Very similar to Cloud Audit logs

Providing logs of accesses to your data by human Googlers (as opposed to automated systems).

Enterprises with appropriate support packages can enable the logs, and receive the log events in near-real time.

Data Access audit logs



The added logging does add to the cost, currently: $0.50 per gigabyte for ingestion.

Observability

Configuring Google Cloud Services for Observability

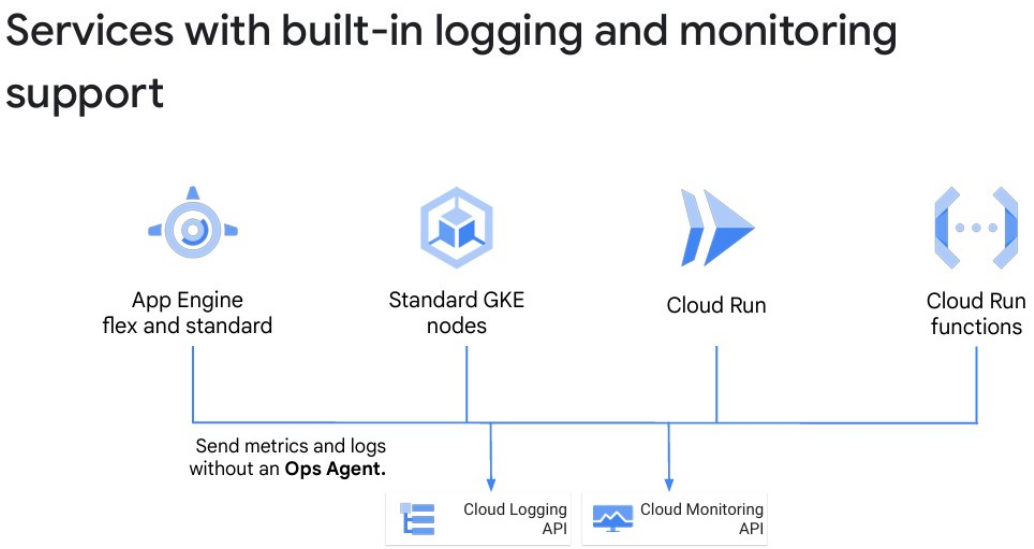
Compute service - VM's -> install ops agent

Why OPS Agent?

The hyper visor (Hyper-V) cannot tell the memory usage inside each vm, just the memory allocated on the Hyper-V itself

Can be installed using cli/ policy/ automated tools (ansible / puppet/ chef)

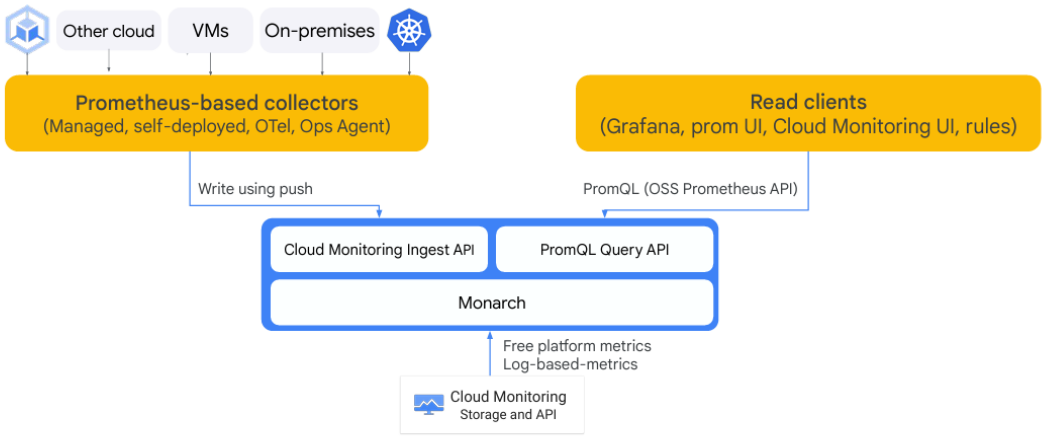
Non VM resources - Server less services – automatic monitoring by default



GKE monitoring using Prometheus

Based on Monarch

The Managed Service for Prometheus ecosystem



Data collection options

Managed data collection	Self-deployed collection	The Ops Agent	OpenTelemetry
• Managed data collection in Kubernetes environments • Recommended approach for all Kubernetes environments	• Prometheus installation managed by the user • Suitable for quick integrations into complex environment	• Prometheus metrics scraped and sent by the Ops Agent • Recommended for sending Prometheus metric data	• Deployed in any compute or Kubernetes environment • Recommended for cross-signal workflows such as exemplars

Managed Data Collection refers to using **cloud-native services** to automatically **collect, process, and store** telemetry data (logs, metrics, traces) from **Kubernetes clusters (GKE or self-managed)** without requiring manual setup.

In **Google Cloud**, **Managed Data Collection** is provided by **Cloud Operations (formerly Stackdriver)** and can be **integrated with GKE out of the box**.

Eventually the cheapest !! (better than get Prometheus on 2 dedicates servers (~ 400 a month)

Monitoring Network

VPC Flow Logs

Packet Mirroring

Packet Mirroring consumes additional bandwidth on the hosts.

Packet Mirroring is useful when you need to monitor and analyze your security status.

Good for security and monitoring.

Comparison of GCP Network Monitoring Logs

Feature	VPC Flow Logs	Packet Mirroring	Firewall Rules Logs	Load Balancer Logs	Cloud NAT Logs
Purpose	Monitors network flows (traffic between VMs, GKE nodes, external sources)	Captures full packets for deep inspection	Tracks allow/deny actions of firewall rules	Logs requests handled by GCP Load Balancers	Monitors traffic using Cloud NAT
Traffic Type	Metadata-only (src/dst IP, bytes, protocol)	Full packet capture (deep packet inspection)	Firewall rule evaluation results	HTTP(S), TCP, UDP, SSL traffic through load balancers	Outbound traffic through Cloud NAT
Use Cases	Performance monitoring, troubleshooting, anomaly detection	Advanced security analysis, forensics, intrusion detection	Debugging dropped connections, rule validation	Analyzing load balancer traffic, debugging performance issues	Monitoring outbound NAT usage, troubleshooting connectivity
Logs Storage	**Cloud Logging**, BigQuery, Pub/Sub	Sent to a **packet analysis system (e.g., IDS, SIEM)**	**Cloud Logging**	**Cloud Logging**	**Cloud Logging**
Visibility	Per VPC network interface	Per **VM NIC** (Network Interface Card)	Per firewall rule action	Per load balancer request	Per NAT gateway instance
Performance Impact	**Low impact** (sampling available)	**High impact** (copies all packets) affect bandwidth	**Low impact** (logs only allow/deny events)	**Moderate impact** (depends on traffic volume)	**Low impact** (logs NAT activity only)

Feature	VPC Flow Logs	Packet Mirroring	Firewall Rules Logs	Load Balancer Logs	Cloud NAT Logs
Cost Considerations	**Moderate cost** (storage cost for logs)	**High cost** (full packet capture, high data volume)	**Low cost** (firewall logs are lightweight)	**Moderate cost** (depends on request volume)	**Low cost** (small amount of log data)

When to Use Each?

Scenario	Recommended Log Type
Monitor network traffic patterns, performance	**VPC Flow Logs**
Investigate **security threats, malware**, advanced troubleshooting	**Packet Mirroring**
Identify **blocked or allowed traffic** due to firewall rules	**Firewall Rules Logs**
Analyze **load balancer performance, traffic distribution**	**Load Balancer Logs**
Troubleshoot **Cloud NAT outbound traffic** issues	**Cloud NAT Logs**

Key Takeaways

- **VPC Flow Logs** are best for general **network monitoring** and troubleshooting.
- **Packet Mirroring** is for **deep packet inspection** and security forensics (**high cost**).
- **Firewall Rules Logs** help debug **access issues due to firewall rules**.
- **Load Balancer Logs** provide insight into **traffic served through GCP Load Balancers**.
- **Cloud NAT Logs** help analyze **outbound internet traffic** from private VMs.

APM - Application Performance Management

Tools

- Error Reporting
- Cloud Trace
- Cloud Profiler

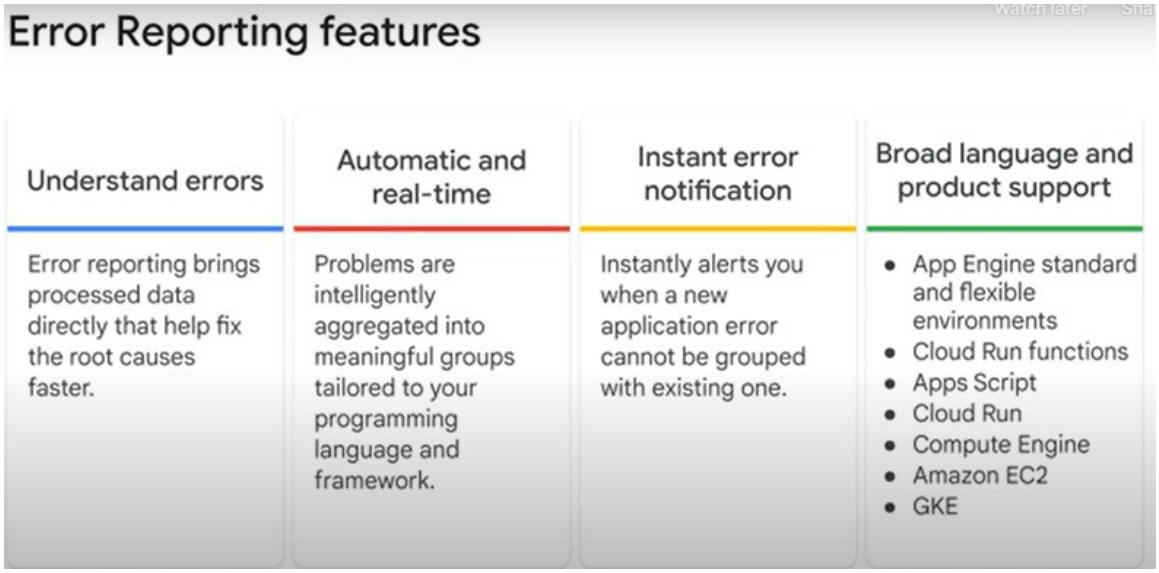
Feature	Error Reporting	Cloud Trace	Cloud Profiler
Purpose	Detects, analyzes, and aggregates **application errors**	Tracks **request latency** across microservices	Measures **CPU and memory usage** to optimize performance
Best For	**Debugging application errors** (e.g., exceptions, crashes)	**Analyzing request latency** in microservices	**Optimizing resource usage** in production
Data Captured	**Stack traces** of errors from logs	**End-to-end request latency** data	**CPU and memory profiling** per function
Use Case	**Find frequent application errors, debug stack traces**	**Identify slow API calls**, bottlenecks, and dependencies	**Optimize hot functions** consuming CPU or memory
Integration	**Works with Cloud Logging**, GKE, App Engine, Cloud Run, Cloud Functions	**Works with GKE, App Engine, Cloud Run, Compute Engine**	**Works with GKE, App Engine, Compute Engine**
Performance Impact	**Minimal** (only processes logs with errors)	**Minimal** (sampling-based tracing)	**Low impact** (sampling-based profiling)
Retention Period	Depends on Cloud Logging storage	30 days by default	30 days by default
Visualization	Cloud Console UI	Latency charts, heatmaps	Call graphs, flame charts
Cost	**Free for basic use** (pricing based on Cloud Logging ingestion)	**Free up to 2.5M spans/month**, then **$0.20 per million spans**	**Free up to 5MiB/s**, then **usage-based pricing**

When to Use Each?

Scenario	Recommended Tool
Debugging **application crashes, stack traces, or common errors**	✅ **Error Reporting**
Analyzing API response time, microservices performance bottlenecks	✅ **Cloud Trace**
Optimizing resource consumption, CPU/memory profiling	✅ **Cloud Profiler**

Key Takeaways

- **Use Error Reporting** for **detecting and debugging application errors**.
- **Use Cloud Trace** to **analyze request latency** and find **slow services**.
- **Use Cloud Profiler** to **reduce CPU/memory overhead** and improve performance.



For App Engine flexible environment and standard environment, Cloud Run, Cloud Run functions, and Apps Script, Error Reporting is automatically enabled.

For Google Kubernetes Engine, add cloud-platform access scope during cluster creation.

For Compute Engine, ensure the service account used has the Error Reporting Writer role.

Outside Google Cloud, provide the Google Cloud project ID and service account credentials to the Error Reporting library for Node.js.

Questions

1. You deployed a new version of a service and all of a sudden significantly more instances are being created in your Kubernetes cluster. Your service scales when average CPU utilization is greater than 70%. What tool can help you investigate the problem?

   Trace

Logging - correct

Profiler

Error Reporting

2. You have an SLO that states that 90% of your http requests need to respond in less than 100 ms. You want a report that compares latency for your last two versions. What tool would you use to most easily create this report? (why not trace?)

   Profiler

   Error Reporting

   Logging - correct

   Trace

3. Which profile signifies the amount of memory allocated in the heap of the program?

   Wall time

   Allocated heap (allocated + freed – this is why it is not correct!)

   CPU time

   Heap – correct

Optimizing the Costs for Google Cloud Observability

Logging incurs cost when using:

- Cloud Load Balancing logs
- Custom logs
- Error reporting costs (if your errors are ingested by Cloud Logging)
- The write operation in the Cloud Logging API
- Logs stored beyond 30 days will incur a retention charge for

  non-required buckets.

Questions

1. What is Cloud Logging pricing based on?

   Read operation in the Cloud Logging API

   Profiler Usage

Number of uptime checks

Volume of chargeable logs - Correct

2. What is one of the best practices to reduce monitoring costs?

Reduce Ops Agents usage - Correct

Increasing custom metric usage.

Increase label usage

Reduce the number of time series

IAP - Identity-Aware Proxy (IAP)?

IAP is a **Google Cloud security feature** that allows secure access to applications and resources (like Compute Engine VMs) **without requiring a VPN or exposing external IPs**. It enforces identity-based access control by verifying users' identities and permissions before allowing a connection.

How IAP Works

- **Acts as a gateway**: It sits between the user and the resource (like a VM) and ensures only authorized users can access it.
- **Uses IAM policies**: Access is controlled via **Google IAM roles and permissions**.
- **Avoids public IP exposure**: You don't need to assign an external IP to a VM; IAP tunnels the connection securely over **Google's private network**.

Use Cases of IAP

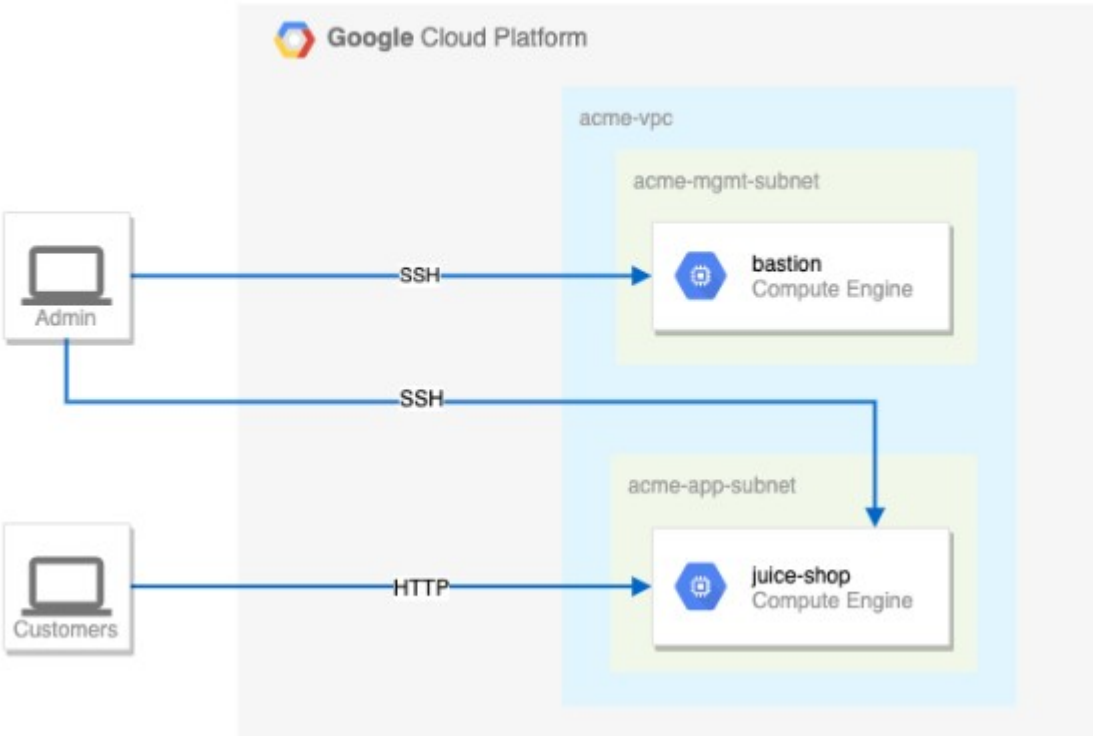
✅ **Secure VM Access** (Linux SSH, Windows RDP)
✅ **Protect Web Applications** (without requiring VPN)
✅ **Enforce Zero Trust Security** (Google's BeyondCorp model)
✅ **Granular Access Control** via IAM policies

Bastion

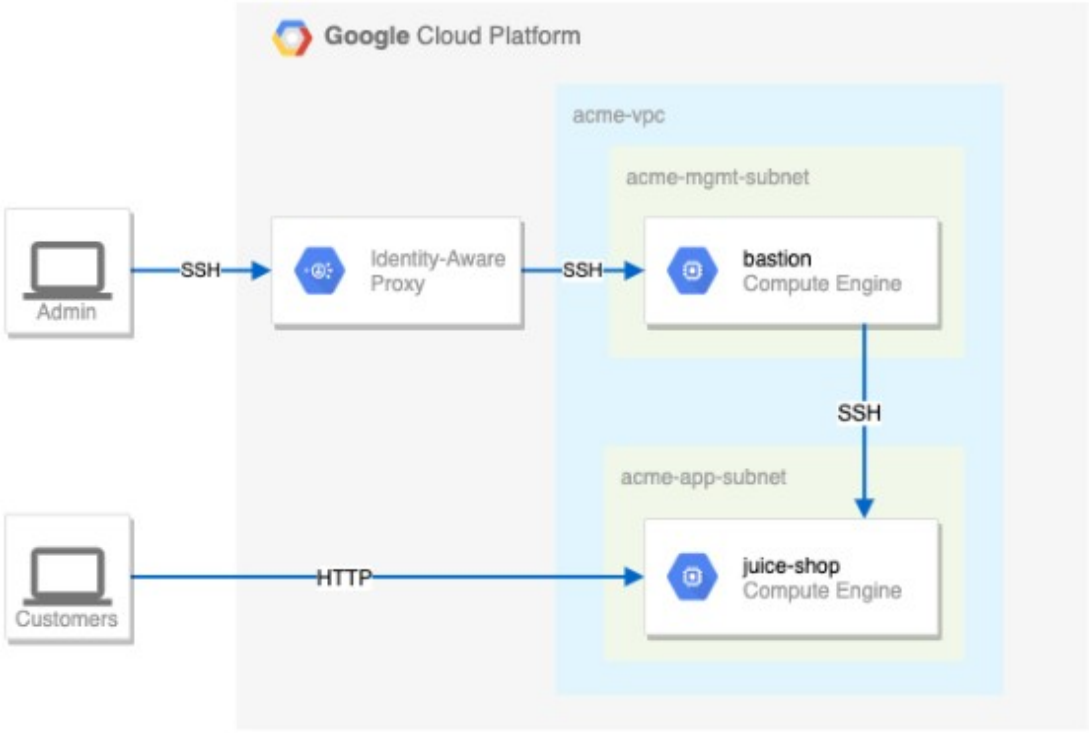
Lab url: https://www.cloudskillsboost.google/course_templates/654/labs/464661

From this:

Below is the current set up:

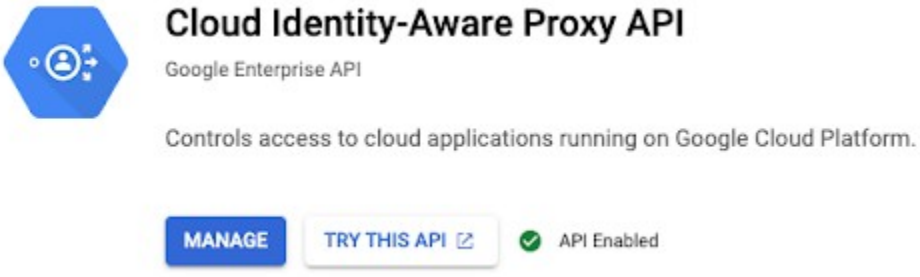


To this:



Steps:

1. Check the firewall rules. Remove the overly permissive rules.
    a. Remove firewall rule on ssh from all addresses (0.0.0.0.0/0) on port22 to bastion and juice-shop servers (network tag ? name ?)
2. Create a firewall rule that allows SSH (tcp/22) from the IAP service.
    a. Enable IAP TCP forwarding in your Google Cloud project
    b. Open the Navigation Menu and select APIs and Services > Library.
    c. Search for IAP and select Cloud Identity-Aware Proxy API.
    d. Click Enable.



    e. gcloud compute firewall-rules create allow-ssh-from-iap \

--network=default \

```
--direction=INGRESS \
--action=ALLOW \
--rules=tcp:22 \
--source-ranges=35.235.240.0/20 \
--target-tags=ssh-iap
```

3.